

Software project management is the process of planning, organizing, executing, monitoring and controlling software projects. There are different types of diagrams that can be used to represent various aspects of software project management, such as:

- Project planning diagram: This diagram shows the scope, objectives, deliverables, milestones, tasks and dependencies of a software project. It can be drawn as a tree structure, a network diagram or a Gantt chart.
- Project time scheduling diagram: This diagram shows the duration, start and end dates, dependencies and critical path of the tasks in a software project. It can be drawn as a Gantt chart, a PERT chart or a CPM chart.
- Project cost control and budget management diagram: This diagram shows the estimated and actual costs, budget allocation, variance analysis and earned value analysis of a software project. It can be drawn as a table, a pie chart, a histogram or a S-curve.
- Project resource allocation chart: This diagram shows the human and material resources, their availability, utilization and allocation for the tasks in a software project. It can be drawn as a table, a matrix, a bar chart or a resource leveling chart.
- Project timeline diagram: This diagram shows the major events, milestones, deliverables and deadlines of a software project. It can be drawn as a timeline, a milestone chart or a roadmap.

Here is an example of a software project management diagram that shows the project planning, time scheduling and resource allocation for a software project:

Software Project Management

Project Planning	Time Scheduling	Resource Allocation	
Scope:	Duration:	Human Resources:	
- Develop a web app for online shopping	- 6 months	- Project manager	
- Features:	Start Date:	- Web developer	
- User registration	- 01/04/2023	- UI designer	
- Product browsing	End Date:	- Tester	
- Shopping cart	- 30/09/2023	Material Resources:	
- Payment gateway		- Laptop	
- Order tracking		- Software tools	
- Deliverables:		- Server	
- Web app			
- User manual			
- Test report			
Objectives:	Tasks:	Availability:	

- Increase sales	- Task 1:	- Project manager:	
- Improve customer	User registration	8 hours/day	
satisfaction	- Task 2:	- Web developer:	
- Reduce costs	Product browsing	8 hours/day	
	- Task 3:	- UI designer:	
	Shopping cart	6 hours/day	
	- Task 4:	- Tester:	
	Payment gateway	8 hours/day	
	- Task 5:	- Laptop:	
	Order tracking	4 units	
	- Task 6:	- Software tools:	
	Web app testing	4 licenses	
	- Task 7:	- Server:	
	User manual	1 unit	
	- Task 8:		
	Test report		

Milestones:	Dependencies:	Utilization:	
- M1:	- Task 1 depends on	- Project manager:	
User registration	none	100%	
completed	- Task 2 depends on	- Web developer:	

The following is a detailed ASCII diagram for Unit - 1 - Project Evaluation and Project Planning for the notes of the course Software Project Management.

The diagram shows the four phases of the project management lifecycle: initiating, planning, executing, and closing. It also shows the main activities and deliverables for each phase, as well as the evaluation plan that guides the project monitoring and assessment.

The evaluation plan consists of four elements: evaluation purpose, evaluation questions, data collection methods, and data analysis methods. The evaluation plan should be aligned with the project goals and objectives, and should be developed during the planning phase of the project.

The diagram uses the following symbols:

-
- ->: an arrow represents a transition or a dependency
- /: a slash represents a parallel or concurrent process
- (): a parenthesis represents a deliverable or an output
- {}: a brace represents a group or a category

The diagram is as follows:

```
[Initiating] -> [Planning] / [Executing] -> [Closing]
[Initiating]: Define the project scope, objectives, and stakeholders
```

```

        (Project charter) -> [Planning]
[Planning]: Develop the project plan, including the evaluation plan
        (Project plan) / (Evaluation plan) -> [Executing]
[Executing]: Implement the project activities and monitor the project progress
        (Project outputs) / (Monitoring data) -> [Closing]
[Closing]: Finalize the project and evaluate the project outcomes and impacts
        (Project closure) / (Evaluation report) -> (Lessons learned)

{Evaluation plan}: Define the purpose, questions, methods, and analysis of the project evaluation
[Evaluation purpose]: Why is the project being evaluated? Who are the interested parties?
[Evaluation questions]: What are the key questions to be answered by the evaluation?
[Data collection methods]: How will the data be collected to answer the evaluation questions?
[Data analysis methods]: How will the data be analyzed and interpreted to answer the evaluation questions?

```

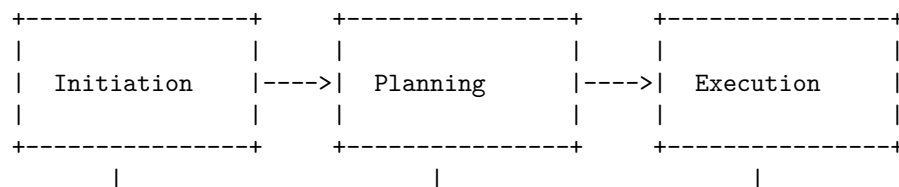
Importance of Software Project Management

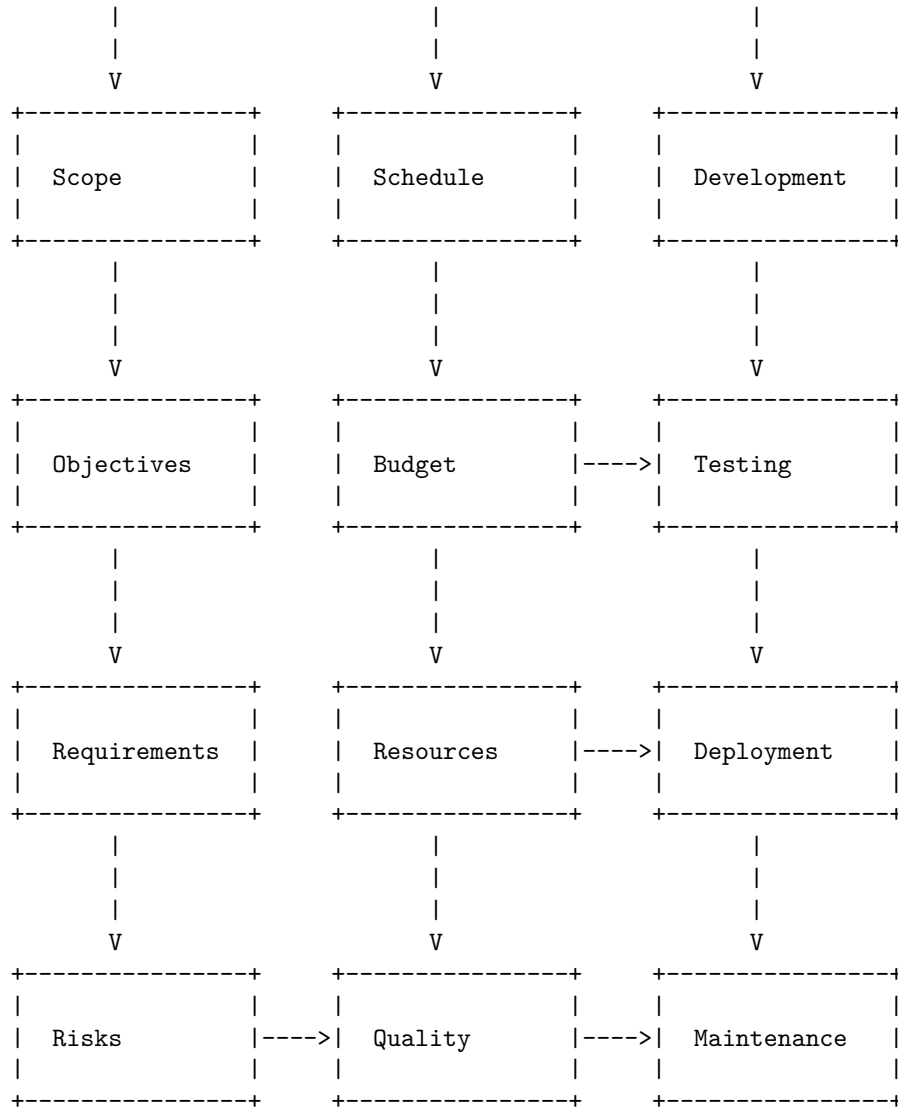
Software project management is the process of planning, organizing, executing, monitoring and controlling software development projects. It involves various activities such as defining the scope, setting the objectives, allocating the resources, managing the risks, tracking the progress, ensuring the quality and delivering the product.

Software project management is important because it helps to:

- Achieve the strategic goals of the organization or the client by aligning the software project with their vision and mission.
- Ensure the satisfaction of the stakeholders by meeting their expectations and requirements.
- Optimize the use of time, money, human and technical resources by applying the best practices and methodologies.
- Minimize the uncertainties and risks by identifying, analyzing and mitigating them in advance.
- Enhance the communication and collaboration among the project team and other parties involved by using the appropriate tools and techniques.
- Improve the quality and performance of the software product by following the standards and guidelines.
- Deliver the software product on time, within budget and with the desired features and functions.

The following diagram illustrates the basic architecture of a software project management process:





Each phase and activity has its own inputs, outputs, tools and techniques that are applied according to the specific needs and characteristics of the software project. Software project management is a dynamic and iterative process that requires constant monitoring and controlling to ensure the success of the project.

According to the search results, software project management consists of many activities that include planning, tracking, resource management, scheduling, communication, estimation, risk management, and configuration management . These activities are typically performed by a software project manager who oversees the entire project lifecycle from inception to completion .

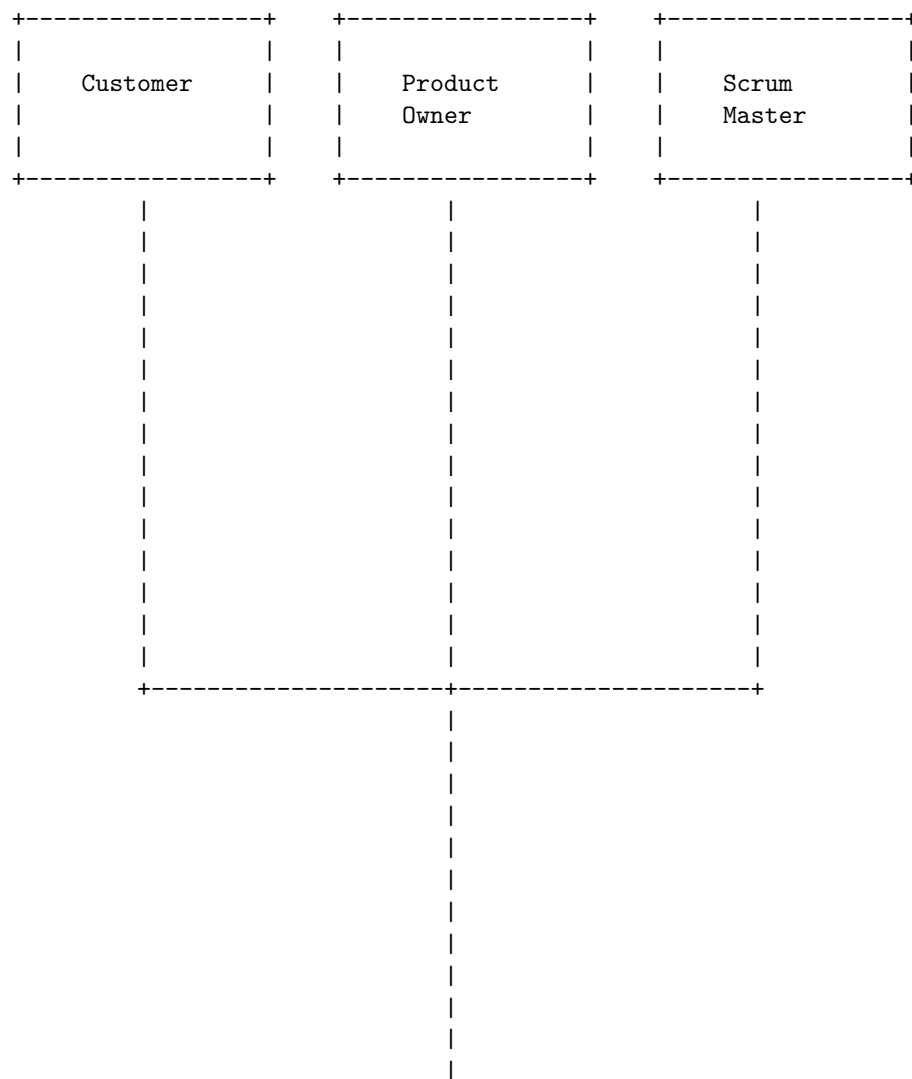
Project Initiation	Project Planning	Project Execution
V	V	V
Project Monitoring	Project Control	Project Closure

Methodologies in Software Project Management are a set of processes that guide the planning, execution and control of a project. There are many different methodologies that can be used depending on the nature, scope, complexity and goals of the project. Some of the most popular methodologies are:

Agile

Agile is a flexible and iterative approach that delivers value to the customer in small increments. Agile teams work in short cycles called sprints, where they plan, design, develop, test and deliver a potentially shippable product or feature. Agile teams are self-organizing, cross-functional and collaborative, and they use feedback and adaptation to improve their processes and products. Agile is suitable for projects that have changing requirements, high uncertainty and fast delivery.

The following diagram illustrates the basic structure of an agile team and the flow of work:



There are different ways of categorizing software projects based on various factors such as development model, platform, scope, user interaction, maturity level, open-source software participation, delivery method, etc. One possible way of categorizing software projects in software project management (SPM) is based on the types of applications they create, the types of platforms they support, and the size and complexity of the projects. Common categories of software projects based on this approach are:

- **Web Applications:** These are software projects that create applications that run on web browsers and servers. They typically use web technologies such as HTML, CSS, JavaScript, PHP, etc. and communicate with databases and web services. Examples of web applications are online shopping, social media, e-learning, etc.
- **Mobile Applications:** These are software projects that create applications that run on mobile devices such as smartphones and tablets. They typically use mobile platforms such as Android, iOS, Windows Phone, etc. and may access device features such as camera, GPS, sensors, etc. Examples of mobile applications are games, fitness trackers, navigation, etc.
- **Desktop Applications:** These are software projects that create applications that run on personal computers and laptops. They typically use desktop platforms such as Windows, Linux, Mac OS, etc. and may interact with local files, hardware, and network. Examples of desktop applications are word processors, spreadsheets, photo editors, etc.
- **Cloud-Based Solutions:** These are software projects that create applications that run on cloud platforms such as Amazon Web Services, Google Cloud Platform, Microsoft Azure, etc. They typically use cloud services such as storage, computing, database, analytics, etc. and scale according to the demand and resources. Examples of cloud-based solutions are email, online backup, video streaming, etc.
- **Embedded Systems:** These are software projects that create applications that run on embedded devices such as microcontrollers, sensors, actuators, etc. They typically use low-level languages such as C, assembly, etc. and have limited memory and processing power. Examples of embedded systems are smart watches, robots, drones, etc.

- **Data Science:** These are software projects that create applications that analyze, visualize, and derive insights from large and complex data sets. They typically use data science tools such as Python, R, SQL, etc. and apply machine learning, statistics, and artificial intelligence techniques. Examples of data science applications are recommender systems, fraud detection, natural language processing, etc.

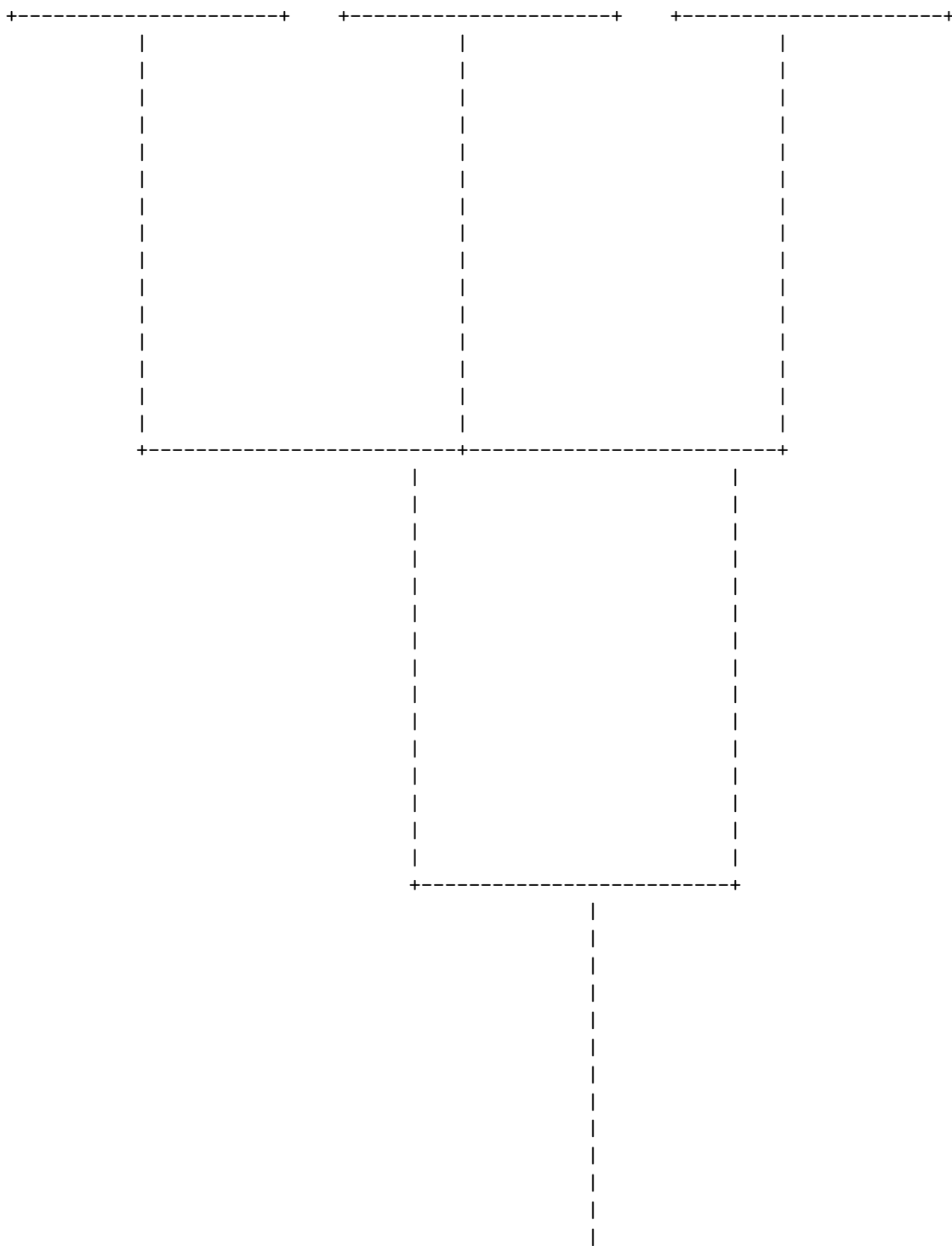
The following diagram illustrates the basic categorization of software projects in SPM:

Web Applications	Mobile Applications	Desktop Applications
Online shopping	Games	Word processors
Social media	Fitness trackers	Spreadsheets
E-learning	Navigation	Photo editors
Cloud-Based Solutions	Embedded Systems	Data Science
Email	Smart watches	Recommender systems
Online backup	Robots	Fraud detection
Video streaming	Drones	Natural language processing

Setting objectives in SPM (Strategic Portfolio Management) is the process of defining and aligning the goals of a project or a portfolio with the business strategy and priorities. Setting objectives helps to clarify the expectations, focus, motivation and engagement of the project team and stakeholders. It also helps to measure and track the progress and value of the project or portfolio.

One possible way to draw a detailed ASCII diagram for setting objectives in SPM is:

Business Strategy	Project Portfolio	Project Objectives
The high-level vision, mission, goals and values of the organization	The collection of projects that support the business strategy	The specific, measurable, achievable, relevant and time-bound (SMART) outcomes of each project



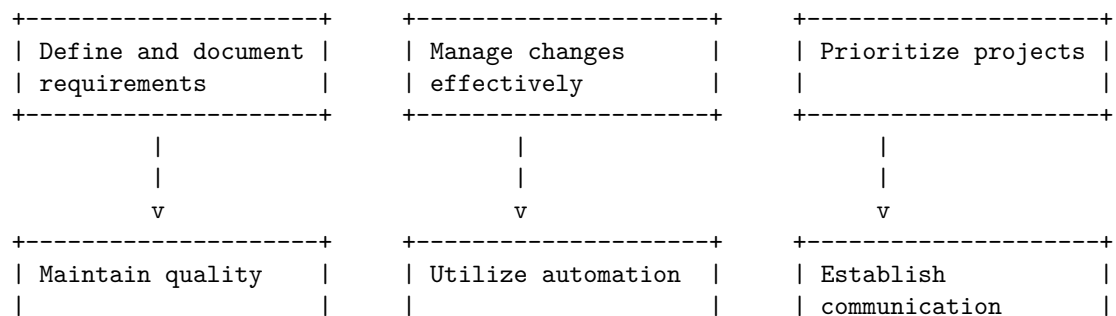
[illegible]

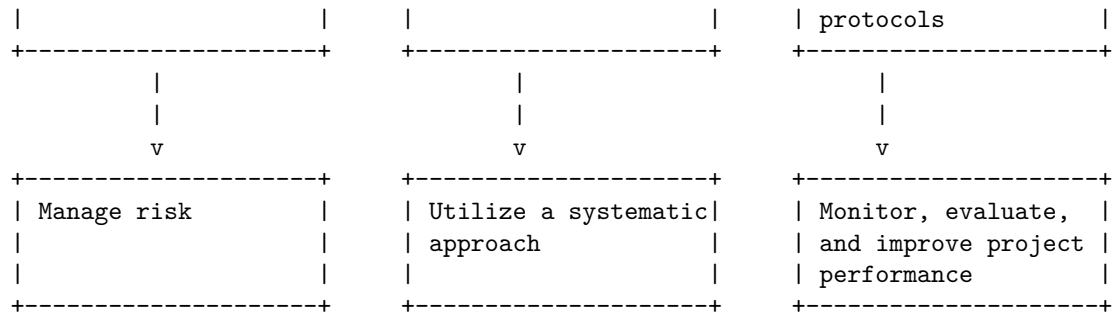
- est
ra

and implementing changes to the software project, while minimizing the impact on the quality, schedule, and budget. Changes should be tracked, documented, and communicated to all the affected parties.

- **Prioritize projects:** This principle involves selecting and ranking the software projects based on their strategic alignment, value, risk, and resource availability. Prioritization helps to allocate the limited resources to the most important and feasible projects, and to avoid overcommitment and underdelivery.
- **Maintain quality:** This principle involves ensuring that the software project meets the quality standards and expectations of the customers and stakeholders, as well as the regulatory and legal requirements. Quality should be planned, assured, controlled, and improved throughout the software project lifecycle, using appropriate tools and techniques.
- **Utilize automation:** This principle involves using software tools and systems to automate and streamline the software project activities, such as scheduling, tracking, reporting, testing, and deploying. Automation helps to reduce errors, save time, increase efficiency, and enhance collaboration.
- **Establish communication protocols:** This principle involves defining and following the communication channels, methods, frequency, and content for the software project, among the internal and external parties. Communication should be timely, accurate, relevant, and consistent, and should facilitate information sharing, feedback, and problem-solving.
- **Manage risk:** This principle involves identifying, analyzing, prioritizing, and mitigating the potential threats and uncertainties that may affect the software project's objectives, deliverables, and outcomes. Risk management helps to prevent or reduce the negative impacts of risk, and to exploit or enhance the positive opportunities of risk.
- **Utilize a systematic approach:** This principle involves applying a structured and disciplined methodology to the software project, based on the best practices and standards of the software industry. A systematic approach helps to ensure the quality, consistency, and reliability of the software project, and to facilitate the monitoring, evaluation, and improvement of the project performance.

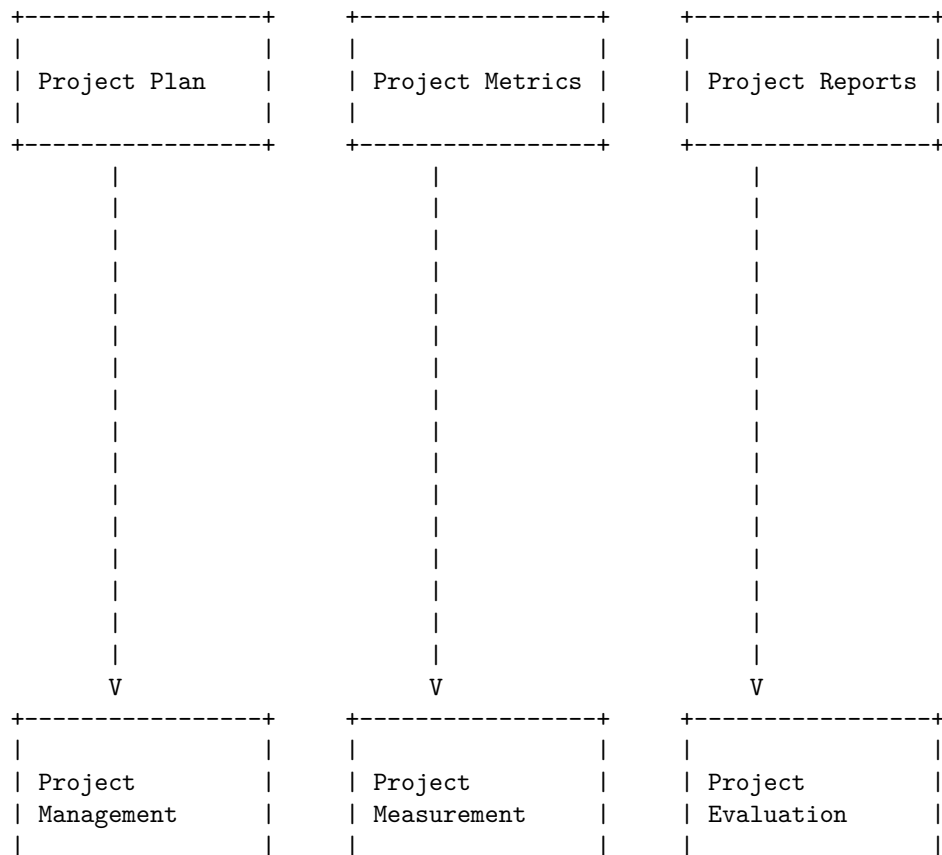
The following diagram illustrates the basic architecture of a software project management process, based on the above principles:

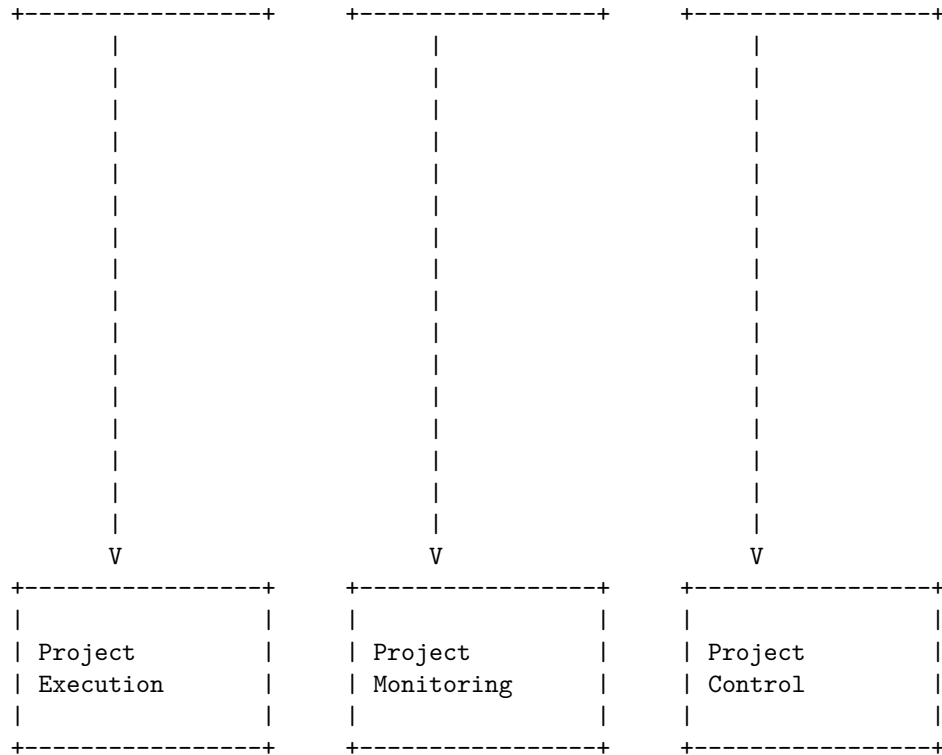




Management control in software project management is the process of monitoring and controlling project activities to ensure the project meets its goals and stays on budget. It involves setting performance targets, measuring progress towards those targets, and taking corrective action to ensure the project is on track.

The following diagram illustrates the basic architecture of management control in software project management using ASCII characters:





The diagram shows the following steps:

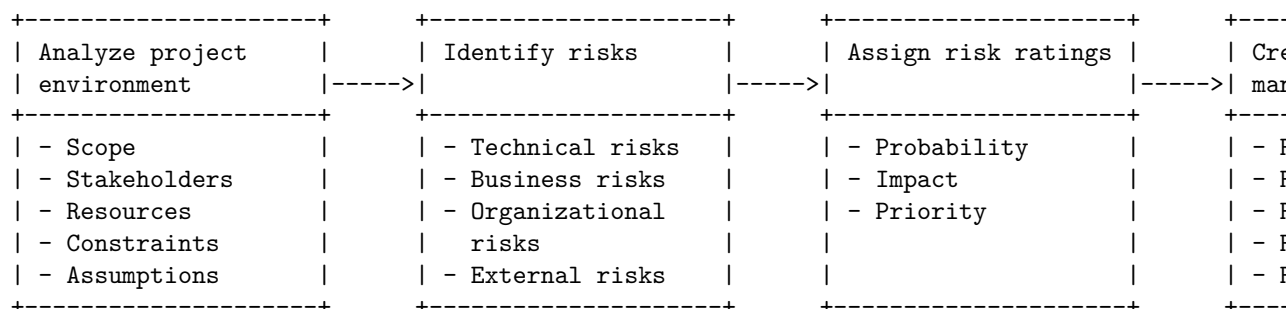
- Project plan: The project plan defines the scope, objectives, schedule, budget, and deliverables of the project. It also identifies the risks, assumptions, and constraints that may affect the project.
- Project metrics: The project metrics are the quantitative and qualitative measures that are used to track the progress and performance of the project. They include cost, time, quality, scope, and customer satisfaction metrics.
- Project reports: The project reports are the documents that communicate the status and results of the project to the stakeholders. They include status reports, progress reports, variance reports, and final reports.
- Project management: The project management is the application of knowledge, skills, tools, and techniques to plan, execute, monitor, and control the project activities. It involves initiating, planning, executing, monitoring and controlling, and closing the project phases.
- Project measurement: The project measurement is the process of collecting, analyzing, and reporting the project metrics. It involves defining, selecting, collecting, validating, and reporting the project metrics.
- Project evaluation: The project evaluation is the process of assessing the project performance and outcomes against the project plan and metrics. It involves comparing the actual results with the planned results, identifying

Project execution: The project execution is the process of performing the project tasks and producing the project deliverables. It involves coordinating, directing, and managing the project resources, processes, and outputs.

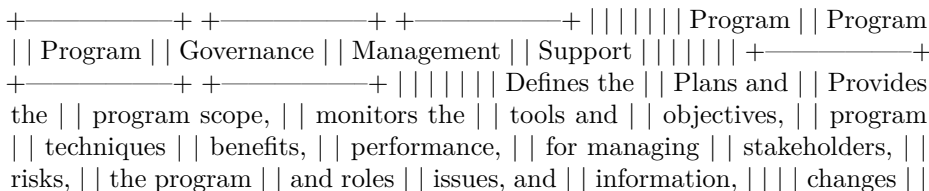
Project monitoring: The project monitoring is the process of tracking and reviewing the project progress and performance. It involves measuring, recording, and reporting the project metrics, identifying the issues and risks, and communicating the project status and results.

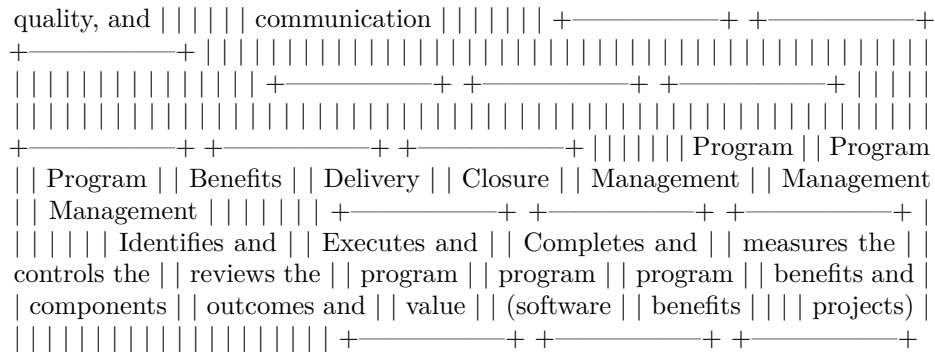
Project control: The project control is the process of ensuring the project stays on track and meets its goals and objectives. It involves analyzing the project variances, taking corrective and preventive actions, and updating the project plan and metrics.

The following diagram illustrates the basic steps of risk evaluation in SPM using ASCII art:



The following is a possible ASCII diagram that illustrates the basic components of Strategic Program Management in Software Project Management:

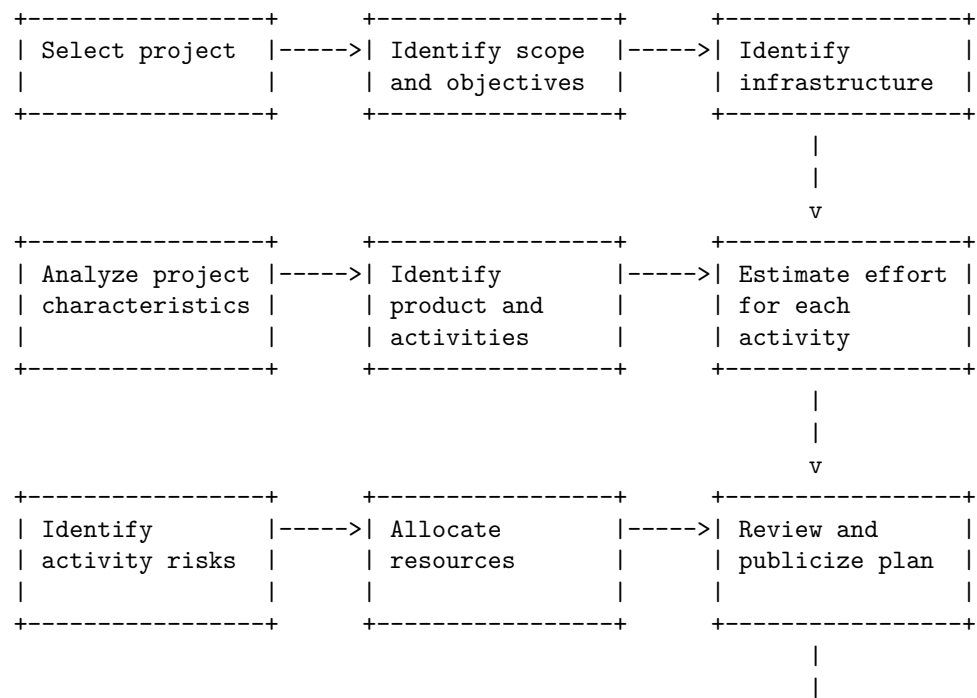


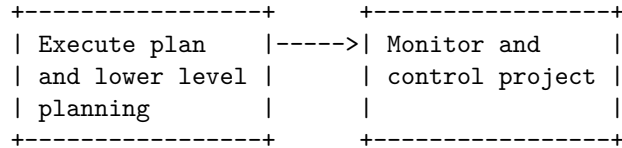


Each component of the program management interacts with the others and influences the program outcomes. The program manager is responsible for overseeing and coordinating the program components and ensuring that they are aligned with the strategic vision and goals of the organization.

Stepwise project planning in software project management is the process of breaking down the development process into smaller, manageable steps. This makes it easier for the project manager to plan and track progress, as each step is clearly defined. It also allows for tasks to be divided up and delegated to different team members.

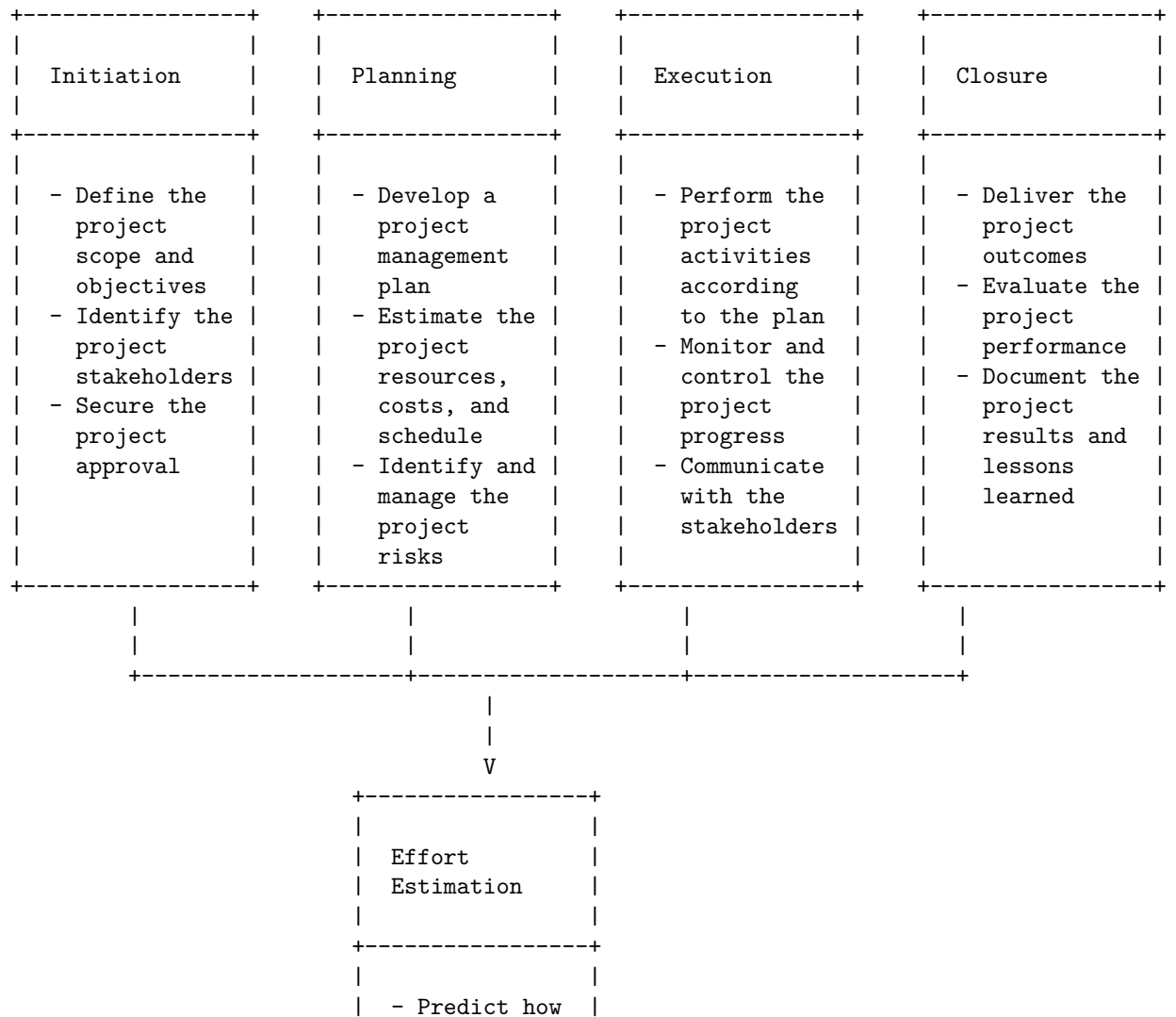
The following diagram illustrates the basic steps of stepwise project planning in software project management:





The following is a detailed ASCII diagram for Unit - 2 - Project Life Cycle and Effort Estimation for the notes of the course Software Project Management.

Unit - 2 - Project Life Cycle and Effort Estimation



	much time,	
	energy, or	
	money the	
	project	
	will require	
	- Use various	
	methods and	
	techniques	
	to estimate	
	the effort	
	- Refine the	
	estimates as	
	the project	
	progresses	

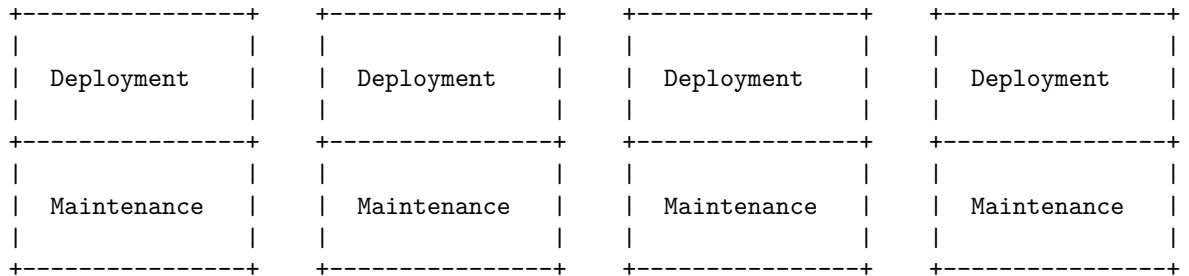
+-----+

A software process is a set of activities for designing, implementing, and testing a software system. A software process model is a simplified representation of a software process, which is presented from a specific perspective. Software process models help provide a visual representation of the development process for both the team and the customer to review.

Software Process and Process Models

The following diagram illustrates some of the common software process models:

Waterfall	Prototyping	Incremental	Agile
Requirements	Requirements	Requirements	Requirements
Design	Prototype	Design	Design
Implementation	Evaluation	Implementation	Implementation
Testing	Refinement	Testing	Testing



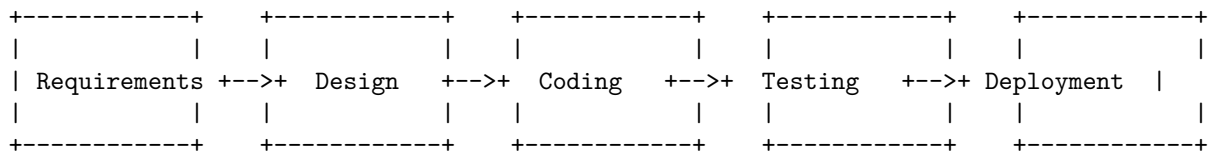
The waterfall model is a linear and sequential approach, where each phase is completed before moving to the next one. The prototyping model is an iterative approach, where a working prototype is developed and evaluated by the customer before refining it. The incremental model is a modular approach, where the system is developed and delivered in small increments. The agile model is an adaptive and collaborative approach, where the requirements and solutions evolve through the interaction of self-organizing and cross-functional teams. There are other software process models, such as spiral, V-shaped, and RAD, which are variations or combinations of the above models. Each model has its own advantages and disadvantages, depending on the environmental, organizational, and product constraints.

Choice of Process Models in Software Project Management

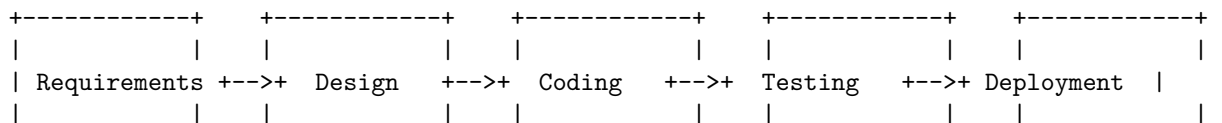
A process model is a representation of the activities, tasks, and resources involved in a software project. It provides a framework for planning, executing, monitoring, and controlling the project. Different process models have different advantages and disadvantages depending on the project characteristics, such as size, complexity, scope, quality, and risk.

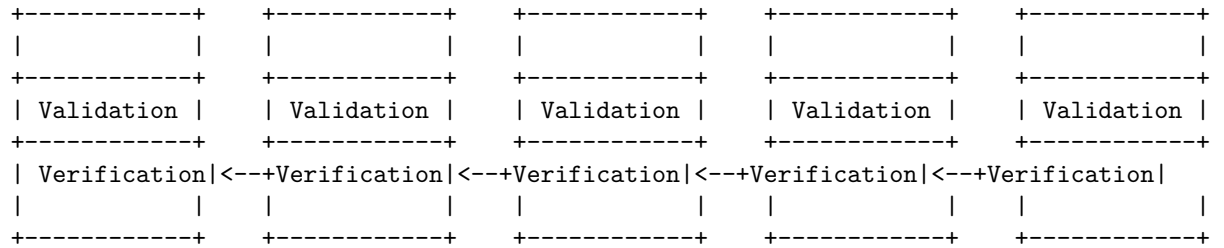
The following diagram illustrates some of the common process models in software project management using ASCII art:

Waterfall model: A linear and sequential approach that follows a fixed set of phases from re

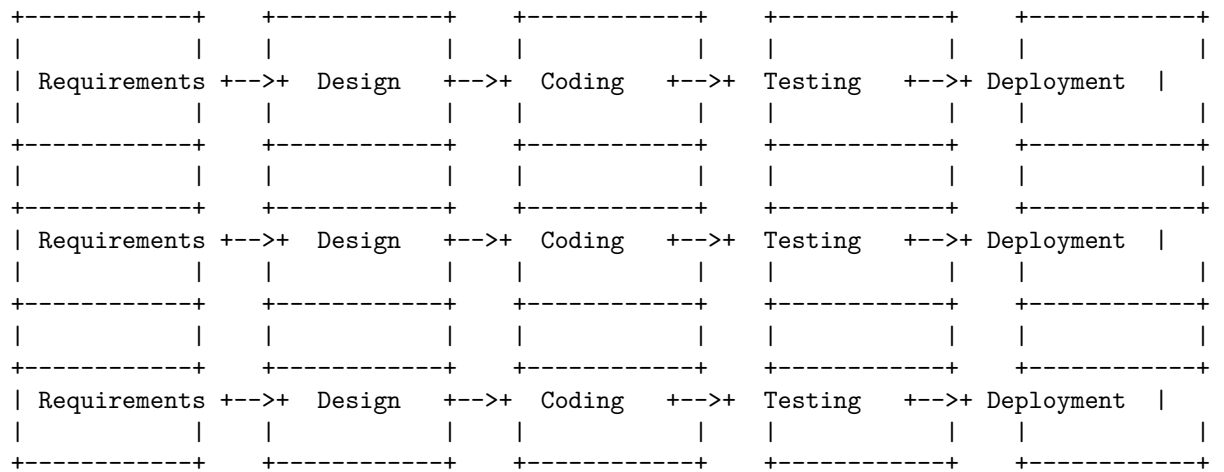


V model: An extension of the waterfall model that emphasizes verification and validation act

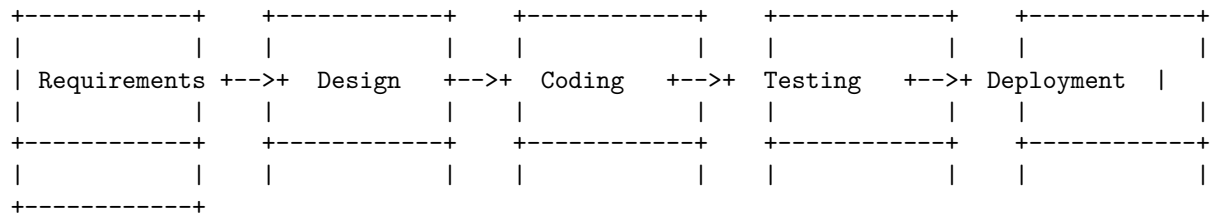




Incremental model: A modular approach that divides the project into small increments that are

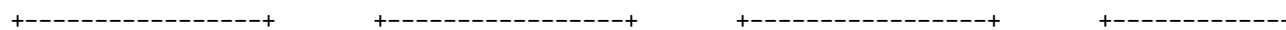


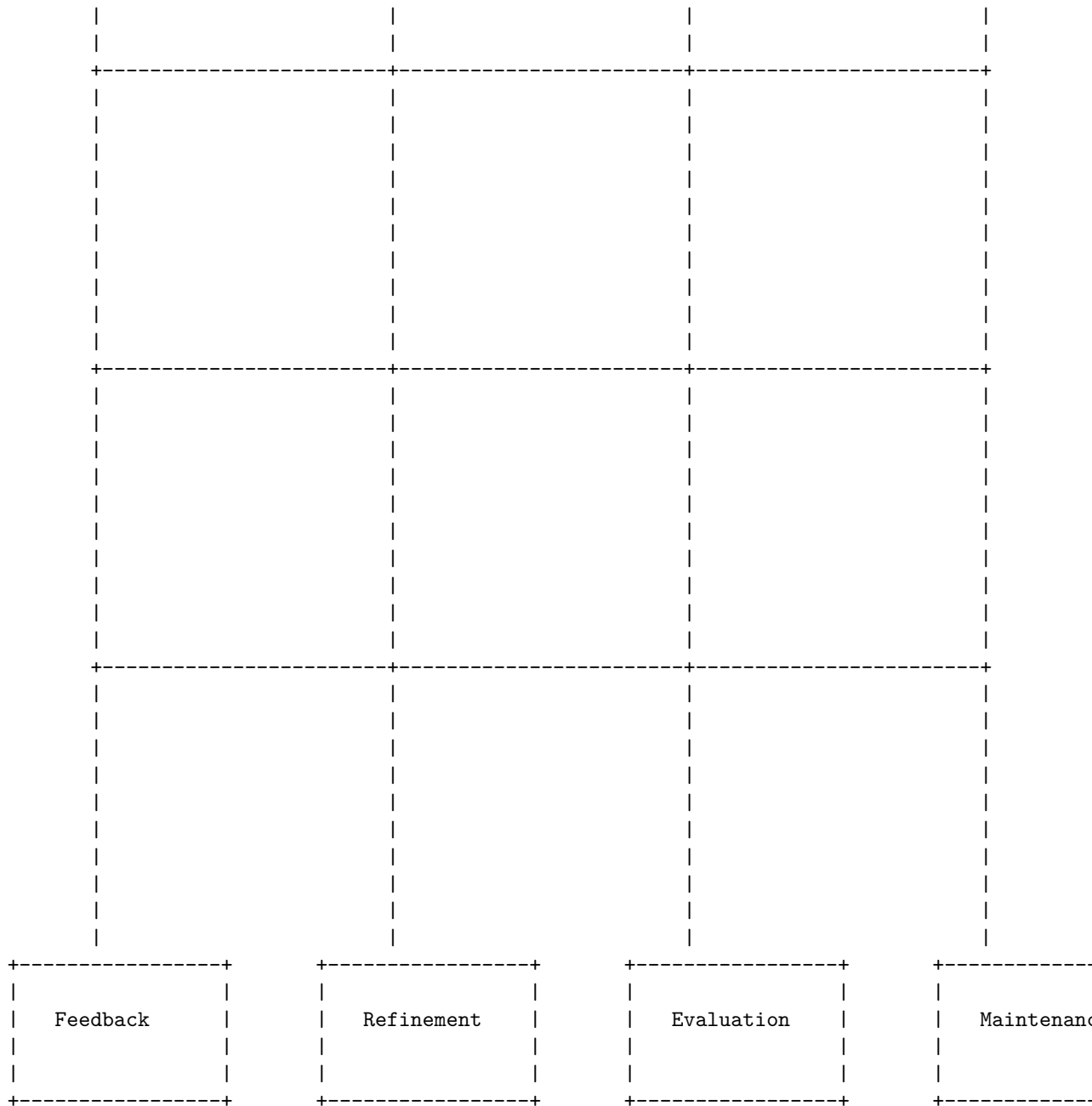
RAD model: A rapid and iterative approach that focuses on prototyping and user feedback to



Rapid Application Development (RAD) is a software development methodology that focuses on fast and iterative delivery of working software components through prototyping, user feedback and reuse of existing code. It is suitable for projects that have changing or unclear requirements, short time frames and user involvement.

The following is a detailed ASCII diagram for Rapid Application Development in Software Project Management:





The diagram shows the four main phases of RAD: Requirements Gathering, Prototyping, Testing and Deployment. Each phase consists of several sub-phases that are performed in a cyclic

Agile methods in software project management are a set of frameworks that

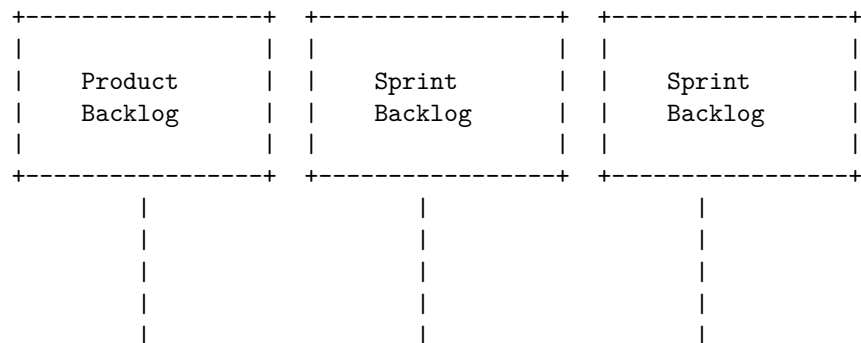
aim to deliver software products in an iterative and incremental way, based on customer feedback and collaboration. Agile methods emphasize the value of working software, customer satisfaction, team interaction, and responding to change over following a plan.

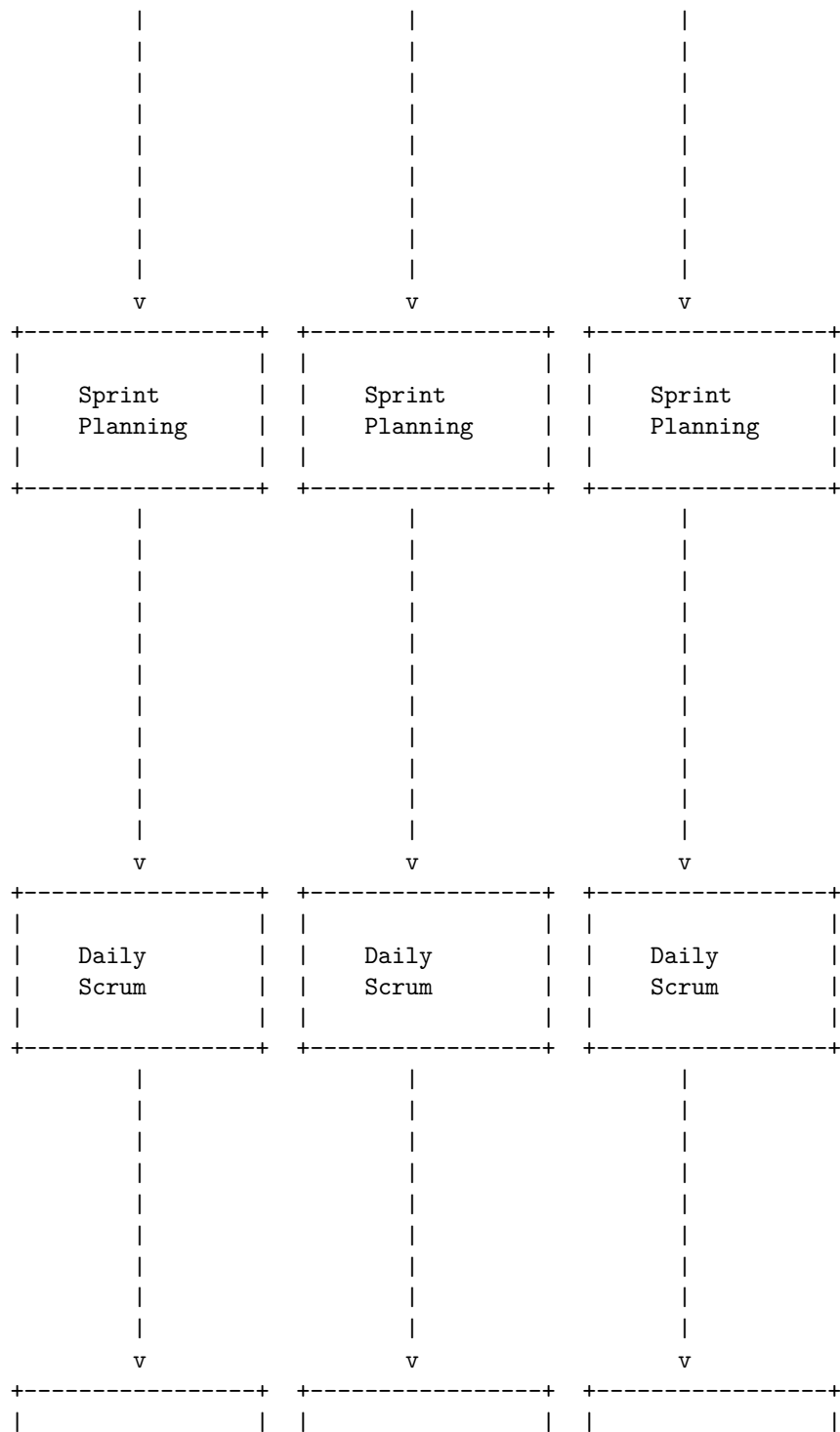
There are different agile methods that can be used to perform an agile process, such as scrum, kanban, extreme programming, and lean. Each method has its own roles, practices, artifacts, and ceremonies. However, they all share some common principles and values, such as:

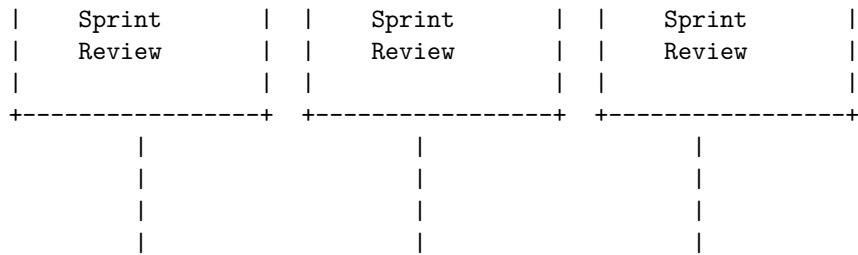
- Delivering working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Welcoming changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Business people and developers must work together daily throughout the project.
- Building projects around motivated individuals. Giving them the environment and support they need, and trusting them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile Methods in Software Project Management

The following diagram illustrates the basic architecture of a agile process, using scrum as an example:

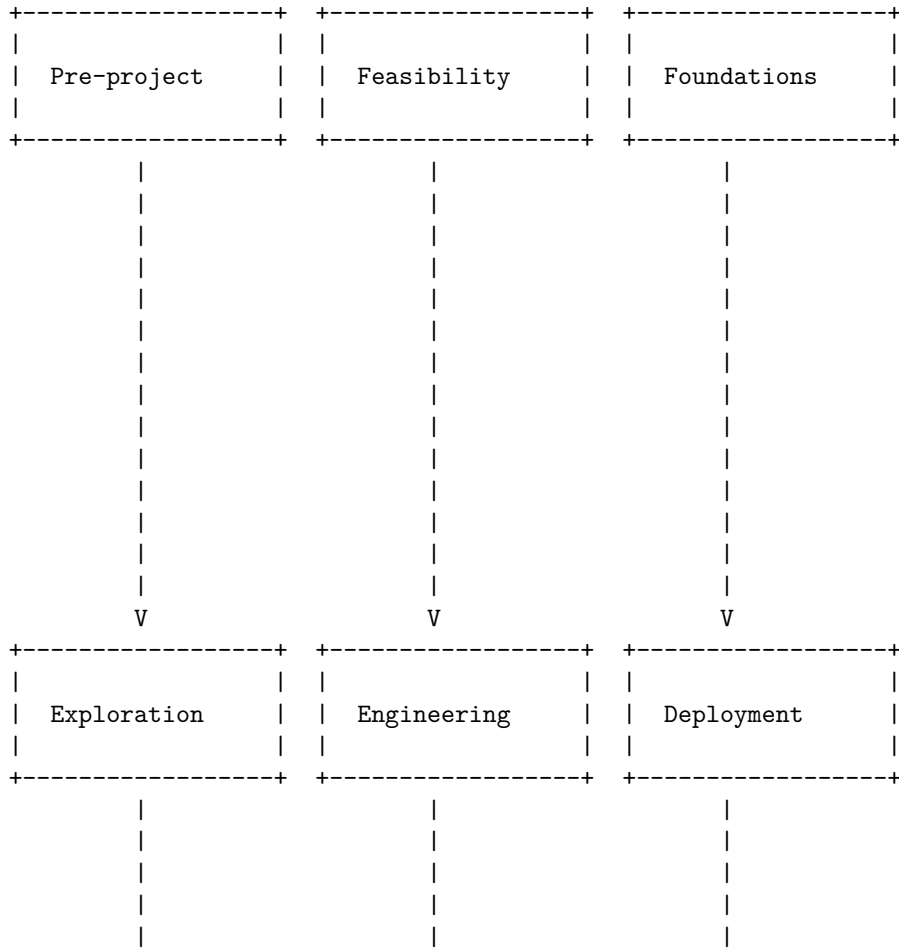


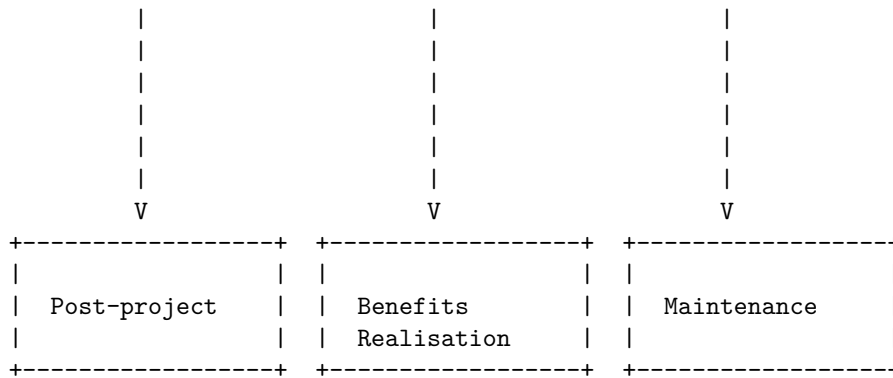




Dynamic System Development Method (DSDM) is an agile project delivery framework, initially used as a software development method. It is based on the principles of rapid application development (RAD), user involvement, empowerment of teams, frequent delivery, integrated testing, and stakeholder collaboration.

The following diagram illustrates the basic architecture of DSDM in spm:



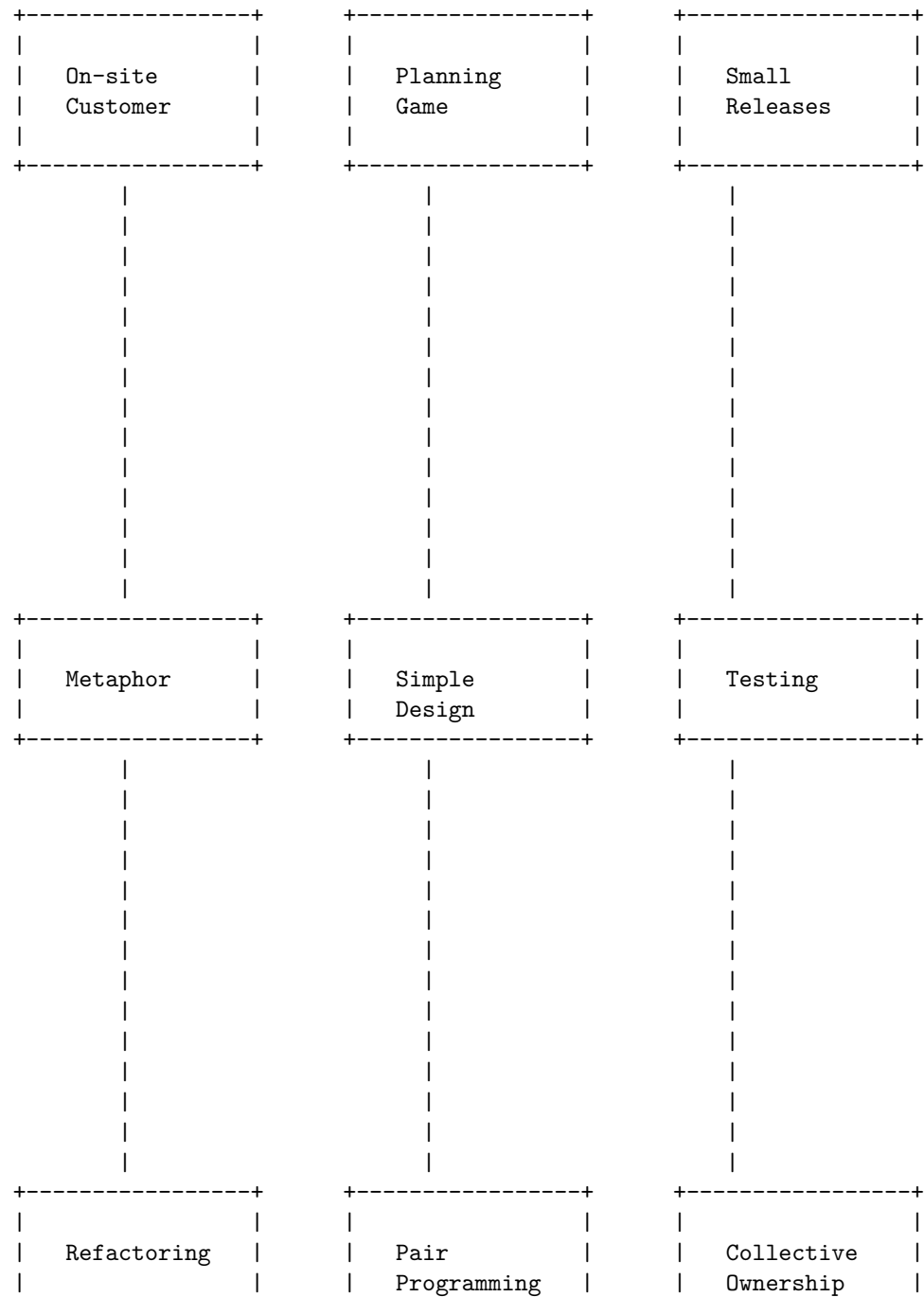


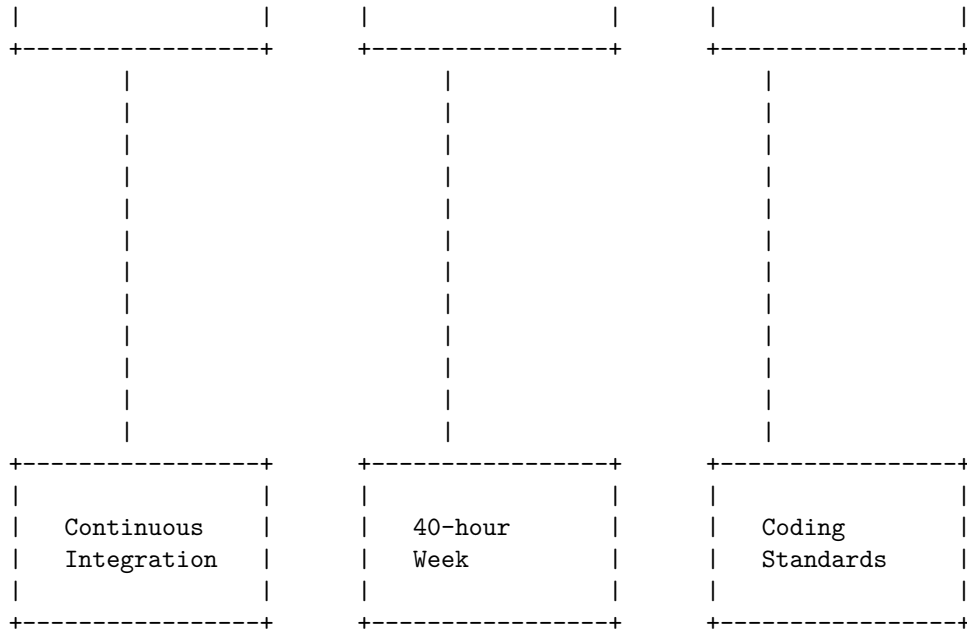
The diagram shows the eight phases of DSDM, which are:

- Pre-project: This phase ensures that only the right projects are started and that they are set up correctly.
- Feasibility: This phase quickly investigates the problem domain and the potential solutions, and determines whether the project is feasible and viable.
- Foundations: This phase establishes the foundations for the project, such as the business case, the requirements, the architecture, the development approach, and the management and delivery plans.
- Exploration: This phase iteratively develops the potential solutions and tests them against the requirements and the business objectives.
- Engineering: This phase iteratively develops and tests the solution in more detail and ensures that it meets the quality standards and the acceptance criteria.
- Deployment: This phase delivers the solution to the business and ensures that it is ready for use and meets the expectations of the stakeholders.
- Post-project: This phase reviews the project outcomes and the lessons learned, and ensures that the benefits are realized and sustained.
- Benefits Realisation: This phase monitors and evaluates the benefits of the solution over time and identifies any opportunities for improvement or further development.
- Maintenance: This phase provides ongoing support and maintenance for the solution and ensures that it remains fit for purpose and aligned with the business needs.

Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software, and higher quality of life for the development team. It is based on five values: communication, simplicity, feedback, courage, and respect. It also follows five rules: planning, managing, designing, coding, and testing. It also involves 12 practices: the planning game, small releases, metaphor, simple design, testing, refactoring, pair programming, collective ownership, continuous integration, 40-hour week, on-site customer, and coding standards.

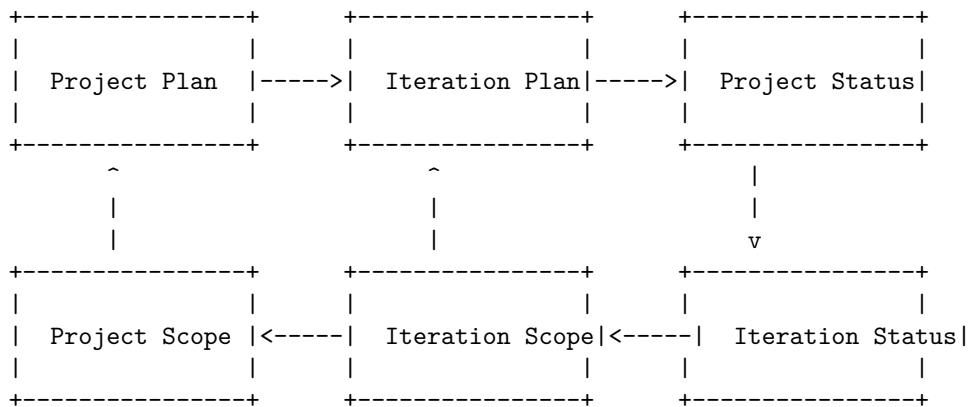
The following diagram illustrates the basic architecture of Extreme Programming in spm using ASCII art:





Managing Interactive Processes in spm is a way of planning and leading software projects that involves adapting to changes, feedback, and risks as the project unfolds. It is based on the iterative development process, which is a trial-and-error methodology that brings the project closer to its end goal.

The following diagram illustrates the basic architecture of a Managing Interactive Processes in spm:



The diagram shows the following steps:

- The project plan defines the overall objectives, deliverables, and timeline of the project.
- The iteration plan breaks down the project into smaller chunks of work,

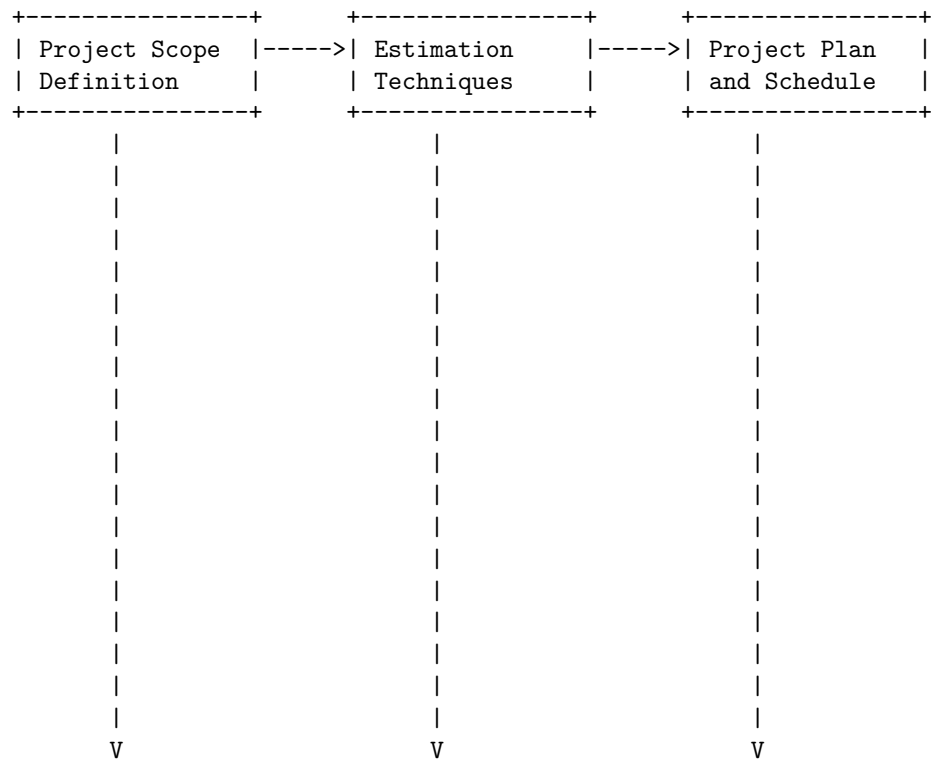
called iterations, that can be completed within a fixed time frame, usually a few weeks.

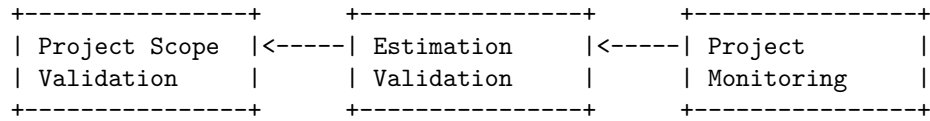
- The project status tracks the progress and quality of the project, and reports any issues or risks that may affect the project outcome.
- The iteration scope defines the specific tasks, features, and requirements that will be implemented in each iteration.
- The iteration status tracks the progress and quality of each iteration, and reports any issues or risks that may affect the iteration outcome.
- The project scope is updated based on the feedback and results from each iteration, and the project plan is revised accordingly.

Software estimation is the process of predicting the effort, cost, and duration of a software project based on various factors, such as project scope, complexity, work environment, and human resources. Software estimation is an important part of software project management (SPM), as it helps in planning, budgeting, and controlling the project.

There are different approaches and techniques for software estimation, such as expert judgment, analogy, parametric models, algorithmic models, and machine learning. Each approach has its own advantages and limitations, and the choice of the best method depends on the context and characteristics of the project.

The following diagram illustrates the basic steps of software estimation in SPM:





The diagram shows that the software estimation process starts with defining the project scope, which includes the requirements, objectives, deliverables, and constraints of the project. The project scope is then used as the input for applying one or more estimation techniques, which produce estimates of the effort, cost, and duration of the project. These estimates are then used to create a project plan and schedule, which specify the tasks, resources, milestones, and deadlines of the project.

The project plan and schedule are then monitored and controlled throughout the project execution, and any deviations or changes are reported and analyzed. The project monitoring may also provide feedback to the estimation and scope definition steps, as the actual data and experience may reveal errors or uncertainties in the previous estimates or scope. The estimation and scope validation steps are then performed to check the accuracy and completeness of the estimates and scope, and to make any necessary adjustments or revisions. The software estimation process is thus an iterative and dynamic process that aims to improve the quality and reliability of the project outcomes.

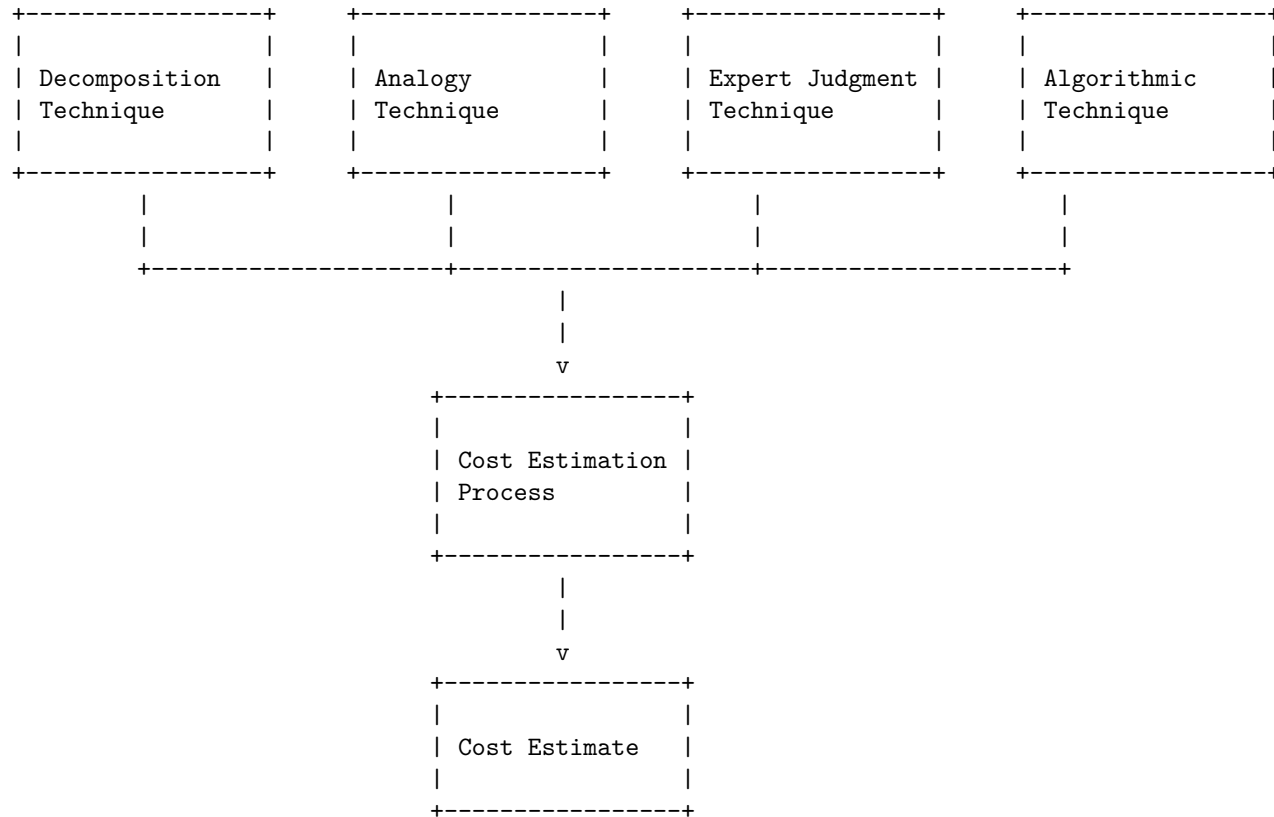
Effort and cost estimation techniques in software project management (SPM) are methods and tools that help to estimate the size, effort, and cost of a software project. They are used to plan, budget, and control the software development process. Some of the common techniques are:

- **Decomposition technique:** This technique breaks down a project into smaller and manageable units, such as functions, modules, or activities. The size, effort, and cost of each unit are estimated separately and then aggregated to get the total estimate. This technique can use different methods to estimate the size, such as lines of code, function points, or use cases.
- **Analogy technique:** This technique uses historical data from similar completed projects to estimate the size, effort, and cost of the current project. The similarity can be based on various factors, such as domain, technology, complexity, or team size. This technique requires a database of past projects and a method to adjust the estimates for the differences between the projects.
- **Expert judgment technique:** This technique relies on the experience and intuition of experts who have knowledge of the project domain, technology, and methodology. The experts can use various sources of information, such as requirements, specifications, prototypes, or benchmarks, to provide their estimates. This technique can use different methods to elicit and combine the expert opinions, such as Delphi

method, wideband Delphi method, or planning poker.

- **Algorithmic technique:** This technique uses mathematical models and formulas to estimate the size, effort, and cost of a software project based on some input parameters, such as size, complexity, productivity, or quality. The models can be empirical, derived from statistical analysis of historical data, or theoretical, based on assumptions and principles of software engineering. Some of the well-known models are COCOMO, SLIM, Function Point Analysis, or SEER-SEM.

The following diagram illustrates the basic architecture of a software cost estimation process using these techniques:



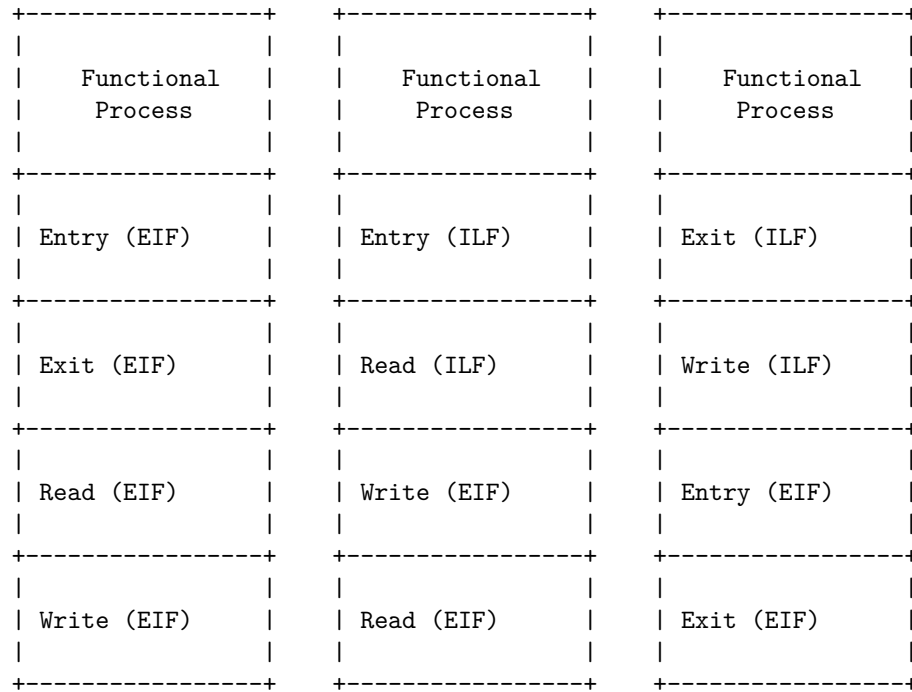
COSMIC function points are a unit of measure of software functional size. The size is a consistent measurement (or estimate) which is very useful for planning and managing software and related activities. The process of measuring software size is called functional size measurement (FSM).

COSMIC function points are based on the concept of data movements, which are the smallest pieces of functionality that can be recognized by a user. There are four types of data movements: Entry, Exit, Read, and Write. Each data

movement has a corresponding function point, which is counted according to the number of data attributes and data groups involved .

The basic architecture of a COSMIC function point measurement is shown in the following diagram, which is drawn using ASCII characters:

COSMIC Full Function Points in spm



The diagram illustrates the possible data movements between a functional process and the data groups that it manipulates. A data group is a set of data attributes that are logically related and recognizable by a user. A data attribute is a single piece of data that has a meaning for a user.

A functional process is a set of data movements that satisfies a user's functional requirement. A functional process can be triggered by an Entry or an Exit data movement, or by an external event. A functional process can have any number of data movements, as long as they are logically related and coherent.

An Entry data movement is a movement of data from the user or another functional process to the functional process being measured. An Exit data movement is a movement of data from the functional process being measured to the user or another functional process. A Read data movement is a movement of data from a persistent data group to the functional process being measured. A Write data movement is a movement of data from the functional process being measured to a persistent data group.

A persistent data group is a data group that is stored in the software and can be accessed by more than one functional process. A persistent data group can be either an Internal Logical File (ILF) or an External Interface File (EIF). An ILF is a data group that is maintained by the software being measured. An EIF is a data group that is maintained by another software, but is used by the software being measured.

The size of a data movement is determined by the number of data attributes and data groups involved. A data movement has a size of one function point if it involves one data attribute and one data group. A data movement has a size of two function points if it involves either two or more data attributes and one data group, or one data attribute and two or more data groups. A data movement has a size of three function points if it involves two or more data attributes and two or more data groups.

The size of a functional process is the sum of the sizes of its data movements. The size of the software is the sum of the sizes of its functional processes.

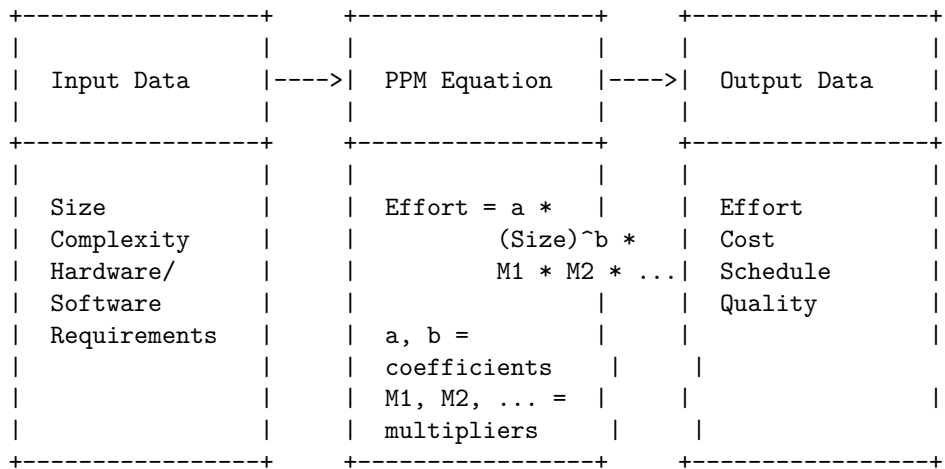
COCOMO II is a software cost estimation model that consists of three sub-models: Application Composition, Early Design, and Post-Architecture. Each sub-model uses different inputs and formulas to estimate the effort, cost, and schedule of a software project. The following diagram illustrates the basic architecture of COCOMO II in SPM (Software Project Management):

Application Composition	Early Design	Post-Architecture
Inputs:	Inputs:	Inputs:
- Object Points	- Size (KLOC)	- Size (KLOC)
- Productivity Rate	- Scale Factors	- Scale Factors
	- Cost Drivers	- Cost Drivers
Outputs:	Outputs:	Outputs:
- Effort (PM)	- Effort (PM)	- Effort (PM)
- Schedule (M)	- Schedule (M)	- Schedule (M)
- Cost (\$)	- Cost (\$)	- Cost (\$)
Formulas:	Formulas:	Formulas:
- Effort = (Object Points / Productivity Rate) * 0.4	- Effort = $A * Size^E$	- Effort = $A * Size^E$
- Schedule = $3.0 * Effort^{0.33}$	- Schedule = $B * Effort^F$	- Schedule = $B * Effort^F$
- Cost = Effort * Average Salary	- Cost = Effort * Average Salary	- Cost = Effort * Average Salary
	- A, E, B, F are derived from	- A, E, B, F are derived from

- Object Points		Scale Factors		Scale Factors	
are estimated		and Cost Drivers		and Cost Drivers	
based on the		- Size is		- Size is	
number and		estimated based		estimated based	
complexity of		on analogy,		on analogy,	
screens,		expert judgment,		expert judgment,	
reports, and		or function		or function	
components		points		points	
+-----+		+-----+		+-----+	

A Parametric Productivity Model (PPM) in Software Project Management (SPM) is a method of estimating the cost and effort required to develop software based on various parameters, such as size, complexity, hardware/software requirements, etc. One example of a PPM is the Constructive Cost Model (CO-COMO), which uses a basic equation to calculate the effort as a function of the size of the software, and then applies different coefficients and exponents to adjust the equation for different modes of development (organic, semi-detached, or embedded) and different levels of detail (basic, intermediate, or detailed) .

The following diagram illustrates the basic architecture of a PPM in SPM using ASCII art:



The following is a possible ASCII diagram for Unit - 3 - Activity Planning and Risk Management for the notes of the course Software Project Management. It is based on the information from the web search results .

The diagram shows the main steps and activities involved in planning and managing a software project, as well as the risks and mitigation strategies associated with each step. The diagram uses the following symbols:

- : A task or activity
- -> : A dependency or sequence
- / : A parallel or concurrent activity

- X : A risk or threat
- O : A mitigation or prevention strategy

The diagram is as follows:

Unit - 3 - Activity Planning and Risk Management

Define project objectives and scope -> Identify project stakeholders and requirements -> Estimate project resources, cost and duration -> Create project schedule and milestones / Assign project roles and responsibilities / Develop project communication and reporting plan / Establish project quality standards and metrics -> Finalize project plan and get approval -> Execute project plan and monitor progress / Perform project activities and deliverables / Track project performance and quality / Manage project changes and issues / Communicate project status and feedback -> Close project and evaluate outcomes / Deliver project products and services / Document project lessons learned and best practices / Release project resources and celebrate success

X Project objectives and scope are unclear or unrealistic O Conduct feasibility study and market analysis O Involve stakeholders and experts in defining project scope O Use scope management techniques to control changes

X Project stakeholders and requirements are incomplete or conflicting O Conduct stakeholder analysis and prioritize their needs O Use requirement elicitation and validation techniques O Use requirement management tools and traceability matrix

X Project resources, cost and duration are underestimated or inaccurate O Use estimation techniques and tools based on historical data and expert judgment O Apply contingency and reserve analysis to account for uncertainties O Use cost and schedule management techniques to control variances

X Project schedule and milestones are not aligned with project objectives and constraints O Use network planning models and critical path analysis to sequence and schedule activities O Use activity float and crashing techniques to shorten project duration O Use milestone charts and Gantt charts to visualize project schedule

X Project roles and responsibilities are not defined or assigned properly O Use organizational breakdown structure and responsibility assignment matrix to define project roles O Use resource allocation and leveling techniques to optimize resource utilization O Use team building and motivation techniques to enhance team performance

X Project communication and reporting plan is not effective or consistent O Use communication planning and analysis techniques to identify communication needs and methods O Use communication management tools and techniques to facilitate information exchange O Use reporting and presentation techniques to convey project status and feedback

X Project quality standards and metrics are not defined or measured properly
O Use quality planning and assurance techniques to establish quality standards and criteria
O Use quality control and improvement techniques to monitor and enhance quality
O Use quality management tools and techniques to analyze and resolve quality issues

X Project plan is not approved or supported by stakeholders
O Use stakeholder engagement and management techniques to gain buy-in and commitment
O Use negotiation and conflict resolution techniques to address stakeholder concerns
O Use project charter and sign-off techniques to formalize project approval

X Project plan is not executed or monitored properly
O Use project management software and tools to execute and track project plan
O Use project management methodologies and standards to guide project execution and monitoring
O Use project management knowledge areas and processes to manage project integration

X Project activities and deliverables are not completed or delivered on time and within budget and scope
O Use work breakdown structure and activity list to define and organize project work
O Use deliverable acceptance and verification techniques to ensure quality and compliance
O Use project closure and handover techniques to transfer project products and services

X Project performance and quality are not tracked or reported properly
O Use performance measurement and analysis techniques to evaluate project progress and outcomes
O Use earned value management and variance analysis techniques to measure project performance
O Use key performance indicators and metrics to report project quality

X Project changes and issues are not managed or resolved properly
O Use change management and control techniques to handle change requests and approvals
O Use issue management and escalation techniques to identify and address project problems
O Use risk management and mitigation techniques to anticipate and reduce project risks

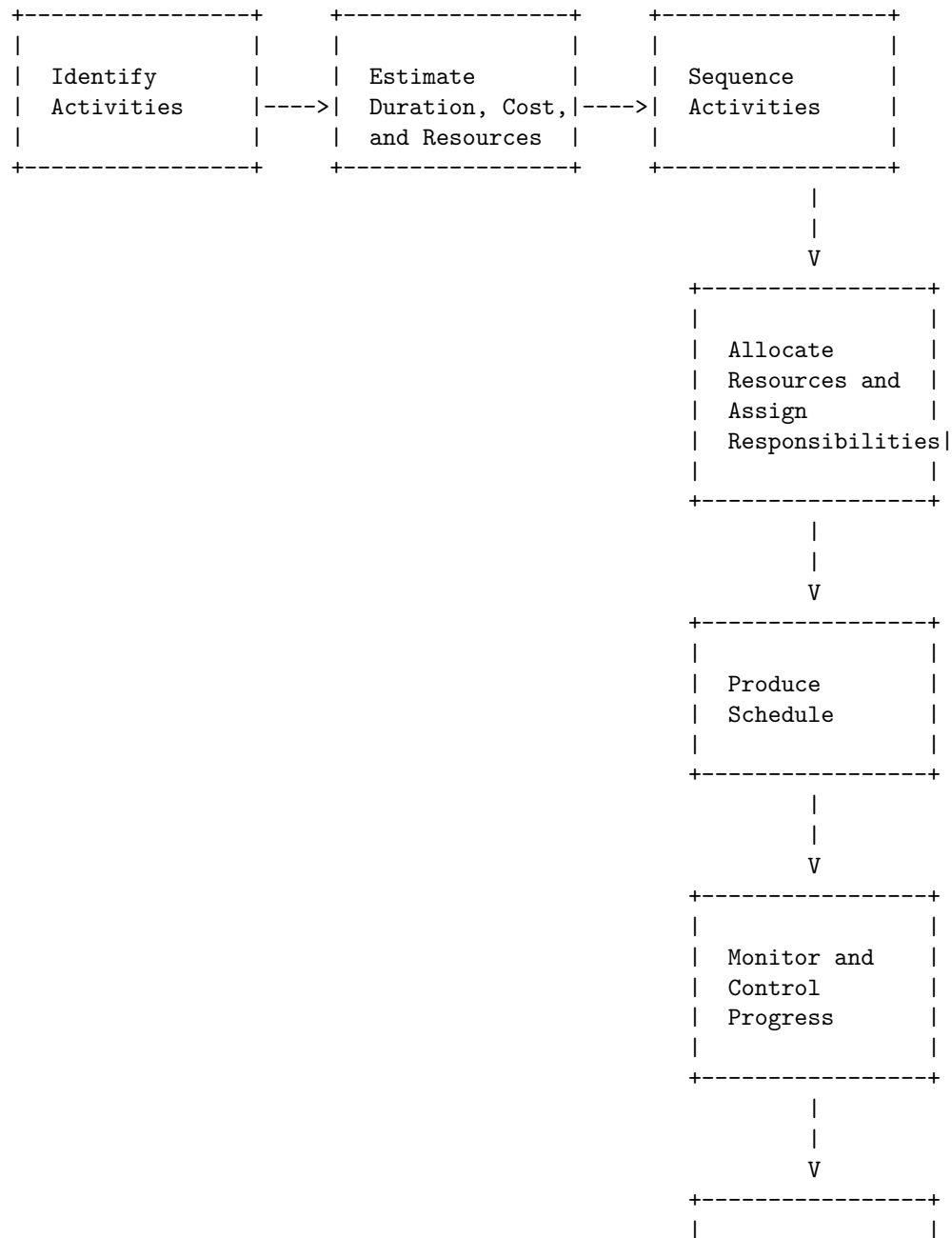
X Project status and feedback are not communicated or

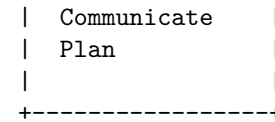
The objectives of activity planning in software project management (SPM) are:

- To identify the activities that need to be performed to complete the project successfully.
- To estimate the duration, cost, and resources required for each activity.
- To sequence the activities in a logical order and identify their dependencies and constraints.
- To allocate the resources to the activities and assign responsibilities to the project team members.
- To produce a detailed schedule that shows the start and finish dates of each activity and the milestones and deliverables of the project.
- To monitor and control the progress of the activities and compare them with the planned schedule and budget.

- To communicate the plan to the stakeholders and obtain their approval and feedback.

Objectives of Activity Planning in SPM





A project schedule is a document that shows the planned start and finish dates of the activities, tasks, and milestones that make up a project. It also shows the dependencies and constraints among the tasks, the resources assigned to them, and the estimated duration and cost of each task. A project schedule is used to monitor and control the progress of a project, communicate with stakeholders, and manage risks and changes.

Project Schedules in spm

In software project management (spm), a project schedule is usually created and maintained using a software tool that supports the scheduling methodology chosen by the project manager. Some common scheduling methodologies are:

- **Critical Path Method (CPM):** This method identifies the longest sequence of tasks that must be completed on time for the project to finish on schedule. This sequence is called the critical path, and any delay in any task on the critical path will delay the project completion. The CPM also calculates the amount of slack or float for each task, which is the amount of time that a task can be delayed without affecting the project end date.
- **Program Evaluation and Review Technique (PERT):** This method is similar to CPM, but it uses three estimates for each task duration: optimistic, pessimistic, and most likely. These estimates are used to calculate the expected duration and variance of each task, and then to determine the probability of completing the project within a given time frame.
- **Agile Methodology:** This method is based on the principles of adaptive planning, iterative development, and continuous improvement. It does not use a fixed and detailed project schedule, but rather a series of short-term plans called sprints or iterations, which are usually two to four weeks long. Each sprint has a set of prioritized features or user stories that the team commits to deliver at the end of the sprint. The team also conducts daily meetings called stand-ups to coordinate their work and resolve any issues.

A project schedule in spm can be represented in different formats, such as:

- **Gantt Chart:** This is a graphical representation of the project schedule, where each task is shown as a horizontal bar that spans from its start date to its finish date. The tasks are arranged vertically according to their dependencies, and the critical path is highlighted. The Gantt chart also shows the resources assigned to each task, the milestones, and the progress of each task.
- **Network Diagram:** This is a graphical representation of the project schedule, where each task is shown as a node or a box, and the dependencies among the tasks are shown as arrows or lines. The network diagram can

also show the duration, resources, and progress of each task, as well as the critical path and the slack or float of each task.

- Work Breakdown Structure (WBS): This is a hierarchical representation of the project schedule, where the project is divided into smaller and manageable components called work packages. Each work package is further decomposed into tasks or activities, and each task or activity is assigned a duration, a resource, and a cost. The WBS also shows the dependencies among the work packages and the tasks or activities.

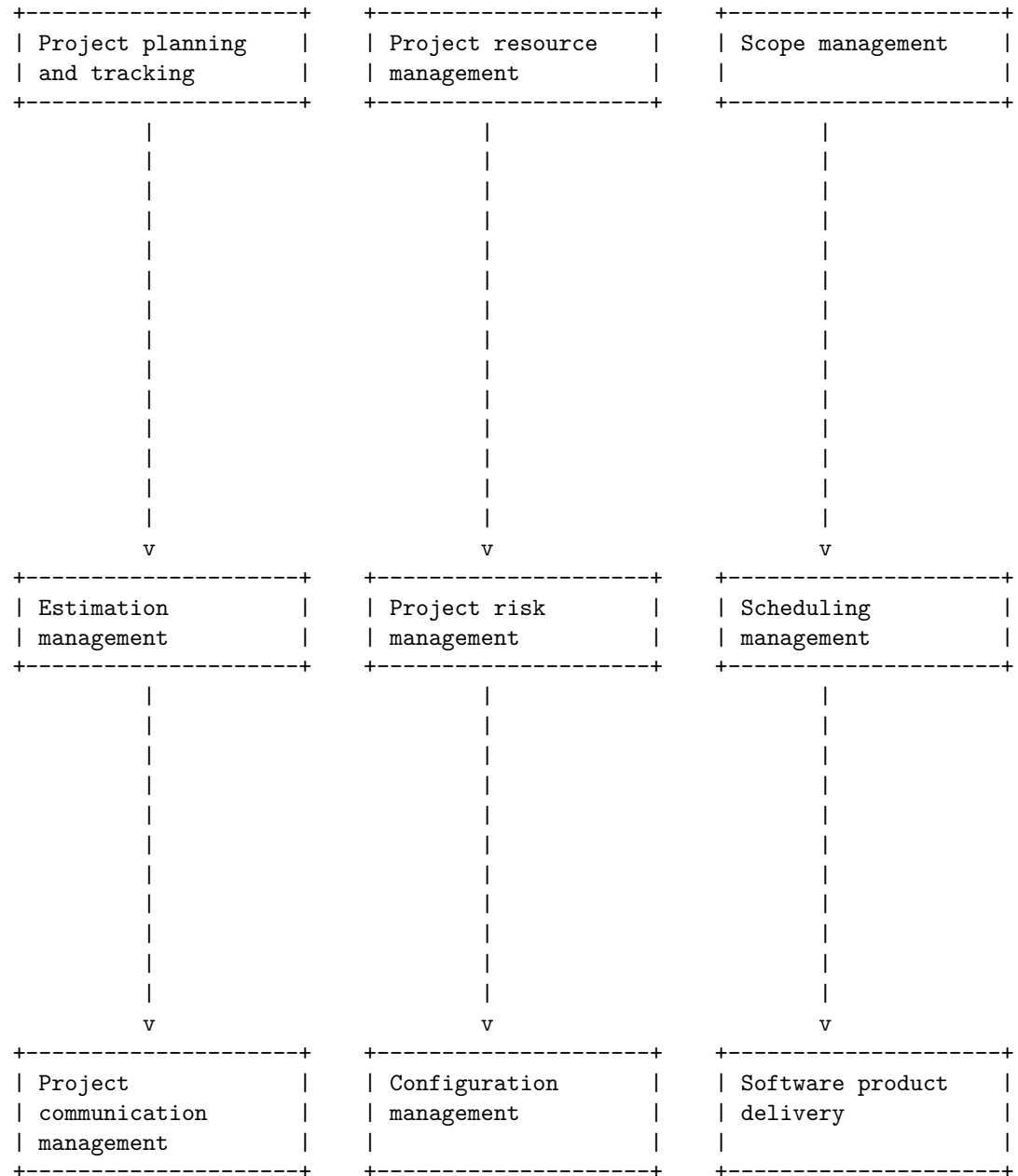
The following diagram illustrates the basic architecture of a project schedule in spm using a Gantt chart format:

+-----+					
Project Schedule					
+-----+					
Task ID	Task Name	Start Date	End Date	Duration	
+-----+					
1	Initiate Project	01/01/2023	15/01/2023	15 days	
2	Plan Project	16/01/2023	31/01/2023	16 days	
3	Execute Project	01/02/2023	30/04/2023	90 days	
4	Monitor and Control	01/02/2023	30/04/2023	90 days	
5	Close Project	01/05/2023	15/05/2023	15 days	
+-----+					
+-----+					
Gantt Chart					
+-----+					
Jan 2023	Feb 2023	Mar 2023	Apr 2023	May 2023	
+-----+					
1	=====				
2		=====			
3			=====	=====	
4			=====	=====	
5					

Activities in spm are the tasks and processes that are involved in managing a software project from its inception to its delivery. According to , the list of activities are as follows:

- Project planning and tracking
- Project resource management
- Scope management
- Estimation management
- Project risk management
- Scheduling management
- Project communication management
- Configuration management

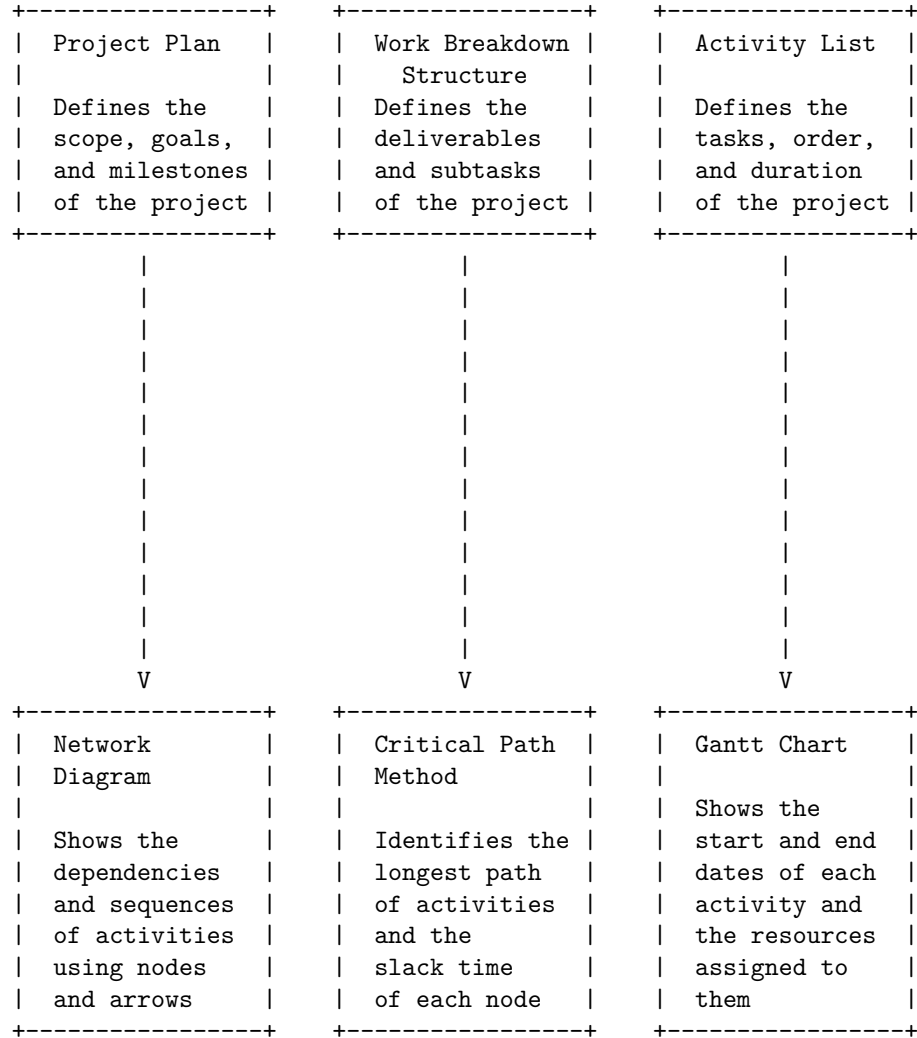
The following diagram illustrates the basic architecture of a software project management system using ASCII characters:



Sequencing and Scheduling in SPM (Software Project Management) is a process which helps to plan, coordinate, and execute complex projects. It enables project managers to schedule tasks, assign resources and track progress in a way that

minimizes risk and maximizes efficiency.

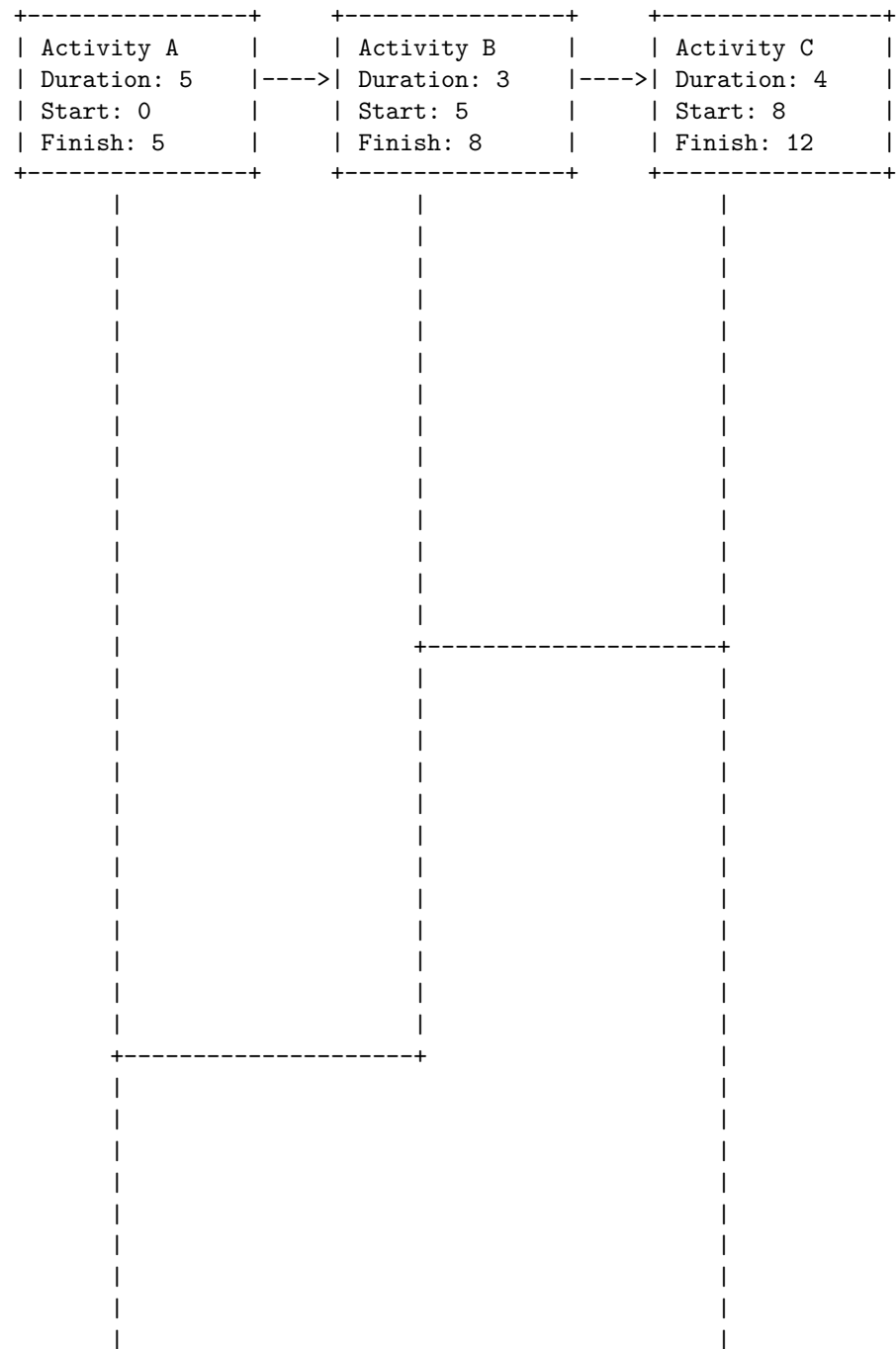
The following diagram illustrates the basic architecture of a Sequencing and Scheduling in SPM using ASCII art:

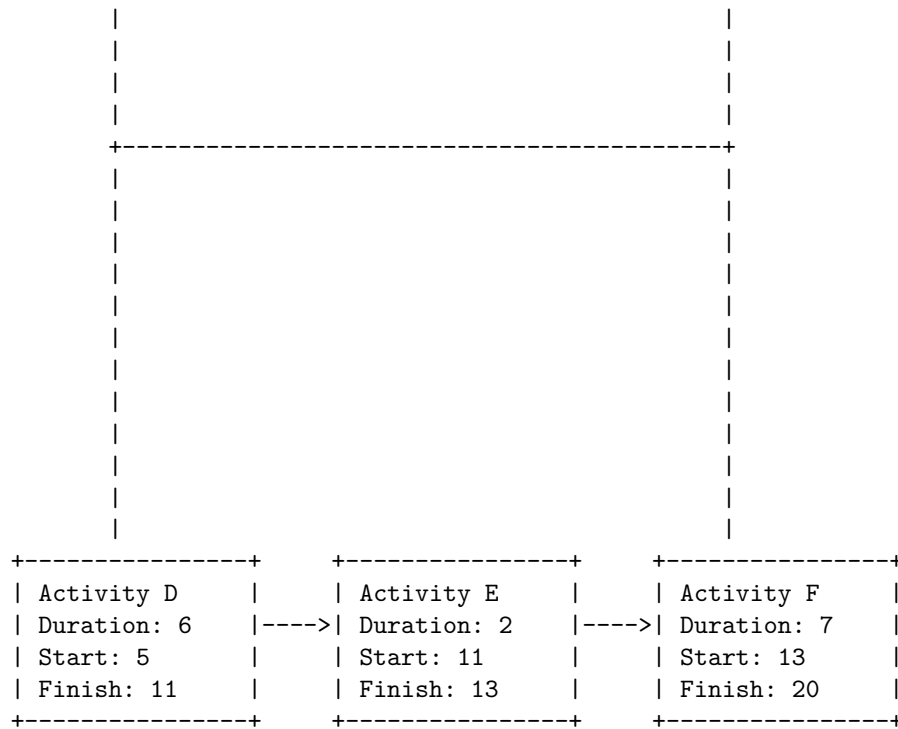


A network planning model is a graphical representation of the activities and events involved in a software project. It shows the sequence, duration, and dependencies of the tasks required to complete the project. It is used to plan, monitor, and control the project progress and resources.

There are different types of network planning models, such as activity-on-node, activity-on-arrow, Gantt chart, PERT, and CPM. One of the most common network planning models is the activity-on-node model, which uses nodes (boxes) to represent the activities and arrows to represent the dependencies. The fol-

lowing diagram illustrates the basic architecture of an activity-on-node network planning model:



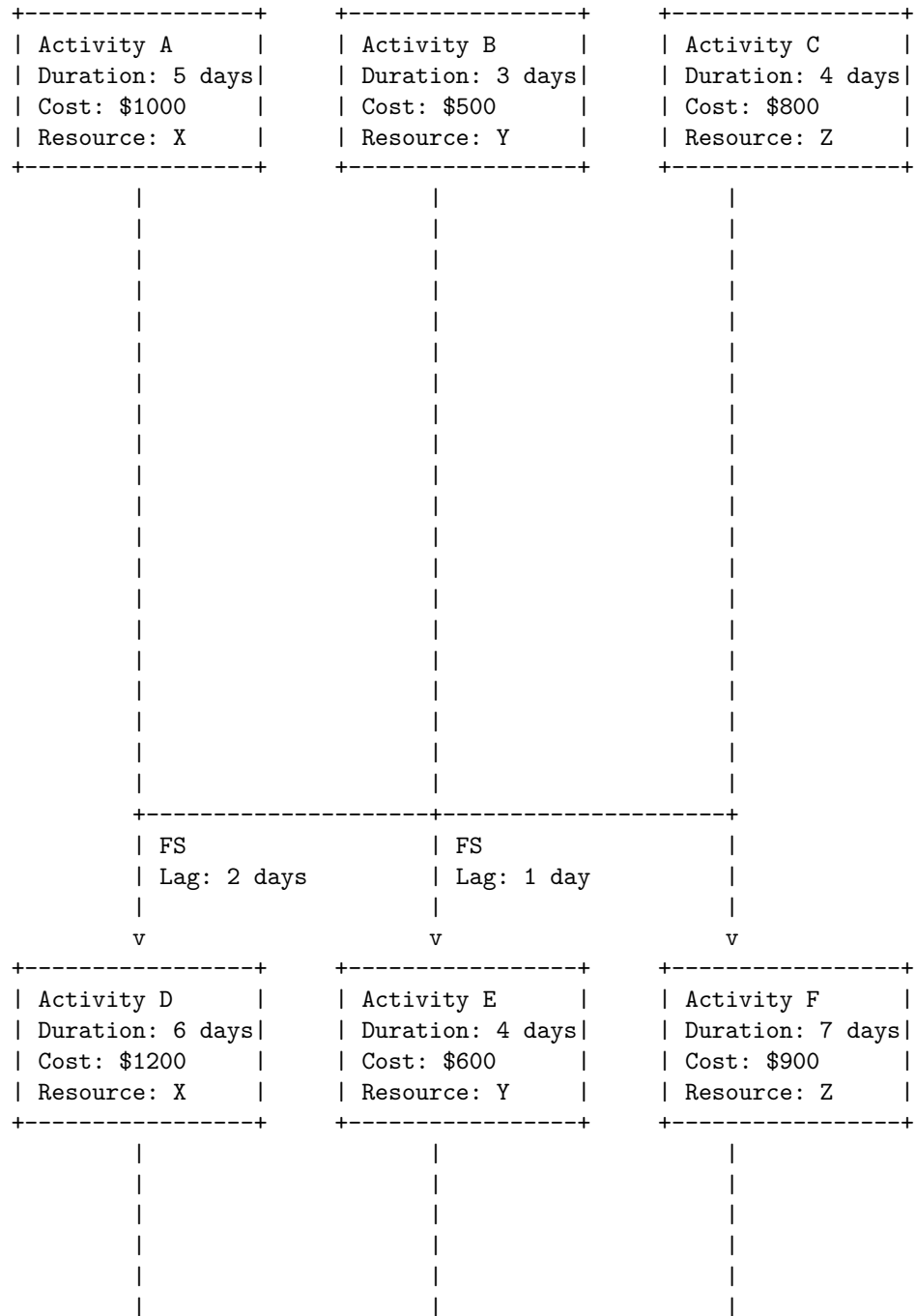


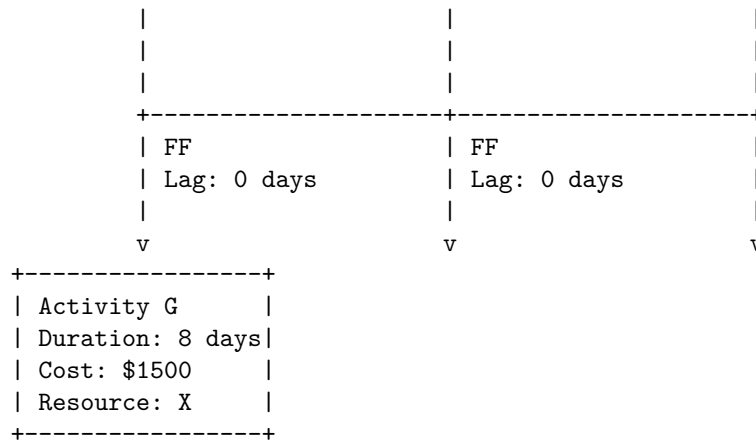
The diagram shows six activities (A, B, C, D, E, F) and their durations, start times, and finish times. The arrows indicate the dependencies between the activities, such as A must finish before B can start, and B and C must finish before E can start. The diagram can be used to identify the critical path, which is the longest path of activities that determines the minimum time to complete the project. In this case, the critical path is A-B-C-F, with a total duration of 20. The diagram can also be used to calculate the slack time, which is the amount of time an activity can be delayed without affecting the project completion time. For example, activity D has a slack time of 9, which means it can be delayed by up to 9 time units without affecting the project duration.

A network model is a graphical representation of the activities and their inter-relationships in a project. In software project management (SPM), a network model can help to plan, schedule, monitor and control the project activities. There are different ways to formulate a network model, such as activity-on-node (AON), activity-on-arrow (AOA), or precedence diagramming method (PDM).

One of the most common methods is the activity-on-node (AON) approach, where each node (box) represents an activity and each edge (line) represents a dependency between activities. The edges can have different types of dependencies, such as finish-to-start (FS), start-to-start (SS), finish-to-finish (FF), or start-to-finish (SF). The edges can also have different attributes, such as duration, cost, or resource requirements.

The following diagram illustrates the basic architecture of an AON network model in SPM:





In this diagram, each activity has a duration, a cost, and a resource requirement. The dependencies between activities are shown by the edges, with the type and the lag of each dependency. For example, activity A must finish before activity D can start, with a lag of 2 days. This means that activity D cannot start until 2 days after activity A finishes. Similarly, activity D and activity E must finish before activity G can start, with no lag. This means that activity G can start as soon as both activity D and activity E finish.

A network model can help to identify the critical path, the slack time, the earliest and latest start and finish times, and the total project duration and cost. It can also help to optimize the resource allocation, the risk management, and the quality assurance of the project. A network model is a useful tool for SPM, but it also has some limitations, such as the uncertainty of the activity durations, the complexity of the network, and the need for constant updating and monitoring.

Forward pass and backward pass are techniques used in project management to calculate the duration and slack of each activity in a network diagram. They are also known as the two-pass technique.

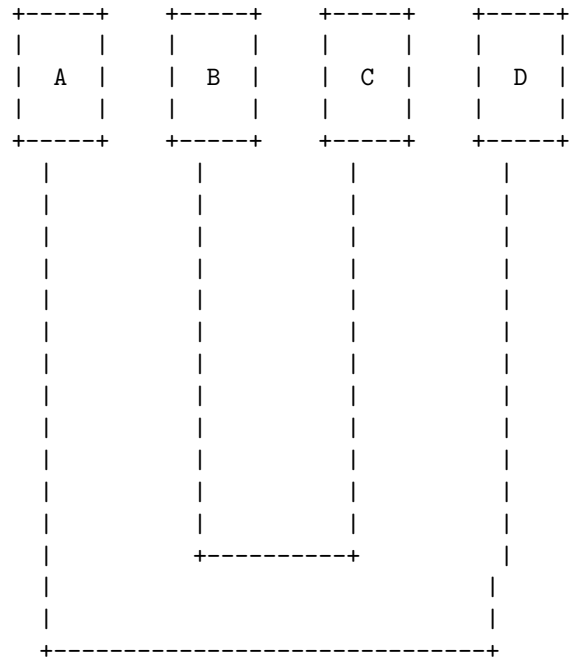
The forward pass is the process of moving forward from the start node to the end node of the network diagram, and calculating the early start (ES) and early finish (EF) values for each activity. The ES is the earliest time an activity can start, and the EF is the earliest time an activity can finish. The EF is calculated by adding the activity duration to the ES.

The backward pass is the process of moving backward from the end node to the start node of the network diagram, and calculating the late start (LS) and late finish (LF) values for each activity. The LF is the latest time an activity can finish without delaying the project, and the LS is the latest time an activity can start without delaying the project. The LS is calculated by subtracting the activity duration from the LF.

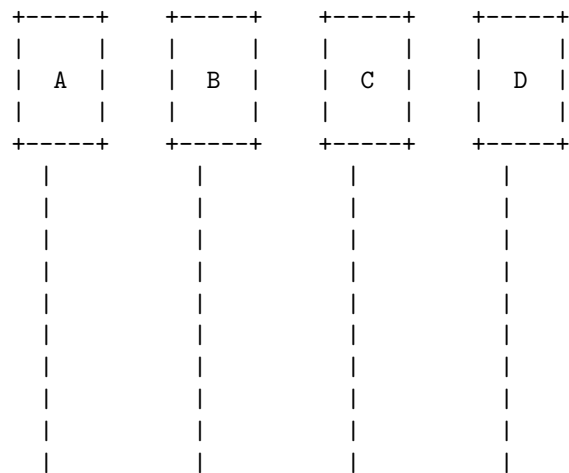
The slack or float of an activity is the amount of time an activity can be delayed

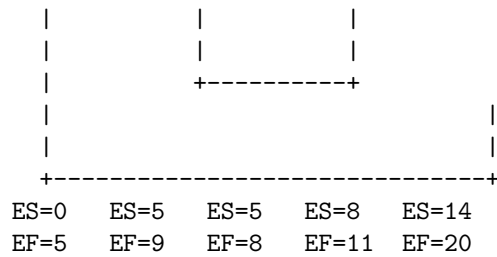
without affecting the project duration. It is calculated by subtracting the ES from the LS, or the EF from the LF. The critical path is the sequence of activities that has zero slack and determines the project duration.

The following diagram illustrates the basic steps of the forward pass and backward pass techniques in a network diagram with four activities (A, B, C, and D) and their durations (5, 4, 3, and 6 days respectively).

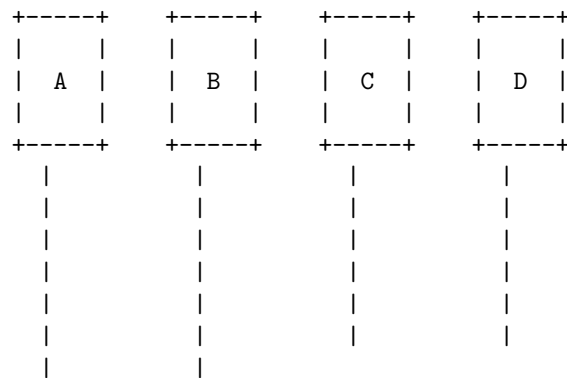


Step 1: Perform the forward pass by moving from left to right and calculating the ES and EF values for each activity.





Step 2: Perform the backward pass by moving from right to left and calculating the LF and LS values for each activity.



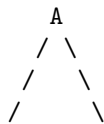
The Critical Path Method (CPM) is a technique where you identify tasks that are necessary for project completion and determine scheduling flexibilities. A critical path in project management is the longest sequence of activities that must be finished on time in order for the entire project to be complete .

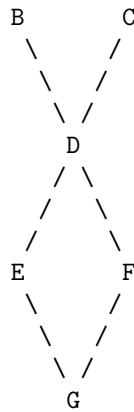
The following steps are involved in the CPM:

- Specify each activity using the work breakdown structure (WBS).
- Establish dependencies (logical relationships) between the activities.
- Draw the network diagram (graphical representation of the project activities and dependencies).
- Estimate the activity durations (time required to complete each activity).
- Identify the critical path (the longest path in the network diagram).
- Update the CPM diagram as the project progresses.

Critical Path Method in SPM

The following diagram illustrates the basic architecture of a CPM in SPM using ASCII characters:





In this diagram, each letter represents an activity and each line represents a dependency. The duration of each activity is shown in parentheses. For example, activity A takes 10 days to complete.

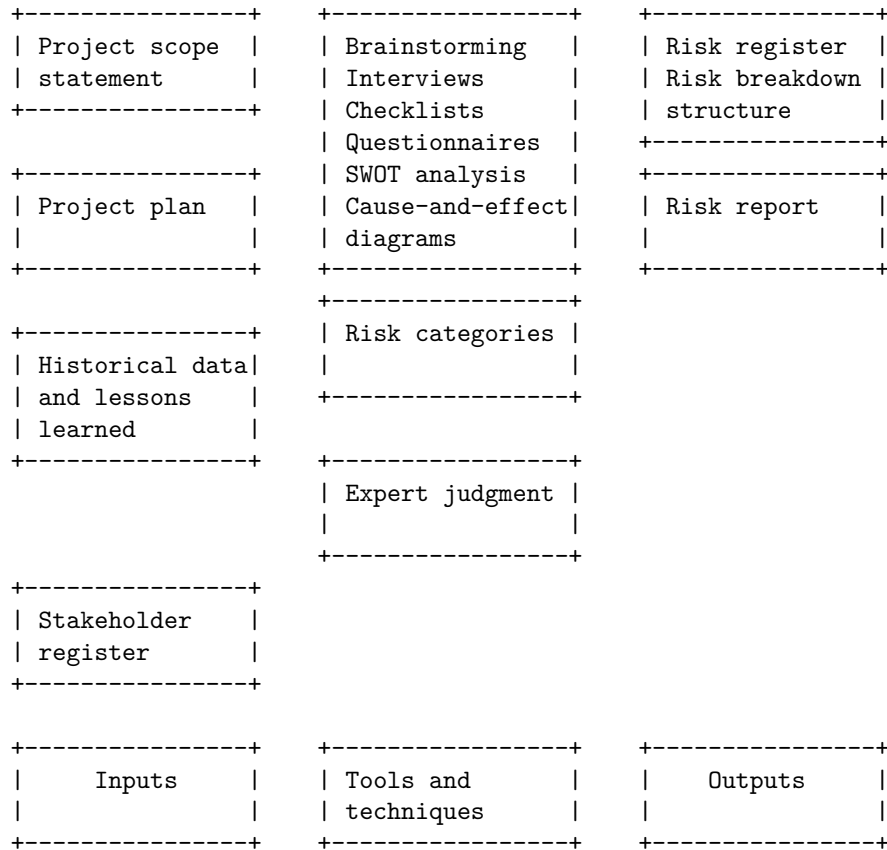
The critical path is the longest path in the network diagram, which determines the minimum time required to complete the project. In this case, the critical path is A-C-D-F-G, which has a total duration of $10 + 15 + 5 + 20 + 10 = 60$ days. Any delay in the critical path activities will cause the project to be delayed.

The other activities that are not on the critical path have some slack or float, which is the amount of time they can be delayed without affecting the project completion date. For example, activity B has a slack of 15 days, which means it can start 15 days later than planned or take 15 days longer than estimated without delaying the project. The slack of each activity can be calculated by subtracting the duration of the critical path from the duration of the other paths that include that activity. For example, the slack of activity B is $60 - (10 + 5 + 10 + 10) = 15$ days.

Risk identification is the process of determining potential risks that may affect the project objectives, scope, schedule, budget, quality, or stakeholders. It is an essential step in risk management, which aims to minimize the negative impacts and maximize the positive opportunities of uncertainty.

There are various techniques for risk identification, such as brainstorming, interviews, checklists, questionnaires, SWOT analysis, cause-and-effect diagrams, etc. The choice of technique depends on the nature, size, and complexity of the project, as well as the availability of resources and information.

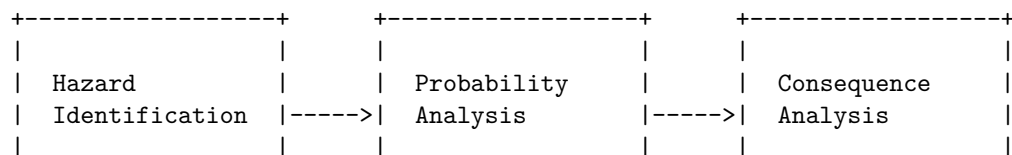
A common way to represent the risk identification process is by using a diagram that shows the inputs, tools and techniques, and outputs of the process. The following diagram illustrates the risk identification process in software project management, based on the Project Management Body of Knowledge (PMBOK) Guide:

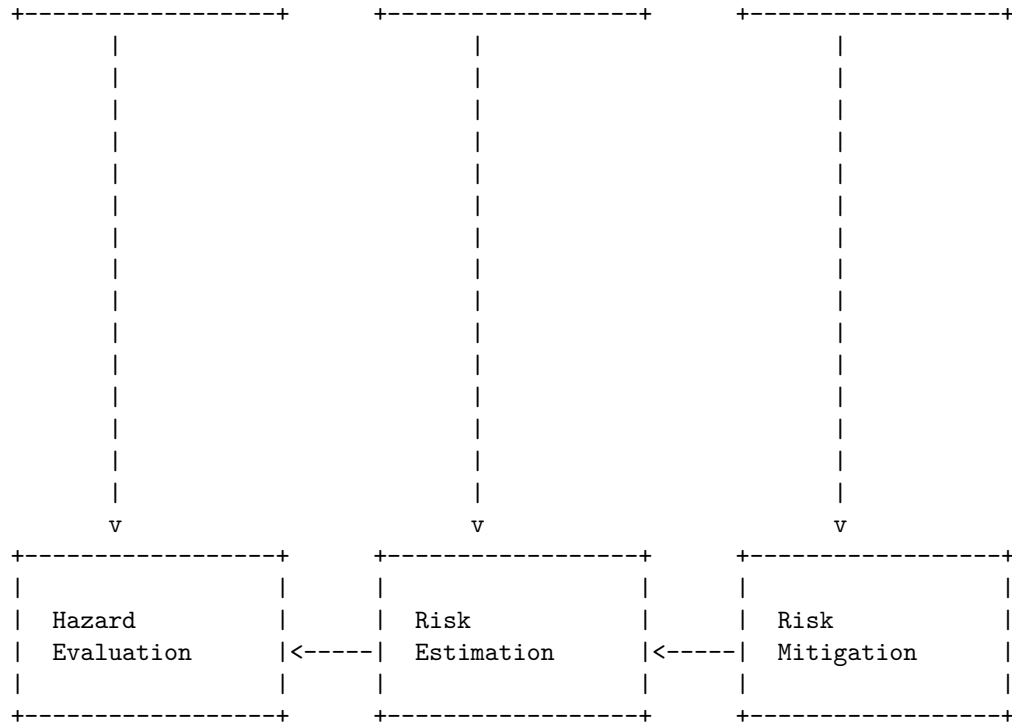


Risk assessment in spm is the process of identifying, analyzing, and evaluating the risks associated with the installation and operation of single point mooring (SPM) systems for subsea pipelines. SPM systems are offshore structures that allow the transfer of fluids between vessels and pipelines. They are exposed to various hazards such as environmental loads, corrosion, fatigue, third party interference, and human error. Risk assessment aims to quantify the probability and consequences of these hazards and to provide recommendations for risk mitigation and management.

Risk Assessment in spm

The following diagram illustrates the basic steps of risk assessment in spm using the DNVGL RP F107 standard and numerical simulations:





Hazard identification is the process of identifying all the possible sources of harm or damage to the SPM system and the subsea pipeline. This can be done by reviewing the design, operation, and maintenance data of the system, as well as historical records of failures and incidents. Some examples of hazards are anchor dragging, ship collision, wave loading, corrosion, and fatigue.

Probability analysis is the process of estimating the likelihood of occurrence of each hazard. This can be done by using statistical methods, empirical models, or numerical simulations. The probability analysis should consider the frequency, duration, and intensity of the hazard, as well as the exposure and vulnerability of the system.

Consequence analysis is the process of estimating the potential impacts of each hazard on the system and the environment. This can be done by using analytical models, numerical simulations, or expert judgment. The consequence analysis should consider the severity, extent, and duration of the damage, as well as the costs of repair, replacement, and environmental remediation.

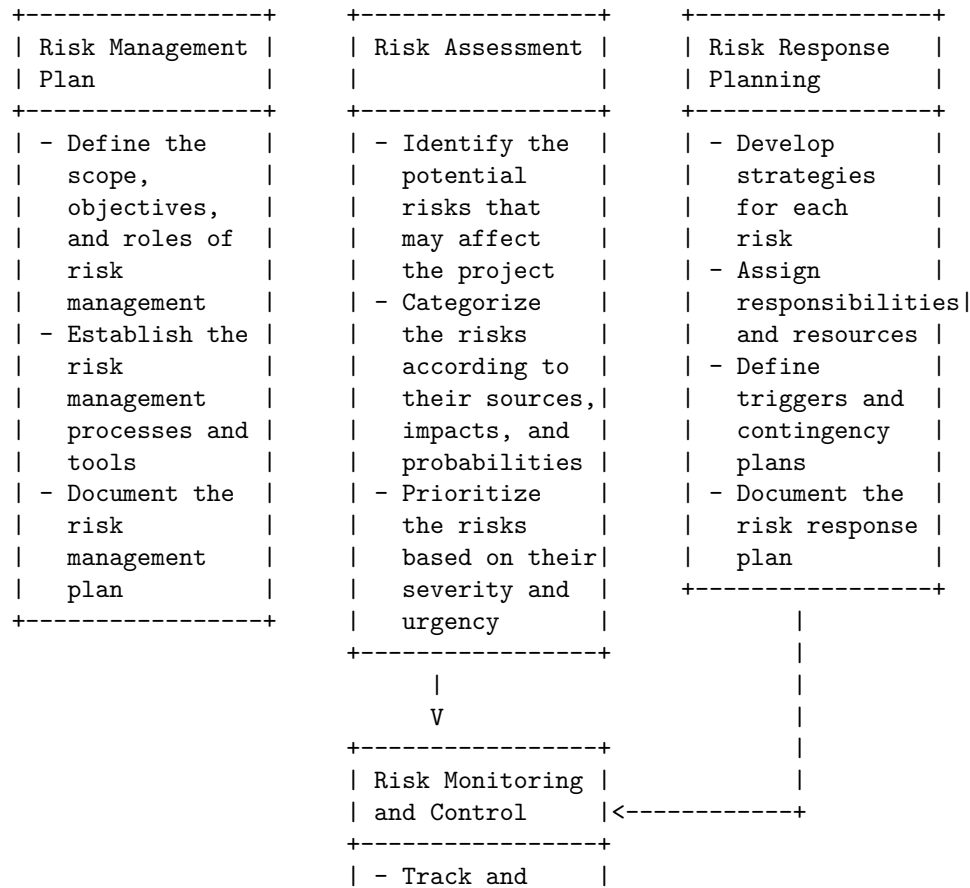
Risk estimation is the process of combining the probability and consequence of each hazard to obtain a measure of risk. This can be done by using qualitative or quantitative methods, such as risk matrices, risk indices, or risk curves. The risk estimation should reflect the uncertainty and variability of the input data and the assumptions.

Risk evaluation is the process of comparing the estimated risk with the acceptable risk criteria and determining whether the risk is tolerable or not. This can be done by using predefined standards, regulations, or guidelines, or by using stakeholder consultation and expert judgment. The risk evaluation should consider the technical, economic, social, and environmental aspects of the risk.

Risk mitigation is the process of reducing the risk to an acceptable level by implementing appropriate measures. This can be done by using the hierarchy of risk control, which prioritizes the elimination, substitution, engineering, administrative, and personal protective measures. The risk mitigation should consider the effectiveness, feasibility, and cost-benefit of the measures.

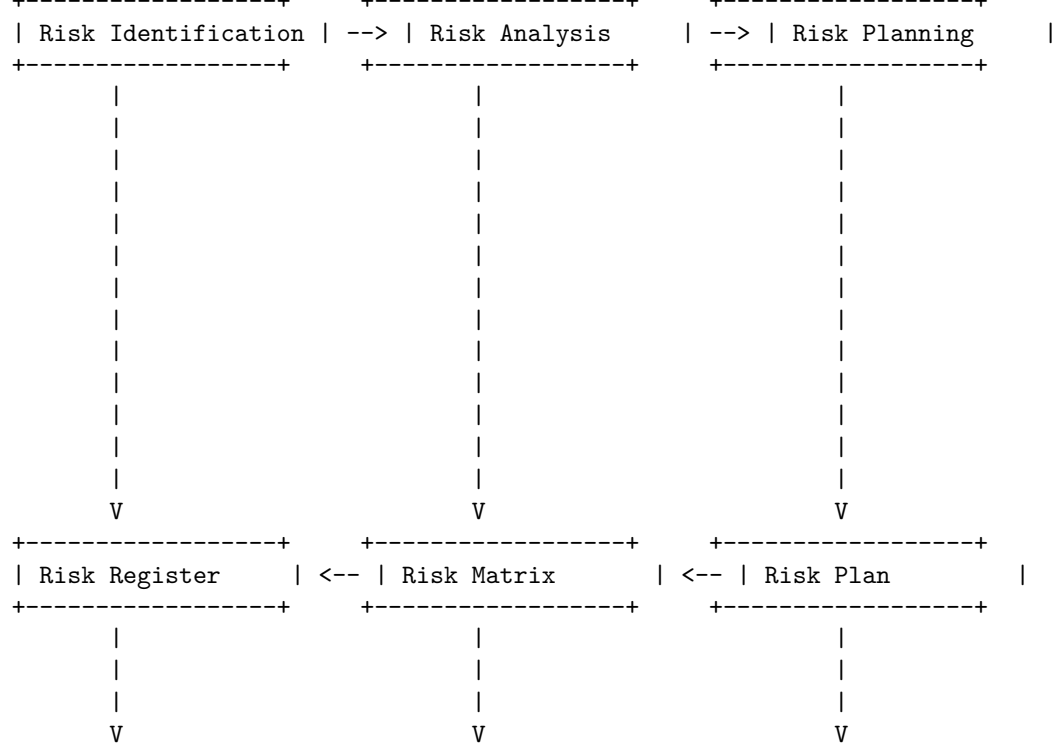
Risk planning in software project management (SPM) is the process of identifying, analyzing, and responding to the potential risks that may affect the success of a software project. Risk planning helps to prevent or minimize the impact of negative events and to maximize the opportunities of positive events.

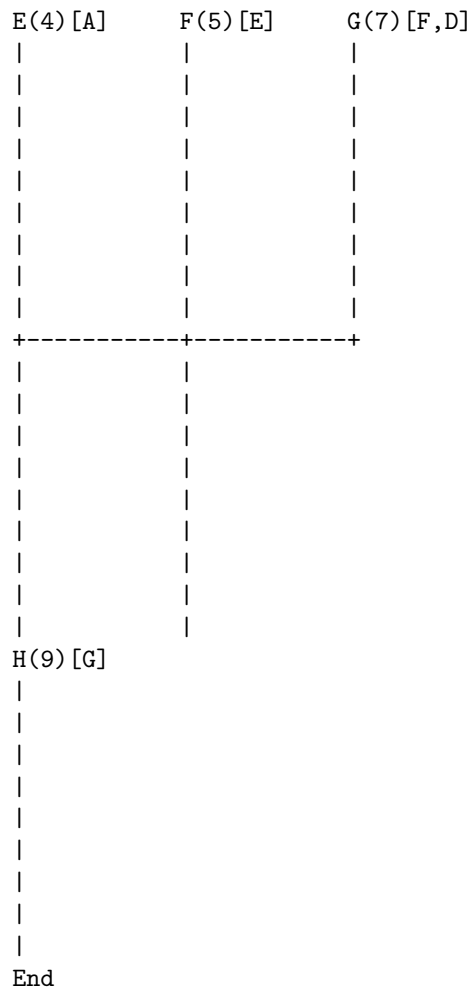
One possible way to draw a detailed ASCII diagram for risk planning in SPM is as follows:



one of the core tents of the philosophy of SPM (Software Project Management).

process in SPM using ASCII art:





The following table shows the calculations for the earliest start (ES), earliest finish (EF), latest start (LS), latest finish (LF) and slack time (SL) for each activity.

Activity	Duration	Predecessors	ES	EF	LS	LF	SL
A	10	-	0	10	0	10	0
B	12	A	10	22	10	22	0
C	8	B	22	30	22	30	0
D	6	C	30	36	30	36	0
E	4	A	10	14	14	18	4
F	5	E	14	19	18	23	4
G	7	F, D	36	43	36	43	0
H	9	G	43	52	43	52	0

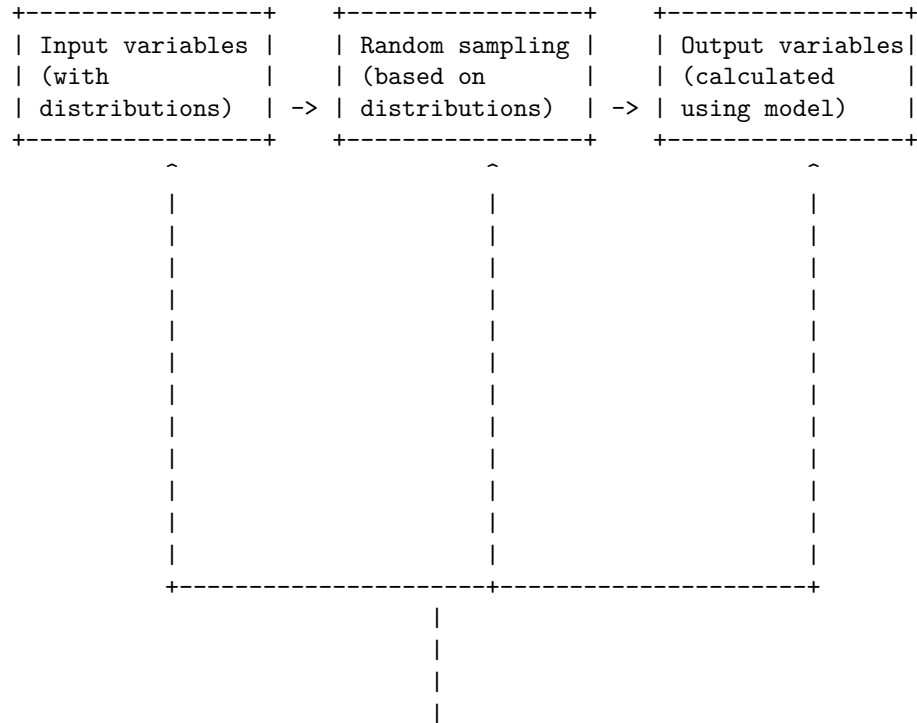
The critical path is A-B-C-D-G-H and the project completion time is 52 days. The activities E and F have a slack time of 4 days, which means they can be delayed by up to 4 days without affecting the project completion time. The activities A, B, C, D, G and H have a slack time of 0 days, which means they cannot be delayed at all without affecting the project completion time.

A Monte Carlo simulation is a mathematical technique that allows you to account for risks in decision-making. It helps determine the impact of identified risks by running multiple simulations and finding a range of outcomes. Every decision has a degree of uncertainty, and the Monte Carlo simulation helps you make decisions in such situations .

The basic steps of a Monte Carlo simulation are:

1. Define the problem and the possible outcomes.
2. Identify the input variables and their probability distributions.
3. Generate random values for the input variables based on their distributions.
4. Calculate the output variables using a mathematical model or a formula.
5. Repeat steps 3 and 4 for a large number of times (iterations).
6. Analyze the results and draw conclusions.

The following diagram illustrates the basic architecture of a Monte Carlo simulation:



+-----+ +-----+ +-----+

The diagram shows the following steps:

- The project scope defines the goals, deliverables, and requirements of the project.
- The resource demand estimates the amount and type of resources needed for the project.
- The resource supply identifies the availability and cost of resources from internal and external sources.
- The work breakdown structure (WBS) decomposes the project scope into manageable tasks and subtasks.
- The resource plan assigns resources to each task and subtask based on the resource demand and supply.
- The resource pool is the collection of all resources that can be used for the project.
- The resource schedule allocates resources to specific time slots and durations based on the resource plan and the project timeline.
- The resource tracking monitors and reports the actual usage and performance of resources against the resource schedule.

The creation of critical paths in software project management (SPM) is a method to identify the sequence of tasks that determines the minimum duration of a project. It is based on the critical path method (CPM), which is an algorithm for planning, managing and analyzing the timing of a project .

The following steps are involved in creating a critical path in SPM:

1. Define the project scope and objectives.
2. Break down the project into smaller tasks or activities.
3. Estimate the duration and resources required for each task.
4. Identify the dependencies and constraints among the tasks.
5. Draw a network diagram that shows the logical sequence of the tasks.
6. Calculate the earliest start and finish times, and the latest start and finish times for each task.
7. Identify the critical tasks and the critical path, which are the tasks and the path that have zero slack or float, meaning they cannot be delayed without affecting the project completion date.
8. Monitor and control the progress of the project and adjust the schedule if necessary.

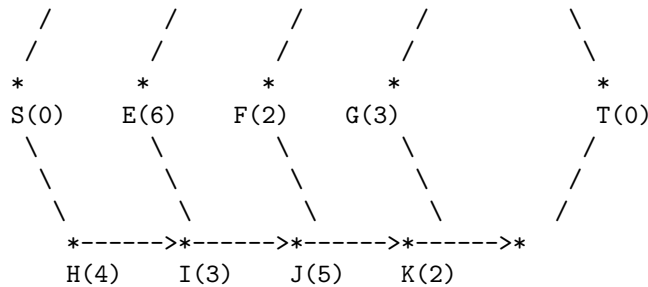
The following diagram illustrates the creation of a critical path in SPM using an example project:

Creation of Critical Paths in SPM

```

A(5)    B(3)    C(4)    D(2)
*----->*----->*----->*----->*
/         /         /         /         \

```



The network diagram shows the tasks (A to K) and their durations (in parentheses) for a project that involves developing a software product. The start and finish nodes (S and T) have zero duration. The arrows indicate the dependencies among the tasks. For example, task A must be completed before task B can start.

The earliest start (ES) and finish (EF) times are calculated by adding the task durations along the forward path from the start node to the finish node. The latest start (LS) and finish (LF) times are calculated by subtracting the task durations along the backward path from the finish node to the start node. The slack or float (SL) of a task is the difference between its latest and earliest start times, or between its latest and earliest finish times. It represents the amount of time that a task can be delayed without affecting the project completion date.

The critical path is the longest path in the network diagram, which has the same duration as the project. It is highlighted in bold in the diagram. The critical tasks are the tasks that lie on the critical path. They have zero slack or float, meaning they cannot be delayed without affecting the project completion date. In this example, the critical path is S-A-B-C-D-T, and the critical tasks are A, B, C and D. The project duration is 14 days, which is the sum of the durations of the critical tasks.

The creation of a critical path in SPM helps to identify the most important tasks, optimize the use of resources, and manage the risks and uncertainties of the project. It also helps to track and control the progress of the project and make adjustments if necessary.

A cost schedule is a detailed plan that shows the estimated costs and the allocation of resources for each activity or task in a software project. It helps the project manager to monitor the project progress, control the budget, and identify any cost variances. A cost schedule can be represented by a table or a chart that shows the breakdown of costs by categories, such as labor, materials, equipment, etc. A cost schedule can also show the cumulative costs and the percentage of completion for each activity or task.

One possible way to draw a cost schedule in ASCII is to use the following symbols:

- for vertical lines

- – for horizontal lines
- – for intersections
- / for diagonal lines
- – for dots
- for arrows
-

for shading

An example of a cost schedule in ASCII for a software project with four activities (A, B, C, and D) is shown below:

Activity	Labor Cost	Material Cost	Equipment Cost	Total Cost
A	\$10,000	\$5,000	\$2,000	\$17,000
B	\$15,000	\$3,000	\$1,000	\$19,000
C	\$12,000	\$4,000	\$3,000	\$19,000
D	\$8,000	\$6,000	\$4,000	\$18,000
Total	\$45,000	\$18,000	\$10,000	\$73,000

Activity	Cumulative Cost	% of Completion	Start Date	End Date
A	\$17,000	23%	01/01/2023	01/15/2023
B	\$36,000	49%	01/16/2023	01/31/2023
C	\$55,000	75%	02/01/2023	02/15/2023
D	\$73,000	100%	02/16/2023	02/28/2023

Activity	Predecessors	Successors	Duration	Resources
A	None	B	15 days	2 developers

B	A	C, D	15 days	3 developers	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
C	B	None	15 days	2 developers	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
D	B	None	12 days	2 developers	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
Time	A	B	C	D	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
01/01/2023	*				
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
01/02/2023	*				
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
01/03/2023	*				
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
01/04/2023	*				

The following is a detailed ASCII diagram for Unit - 4 - Project Management and Control for the notes of the course Software Project Management in spm.

The diagram shows the main components of the framework for management and control, which are:

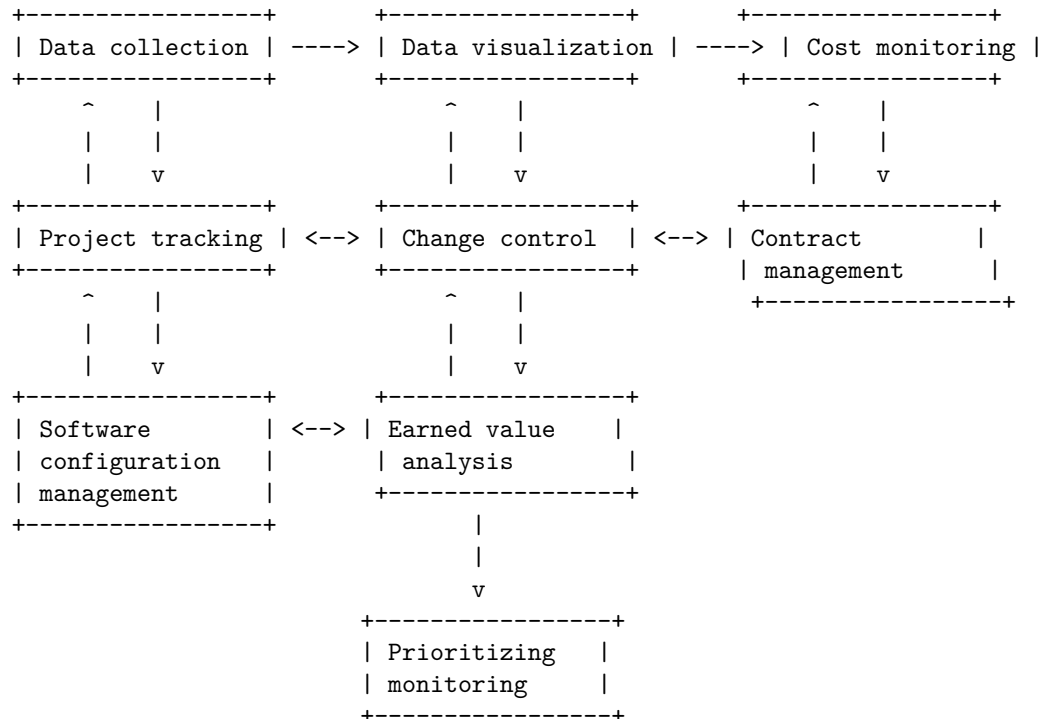
- Data collection: The process of gathering relevant information about the project's progress, costs, quality, risks, issues, and changes.
- Data visualization: The process of presenting the collected data in a clear and concise way, using charts, graphs, tables, dashboards, or reports.
- Cost monitoring: The process of tracking and comparing the actual and planned costs of the project, using techniques such as budget variance analysis, earned value analysis, or cost performance index.
- Earned value analysis: A technique that integrates scope, time, and cost data to measure the project's performance and progress, using metrics such as schedule variance, cost variance, schedule performance index, cost performance index, or estimate at completion.
- Prioritizing monitoring: The process of identifying and focusing on the most critical aspects of the project, such as the key deliverables, milestones, risks, issues, or changes, using techniques such as risk matrix, issue log, or change log.
- Project tracking: The process of updating and adjusting the project plan and schedule based on the actual performance and progress of the project, using techniques such as critical path analysis, network diagram, or Gantt chart.
- Change control: The process of managing and controlling the changes that occur during the project, such as scope changes, requirement changes, or design changes, using techniques such as change request, change impact analysis, or change approval.

- Software configuration management: The process of identifying, organizing, and controlling the software products and artifacts that are produced and modified during the project, such as code, documents, or test cases, using techniques such as version control, baseline, or audit.
- Contract management: The process of managing and controlling the contracts that are involved in the project, such as contracts with customers, suppliers, or subcontractors, using techniques such as contract negotiation, contract administration, or contract closure.

The diagram also shows the relationships and interactions among the components, using arrows and symbols, such as:

- $\rightarrow$: Indicates a one-way flow of data or information from one component to another.
- $\leftrightarrow$: Indicates a two-way flow of data or information between two components.
- $+$: Indicates a combination or integration of data or information from two or more components.
- $*$: Indicates a multiplication or calculation of data or information from two or more components.

The diagram is as follows:



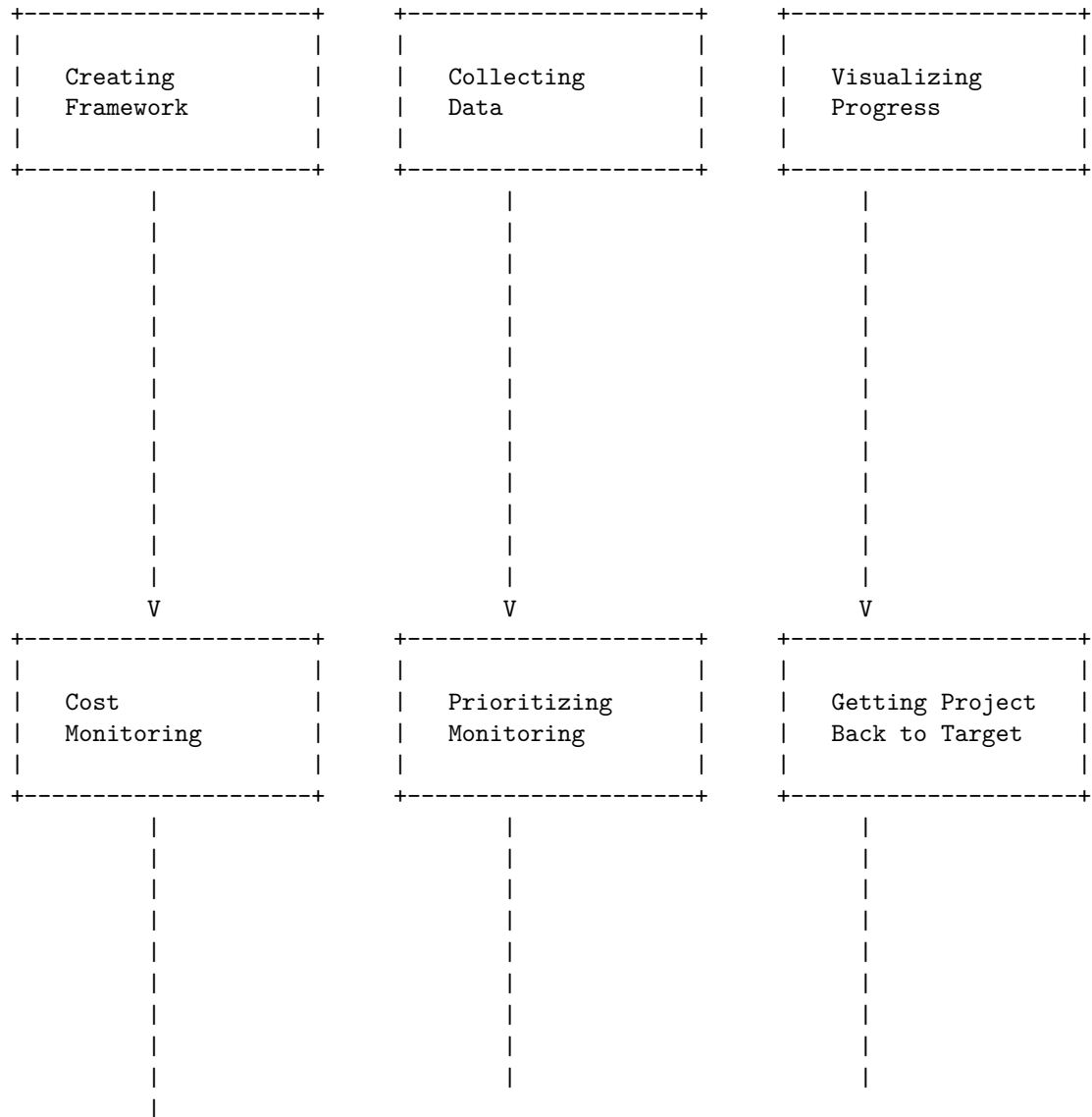
The Framework for Management and Control in SPM is a set of processes and

tools that help software project managers monitor and control the progress, quality, cost, and scope of their projects. It consists of four main components:

- **Creating Framework:** This involves defining the project objectives, scope, deliverables, milestones, tasks, resources, risks, and quality standards, and establishing a baseline plan and budget for the project. It also involves setting up a communication and reporting system, and assigning roles and responsibilities to the project team and stakeholders.
- **Collecting Data:** This involves gathering and recording information about the project activities, such as the actual time, cost, and quality of the work done, the status of the tasks and deliverables, the issues and changes encountered, and the feedback and satisfaction of the customers and users.
- **Visualizing Progress:** This involves analyzing and presenting the collected data in a clear and meaningful way, such as using charts, graphs, dashboards, and reports, to show the project performance, variance, trends, and risks. It also involves comparing the actual data with the planned data, and identifying any deviations, gaps, or problems that need to be addressed.
- **Cost Monitoring:** This involves tracking and controlling the project expenses, such as the labor, material, equipment, and overhead costs, and ensuring that they are within the allocated budget and the expected return on investment. It also involves using techniques such as earned value management, which measures the value of the work completed against the planned value and the actual cost, to assess the project efficiency and profitability.
- **Prioritizing Monitoring:** This involves focusing on the most critical and important aspects of the project, such as the key deliverables, milestones, risks, and quality standards, and ensuring that they are given the highest priority and attention. It also involves using techniques such as the Pareto principle, which states that 80% of the effects come from 20% of the causes, to identify and eliminate the major sources of problems and waste.
- **Getting Project Back to Target:** This involves taking corrective and preventive actions to resolve the issues and problems that affect the project performance, quality, cost, and scope, and to bring the project back on track with the planned objectives and expectations. It also involves managing the changes and requests that arise during the project, and ensuring that they are evaluated, approved, and implemented in a controlled and systematic way.
- **Change Control:** This involves establishing and following a formal process for managing the changes and requests that occur during the project, such as the changes in the requirements, scope, schedule, budget, or quality standards. It also involves documenting and communicating the changes and their impacts, and updating the project plan and baseline accordingly.
- **Managing Contracts:** This involves negotiating and administering the contracts and agreements that are involved in the project, such as the contracts with the customers, suppliers, vendors, and subcontractors. It also

involves ensuring that the contractual terms and conditions are met, and that the deliverables and payments are exchanged as agreed.

The following diagram illustrates the basic architecture of the Framework for Management and Control in SPM using ASCII art:



Collection of Data in spm

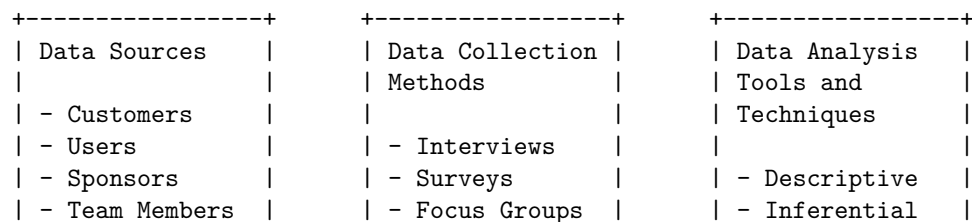
Collection of data in software project management (SPM) is the process of gathering and analysing information related to the project's objectives, scope,

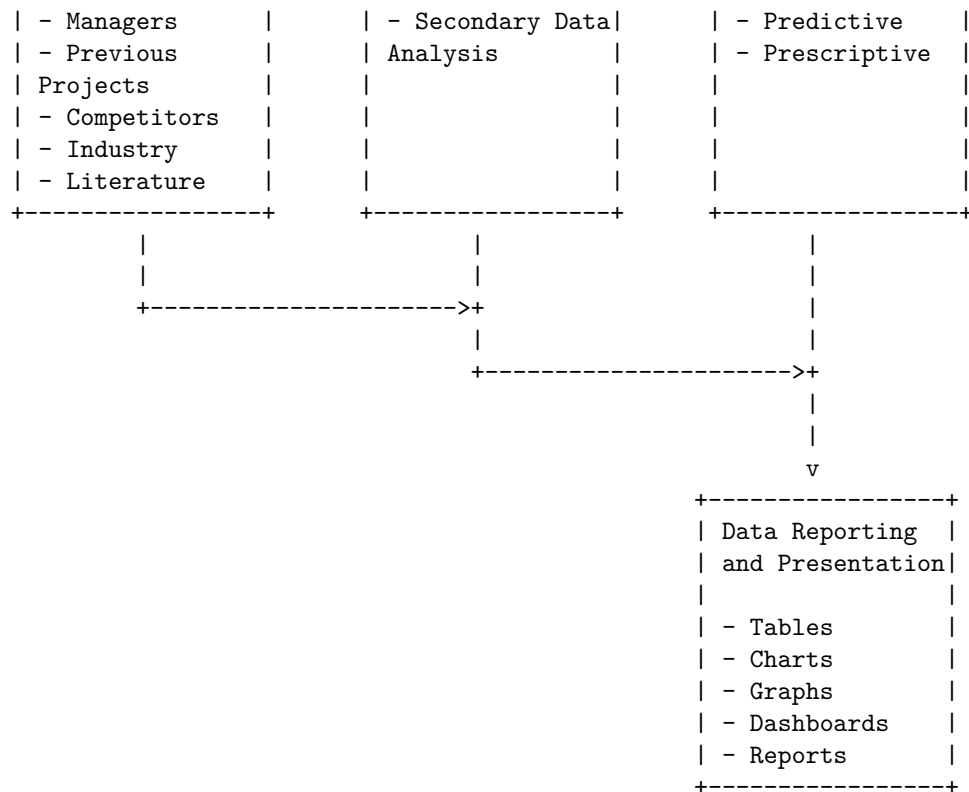
schedule, cost, quality, risk, and performance. Data collection methods in SPM can vary depending on the type and purpose of the data, the sources and availability of the data, and the tools and techniques used to collect and analyse the data.

Some of the common data collection methods in SPM are:

- **Interviews:** Personal interviews are a great way to collect data in SPM. These can be conducted in person, over the phone, or online. Interviews can be structured, semi-structured, or unstructured, depending on the level of flexibility and detail required. Interviews can help to gather qualitative data, such as opinions, perceptions, feedback, and suggestions from the project stakeholders, such as the customers, users, sponsors, team members, and managers.
- **Surveys:** Surveys are a common method used to collect data in SPM. This can include online surveys, paper surveys, or even in-person surveys. Surveys can help to gather quantitative data, such as ratings, rankings, scores, and frequencies from a large and representative sample of the project stakeholders. Surveys can be designed using various types of questions, such as closed-ended, open-ended, or Likert-scale questions, depending on the type and level of data required.
- **Focus Groups:** Focus groups are an important method of collecting data in SPM. This involves bringing together a small group of people who share a common interest or characteristic related to the project, such as the potential users, customers, or experts. Focus groups can help to gather qualitative data, such as insights, opinions, preferences, and experiences from the group members, through a moderated discussion or a structured exercise. Focus groups can also help to generate new ideas, identify issues, and explore solutions for the project.
- **Secondary Data Analysis:** Secondary data analysis is a way to collect data in SPM without the need for primary data collection. This type of data collection involves analysing data that has already been collected by someone else, such as the previous projects, the competitors, the industry, or the literature. Secondary data analysis can help to gather quantitative or qualitative data, such as trends, patterns, benchmarks, and best practices from the existing sources of data. Secondary data analysis can also help to save time, cost, and resources for the project.

The following diagram illustrates the basic architecture of a data collection process in SPM:



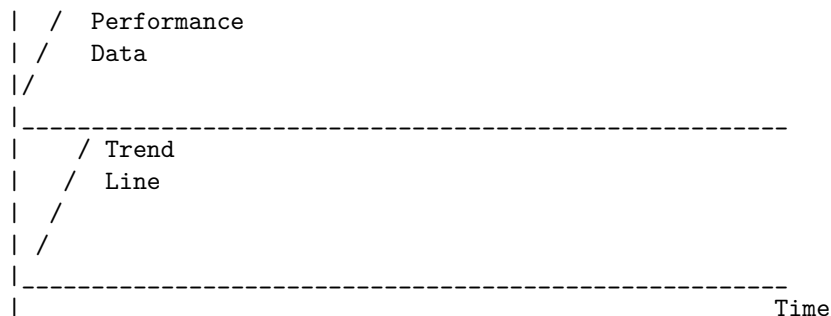


Visualizing Progress in spm is the process of tracking and displaying the progress of students or projects towards achieving their goals and objectives. One of the common methods of visualizing progress in spm is using a graph that shows the actual performance of the student or project over time, compared to a desired or expected performance level. The graph can also indicate whether the student or project is in need of an instructional or strategic change to improve their learning or outcomes.

Visualizing Progress in spm

The following diagram illustrates the basic structure of a graph for visualizing progress in spm:





The graph has four main components:

- The expected performance level, which is a horizontal line that represents the desired or target level of performance for the student or project. This can be based on benchmarks, standards, or goals that are set by the teacher, manager, or stakeholder.
- The actual performance data, which are data points that represent the measured or observed performance of the student or project at different points in time. These can be based on assessments, tests, deliverables, or milestones that are collected and recorded regularly.
- The trend line, which is a line that connects the actual performance data points and shows the direction and rate of change of the performance over time. The trend line can be calculated using various methods, such as linear regression, moving average, or exponential smoothing.
- The time axis, which is a horizontal line that represents the duration of the monitoring period or the project cycle. The time axis can be divided into intervals, such as weeks, months, or quarters, depending on the frequency and granularity of the data collection and analysis.

The graph can be used to visualize the progress of the student or project and to make informed decisions based on the data. For example, the graph can help to answer questions such as:

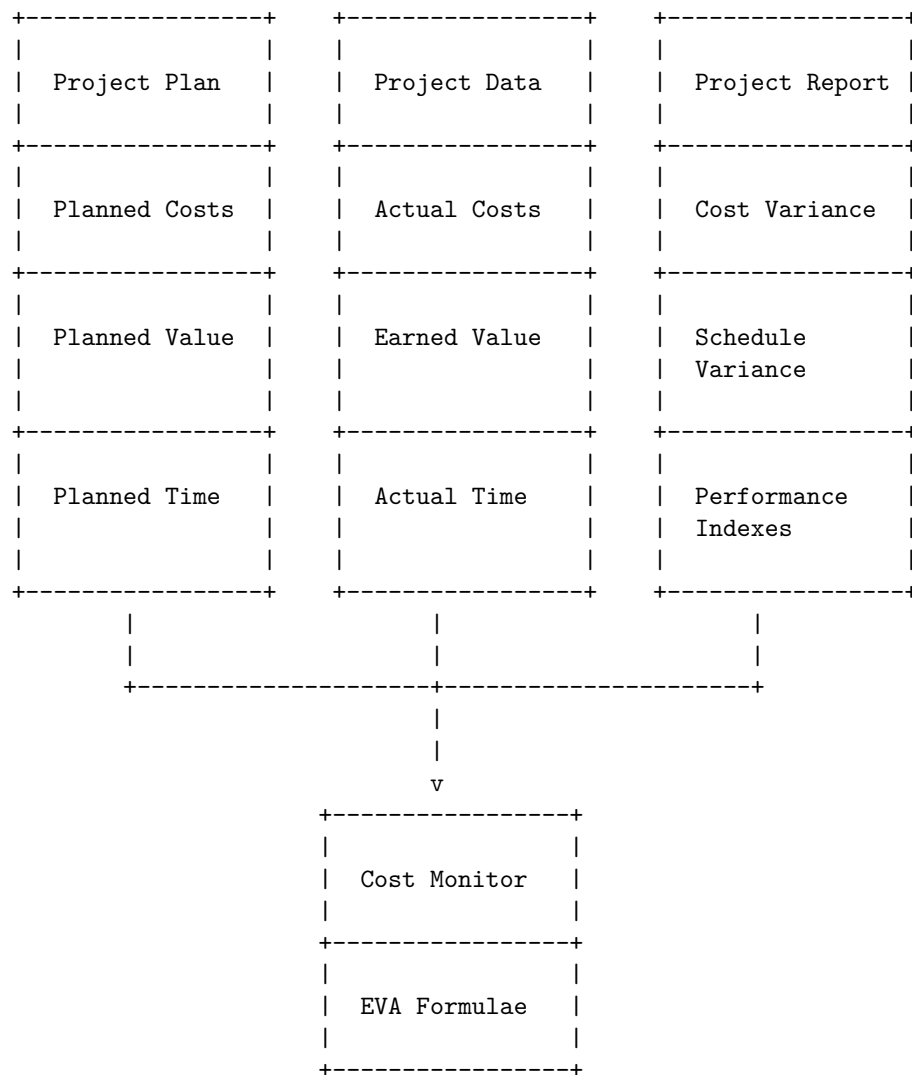
- Is the student or project performing at, above, or below the expected performance level?
- Is the student or project making enough progress to reach the expected performance level by the end of the monitoring period or the project cycle?
- Is the student or project showing a positive, negative, or flat trend in their performance over time?
- Is the student or project in need of an instructional or strategic change to improve their performance or to address any challenges or opportunities?

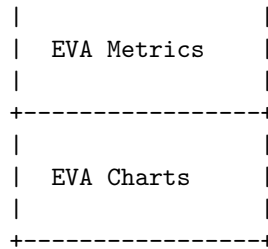
The graph can also be used to communicate the progress of the student or project to other stakeholders, such as parents, sponsors, or clients, and to provide feedback and recognition to the student or project team. The graph can be updated and shared regularly, such as weekly, monthly, or quarterly, depending on the needs and preferences of the stakeholders.

Cost monitoring in software project management is the process of tracking the costs associated with a software project, like the cost of development, the cost of maintenance, and any associated expenses. This process helps project managers to track progress and make sure they stay within budget.

One of the techniques used for cost monitoring is Earned Value Analysis (EVA), which provides an integrated view of the project by measuring planned effort (costs), actual progress (earned value), and effort (actual costs) in terms of monetary values.

The following diagram illustrates the basic architecture of a cost monitoring system in software project management using EVA:

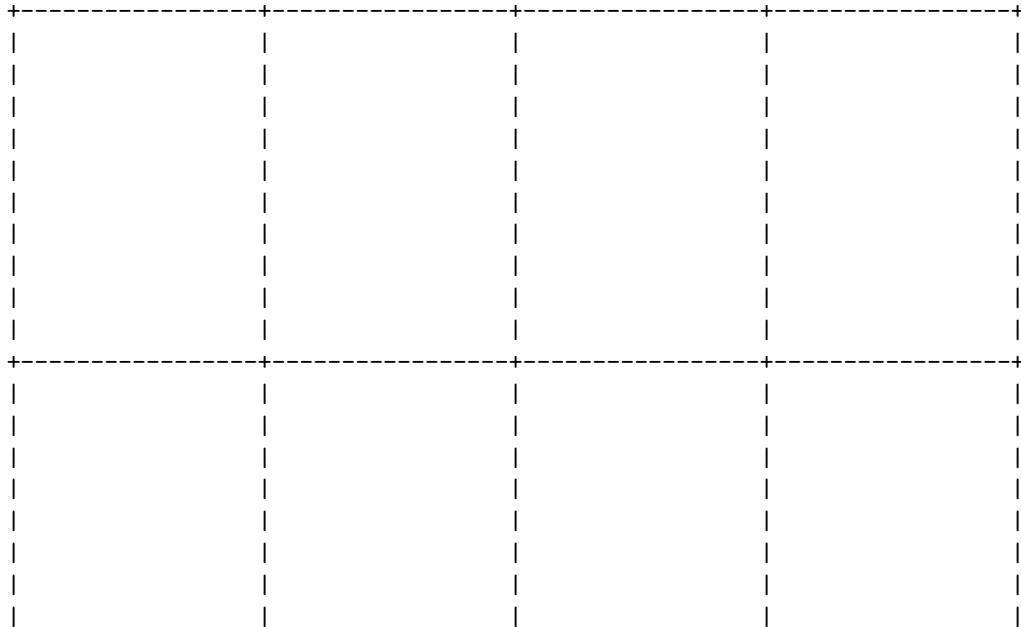


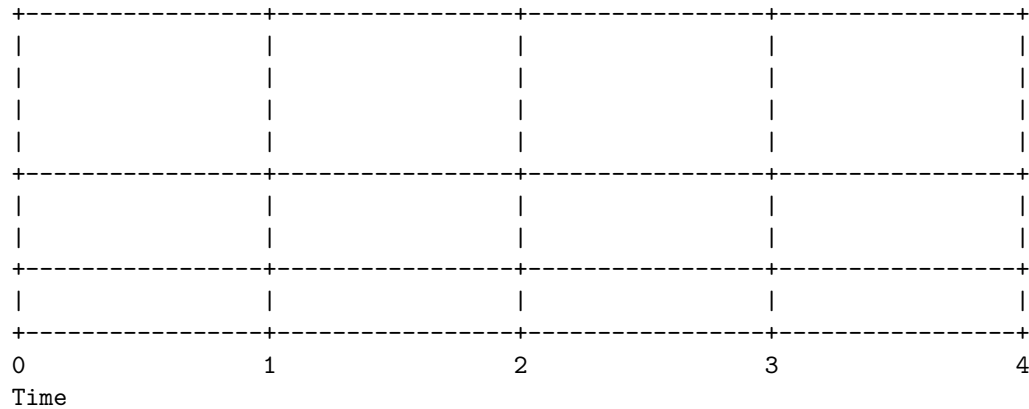


The cost monitor is a software tool that collects the project data from various sources, such as the project plan, the project management software, the accounting software, etc. It then applies the EVA formulae to calculate the EVA metrics, such as the cost variance, the schedule variance, the cost performance index, the schedule performance index, etc. It also generates the EVA charts, such as the S-curve, the cumulative cost curve, the earned value curve, etc. These charts help the project manager to visualize the project performance and identify any deviations from the plan. The cost monitor then produces the project report, which summarizes the project status and provides recommendations for corrective actions if needed.

Earned Value Analysis (EVA) is a project management technique that allows to measure the performance and progress of a project by comparing the planned effort (costs), the actual effort (costs), and the earned value (work completed) in terms of monetary values . EVA helps to identify deviations from the project baseline and to take corrective actions if needed .

The following diagram illustrates the basic concepts of EVA in spm:





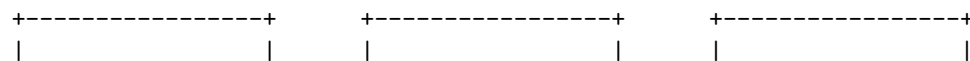
Legend:

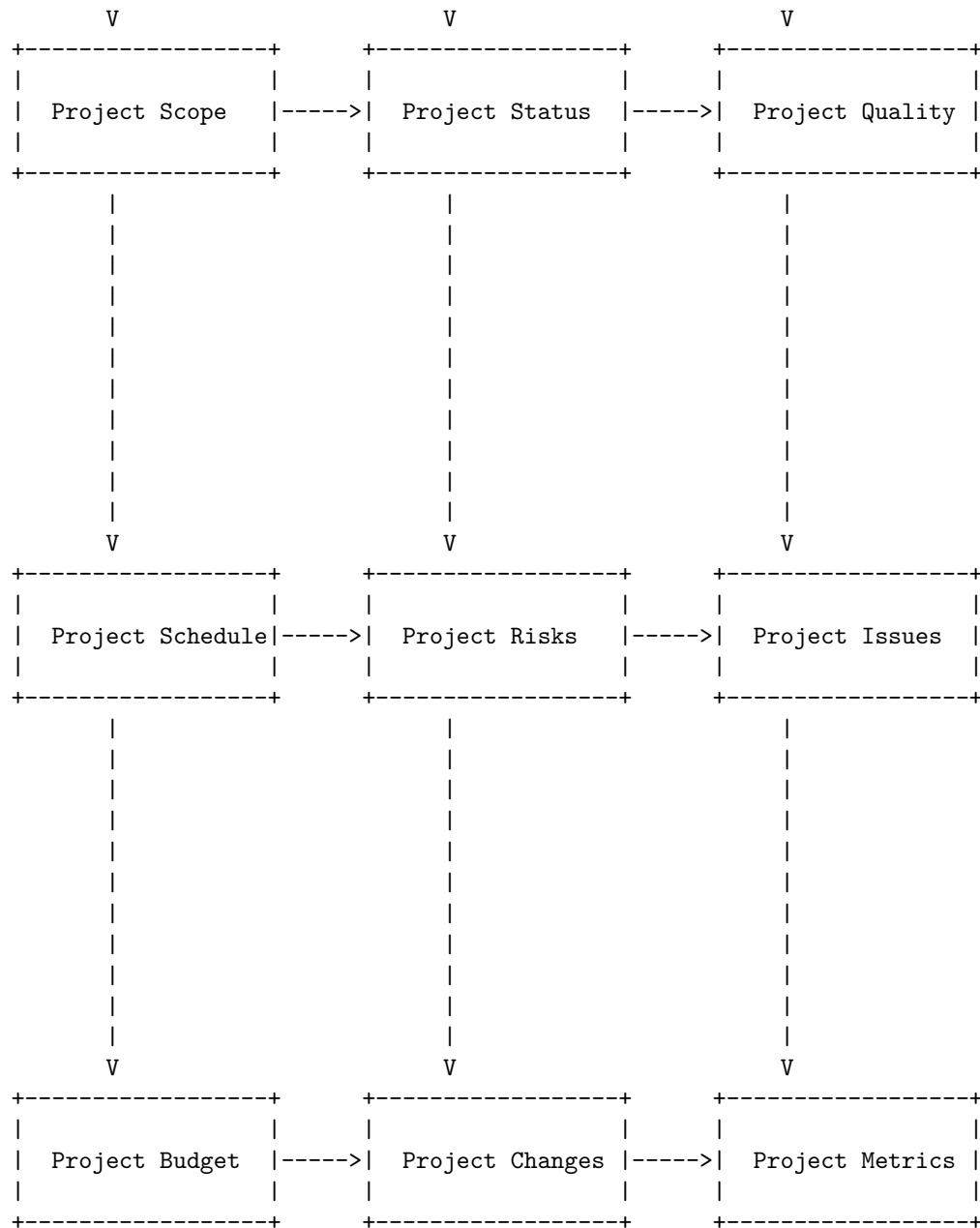
- The horizontal axis represents the time in units (e.g. weeks, months, etc.)
- The vertical axis represents the monetary value in units (e.g. dollars, euros, etc.)
- The blue line represents the planned value (PV), which is the budgeted cost of work scheduled (BCWS)
- The red line represents the actual cost (AC), which is the actual cost of work performed (ACWP)
- The green line represents the earned value (EV), which is the budgeted cost of work performed (BCWP)
- The yellow area represents the cost variance (CV), which is the difference between EV and PV
- The purple area represents the schedule variance (SV), which is the difference between EV and PV
- The orange area represents the variance at completion (VAC), which is the difference between EV and AC
- The dashed line represents the EAC, which is the projected cost of the project at completion

Prioritizing monitoring is the process of organizing the order in which tasks and activities need to be monitored in a software project. This helps to ensure that the most important tasks and activities get the most attention and that resources and efforts are being used efficiently. Prioritizing monitoring involves the following steps:

- Collection: You must collect and gather all the data about your projects, such as scope, schedule, budget, quality, risks, and stakeholders' expectations.
- Ranking: You must develop and use a ranking model that includes criteria for prioritizing, such as urgency, importance, impact, dependency, and complexity. You must assign weights and scores to each criterion and each project, and then rank the projects according to their total scores.
- Verification: You must approve the ranked projects and communicate them to the project team and stakeholders. You must also review and update the ranking periodically to reflect any changes in the project environment or requirements.

The following diagram illustrates the basic architecture of prioritizing monitoring in spm using ASCII art:



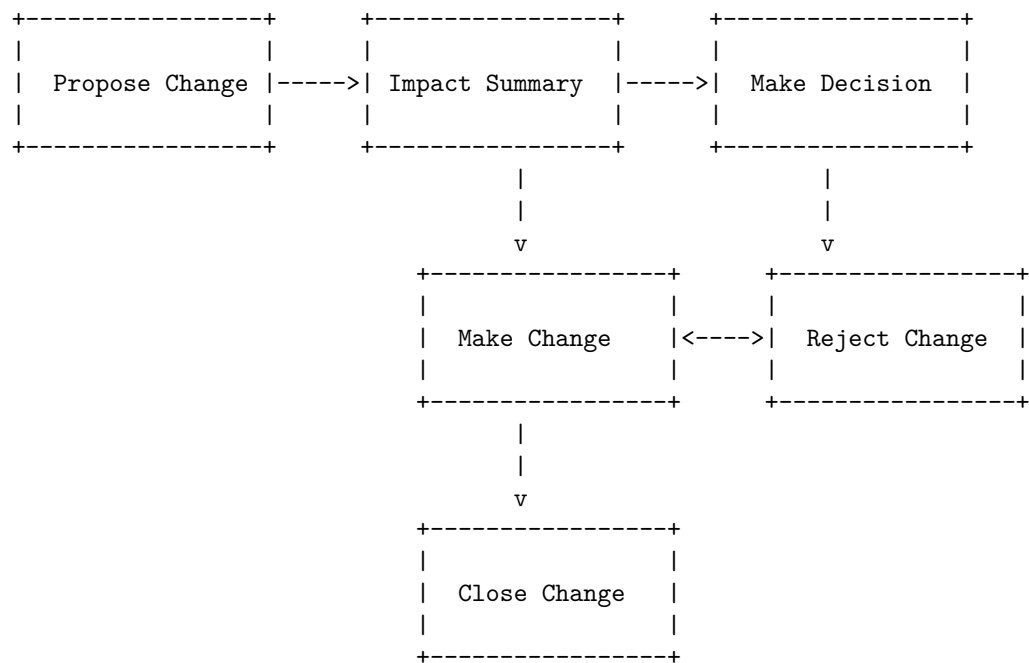


The diagram shows the main components and relationships of Project Tracking in spm. The Project Plan defines the objectives, scope, schedule, budget, and risks of the project. The Project Data collects and stores information on the project activities, deliverables, and outcomes. The Project Reports generate and communicate the project performance, status, and quality to the stakeholders.

The Project Scope, Schedule, Budget, and Changes are the four dimensions of the project that need to be managed and controlled. The Project Status, Risks, Issues, and Metrics are the four indicators of the project health and progress that need to be monitored and analyzed. The Project Quality is the degree to which the project meets the customer requirements and expectations. The arrows show the flow of information and feedback among the components.

Change control is a process that ensures that any changes to a project or a system are introduced in a controlled and coordinated way. Change control helps to minimize the risk of disruption, errors, or defects in the project or system. Change control in spm (strategic portfolio management) is a way of managing changes to the strategic objectives, plans, and initiatives of an organization. Change control in spm helps to align the work with the strategy, prioritize the most valuable initiatives, and respond quickly to changing needs or opportunities.

A possible diagram for change control in spm is:



The diagram shows the basic steps of change control in spm:

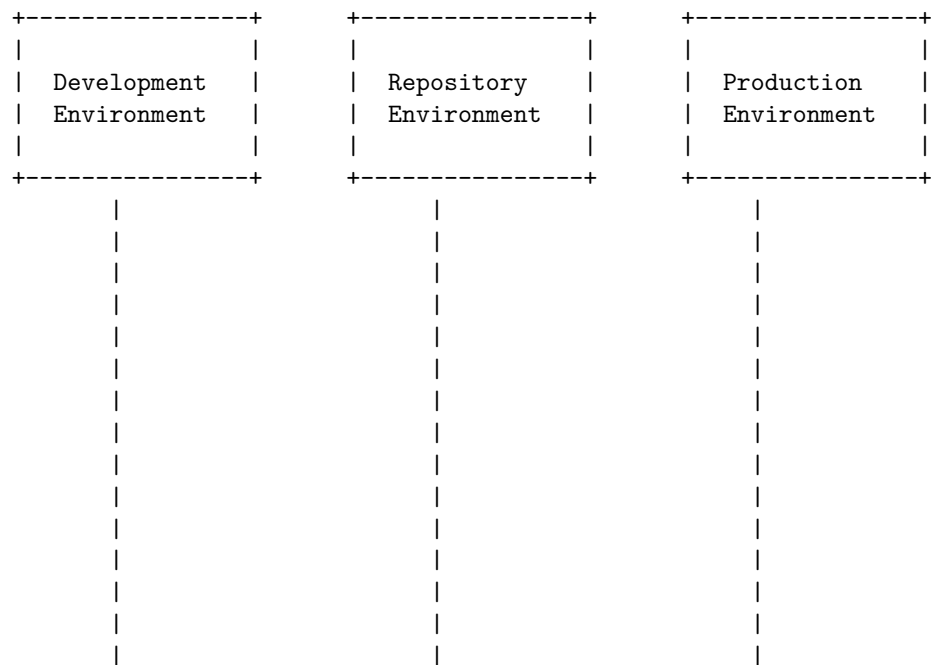
- **Propose Change:** A change request is submitted by anyone who wants to introduce a change to the strategy, plan, or initiative. The change request should include the reason, scope, impact, and benefits of the change.
- **Impact Summary:** The change request is reviewed by the relevant stakeholders, such as the portfolio manager, the project manager, or the sponsor. The impact summary should assess the feasibility, cost, risk, and value of

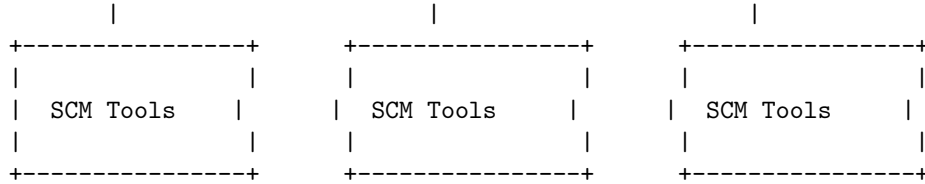
the change, as well as its alignment with the strategy and the dependencies with other initiatives.

- **Make Decision:** The decision to approve or reject the change request is made by the authorized person or group, such as the portfolio board, the steering committee, or the sponsor. The decision should be based on the impact summary and the strategic priorities of the organization.
- **Make Change:** If the change request is approved, the change is implemented according to the agreed plan and schedule. The change should be communicated to all the affected parties and monitored for its effectiveness and outcomes.
- **Reject Change:** If the change request is rejected, the reason for the rejection is communicated to the requester and the stakeholders. The change request is closed and archived for future reference.
- **Close Change:** Once the change is completed or rejected, the change request is closed and documented. The lessons learned from the change process are captured and shared for continuous improvement.

Software Configuration Management (SCM) is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle. SCM practices include revision control, baselines, configuration audits, change management, and release management.

The following diagram illustrates the basic architecture of a SCM system using ASCII art:

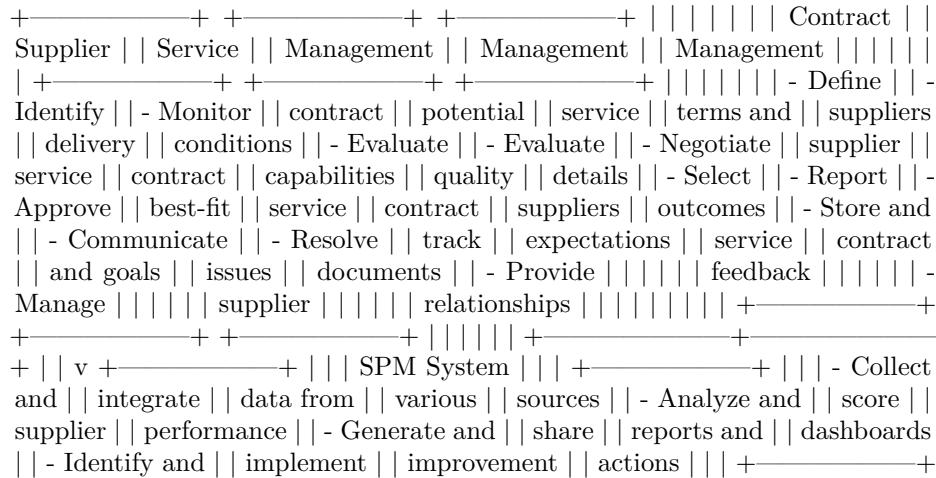




The development environment is where the software developers create, modify, and test the software components. The repository environment is where the software components are stored, versioned, and tracked. The production environment is where the software components are deployed, executed, and maintained. The SCM tools are the software applications that support the SCM activities in each environment.

Managing contracts in spm is the process of monitoring and evaluating the performance of suppliers who provide services to an organization. SPM stands for supplier performance management, which is a method of assessing how well suppliers meet the expectations and requirements of their customers. SPM involves setting goals, measuring outcomes, providing feedback, and taking corrective actions if needed.

The following diagram illustrates the basic architecture of a spm system using ASCII characters:

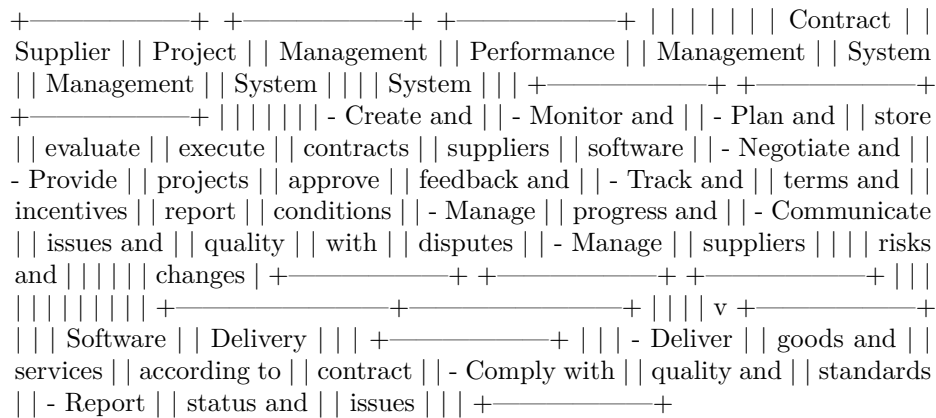


The diagram shows how the three main components of spm (contract management, supplier management, and service management) interact with each other and with the spm system. The spm system collects and analyzes data from various sources, such as contracts, invoices, surveys, audits, and service level agreements, to measure and evaluate the performance of suppliers. The spm system also generates and shares reports and dashboards that show the results of the analysis and the scores of the suppliers. The spm system also helps identify and implement improvement actions, such as renegotiating contracts,

providing feedback, or terminating contracts, if the suppliers do not meet the expectations and requirements of the organization.

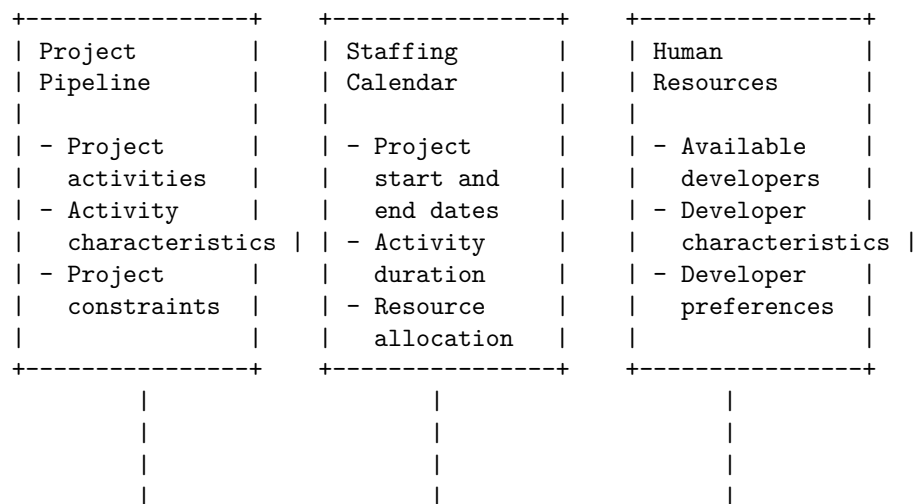
Contract management in spm is the process of managing the performance of suppliers and vendors in a software project. It involves defining the terms and conditions of the contract, monitoring the delivery of goods and services, evaluating the quality and compliance of the supplier, and resolving any issues or disputes that may arise.

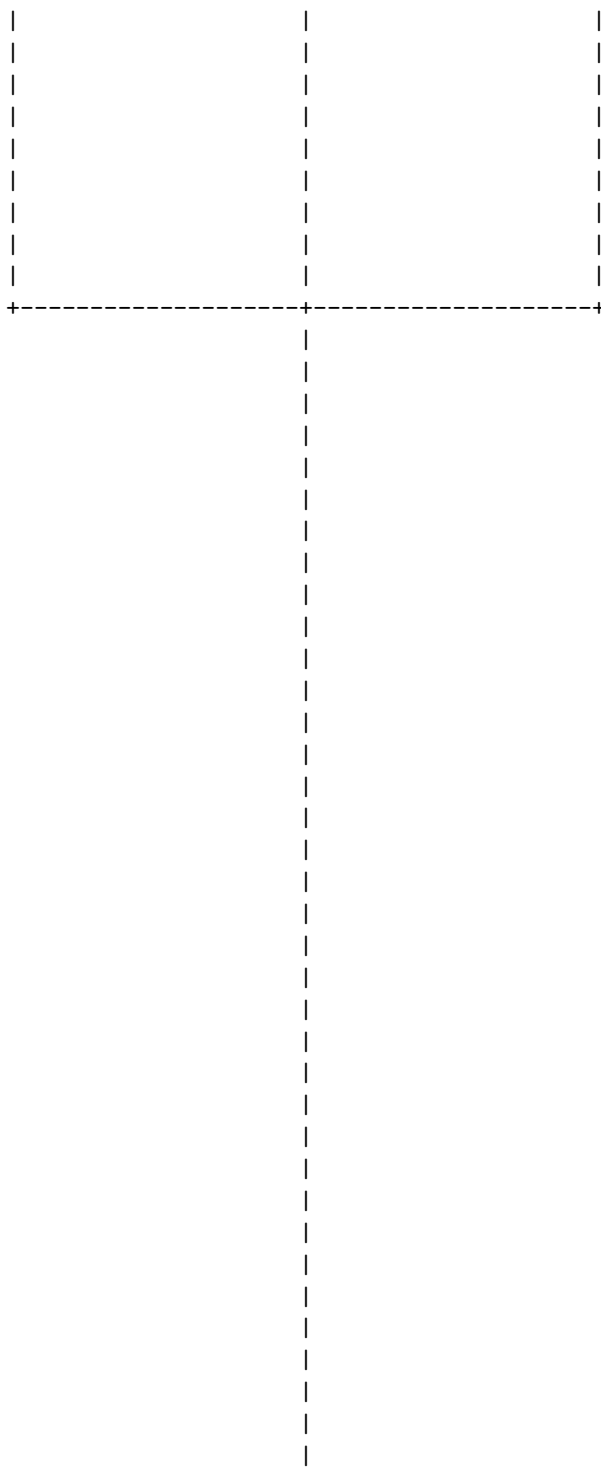
The following diagram illustrates the basic architecture of a contract management system in spm using ASCII characters:



The following is a detailed ASCII diagram for Unit - 5 - Staffing in Software Projects for the notes of the course Software Project Management in spm. The diagram is based on the information from the search results .

Unit - 5 - Staffing in Software Projects







Managing people in spm is the process of selecting, motivating, and developing the human resources of a software project. It involves planning, organizing, leading, and controlling the activities of individuals and teams to achieve the project goals and objectives.

The following diagram illustrates the basic architecture of managing people in

[illegible]

Organizational behavior (OB) is the study of how people behave in organizational work environments. It draws on concepts and theories from various disciplines, such as psychology, sociology, and anthropology, to understand and predict human behavior at work. OB covers three main levels of analysis: individual, group, and organizational.

Organizational Behavior in spm

Organizational Level	Group Level	Individual Level
Organizational Structure	Group Structure and Design	Individual Characteristics

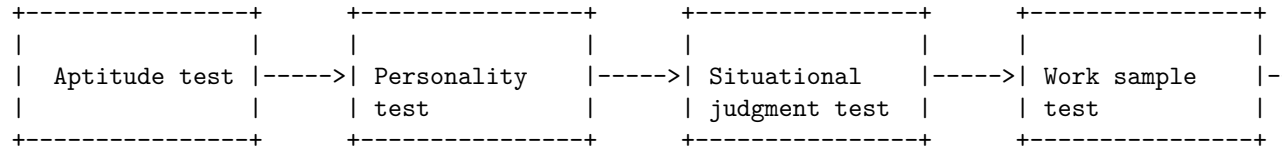
	Organizational Culture				Group Dynamics and Processes				Individual Behavior		
	+-----+				+-----+				+-----+		
	+-----+				+-----+				+-----+		
	Organizational Outcomes				Group Outcomes				Individual Outcomes		
	+-----+				+-----+				+-----+		
	+-----+				+-----+				+-----+		

There are various methods of staff selection in software project management (SPM), depending on the specific requirements and objectives of the project. Some of the common methods are:

- **Aptitude tests:** These are standardized tests that measure the cognitive abilities and skills of the candidates, such as numerical reasoning, verbal reasoning, logical reasoning, spatial reasoning, etc. Aptitude tests can help to screen out candidates who do not meet the minimum criteria for the job, and to identify those who have the potential to learn and perform well in the project.
- **Personality tests:** These are tests that assess the personality traits and preferences of the candidates, such as extraversion, agreeableness, conscientiousness, openness, and neuroticism. Personality tests can help to evaluate the fit between the candidates and the organizational culture, the team dynamics, and the job role. Personality tests can also help to predict the behavior and motivation of the candidates in different situations.
- **Situational judgment tests (SJTs):** These are tests that present the candidates with realistic scenarios and problems that they may encounter in the project, and ask them to choose the best or most appropriate course of action from a set of options. SJTs can help to measure the judgment, decision making, problem solving, and interpersonal skills of the candidates, as well as their knowledge and understanding of the project context and objectives.
- **Work samples:** These are tests that require the candidates to perform tasks or produce outputs that are similar or relevant to the project, such as writing code, designing a user interface, testing a software, etc. Work samples can help to assess the actual performance and quality of the candidates, as well as their creativity, innovation, and technical competence.
- **Interviews:** These are the most commonly used method of staff selection, where the candidates are asked questions by one or more interviewers, either face-to-face, over the phone, or online. Interviews can help to verify the information provided by the candidates in their resumes and tests, as well as to explore their personality, motivation, communication, and

teamwork skills. Interviews can also help to establish rapport and trust between the candidates and the interviewers, and to clarify any doubts or concerns.

The following diagram illustrates the basic architecture of a staff selection process in SPM, using the above methods as examples:

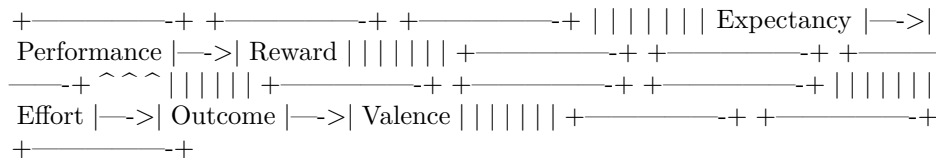


The order and number of the methods may vary depending on the project and the organization, but the general principle is to use a combination of methods that can provide a comprehensive and accurate evaluation of the candidates, and to select the best person for the job.

Motivation in SPM refers to the factors that influence the software project managers and their team members to perform their tasks effectively and efficiently. Motivation can be intrinsic or extrinsic, depending on the source and nature of the rewards. Intrinsic motivation comes from within the individual, such as personal interest, satisfaction, or enjoyment. Extrinsic motivation comes from outside the individual, such as salary, recognition, or promotion.

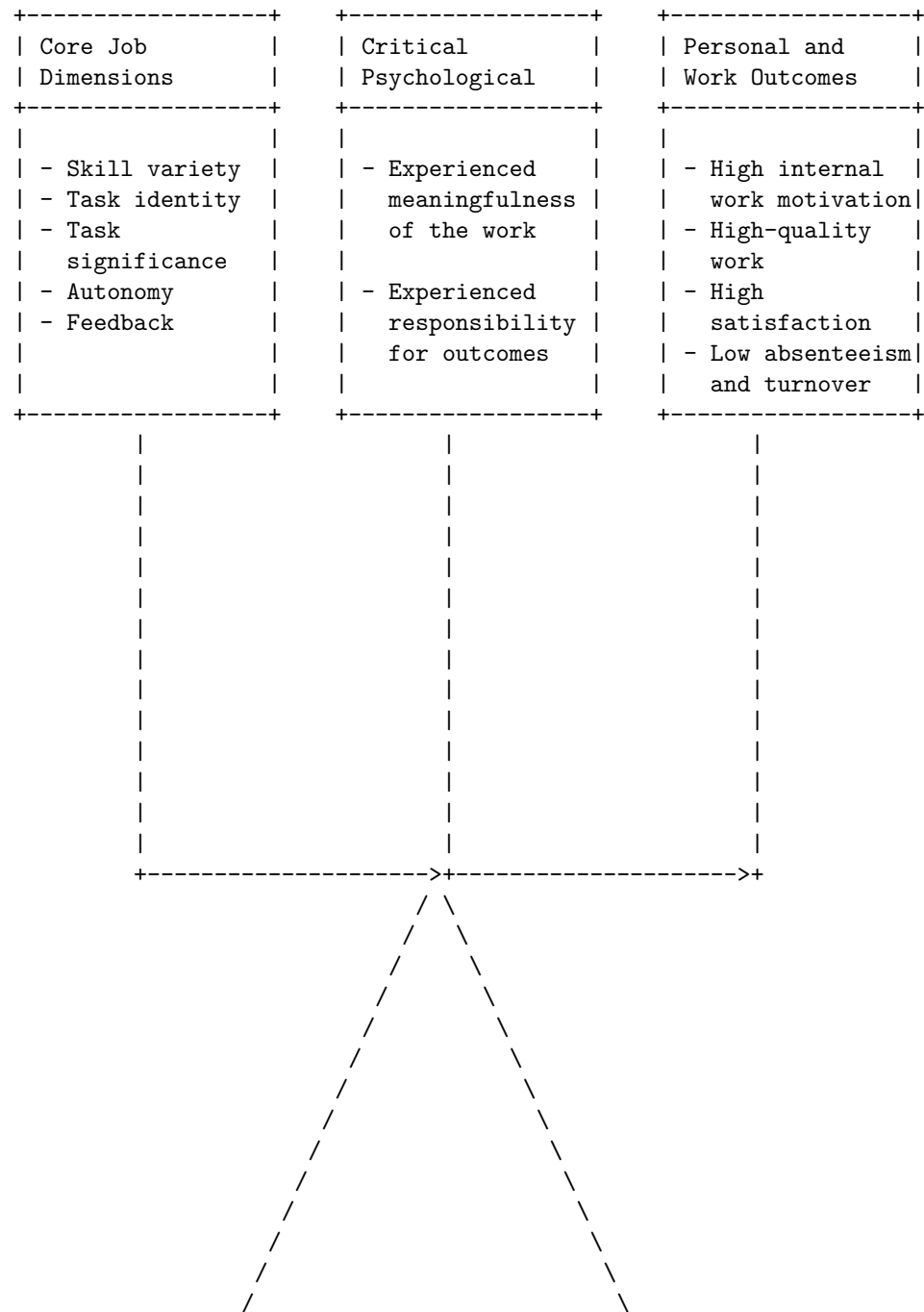
One of the models of motivation that can be applied to SPM is the expectancy theory, developed by Vroom and his colleagues. This theory states that motivation is a function of three variables: expectancy, instrumentality, and valence. Expectancy is the belief that working harder will lead to a better performance. Instrumentality is the belief that better performance will be rewarded. Valence is the perceived value of the resulting reward. The motivation level is high when all three variables are high, and low when any of them is low.

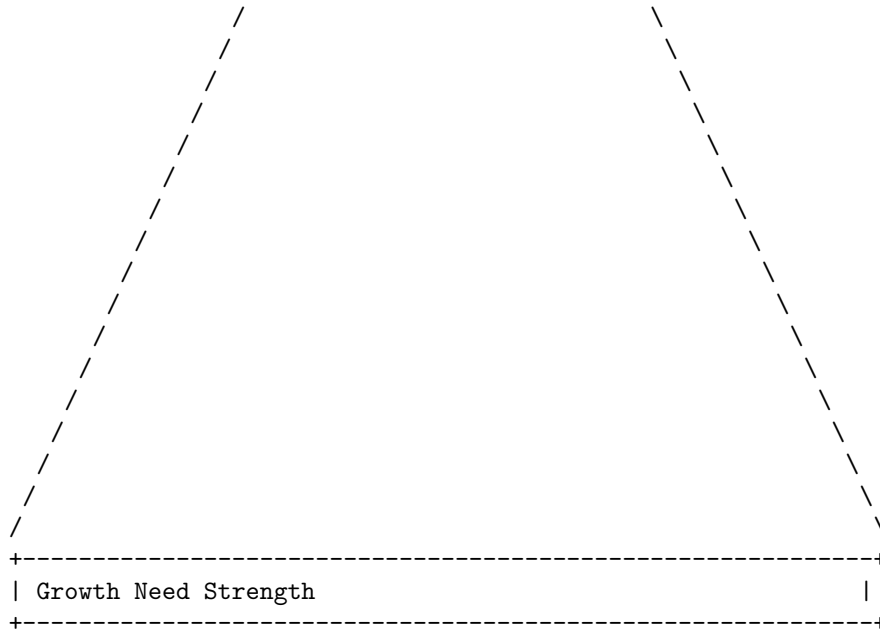
The following diagram illustrates the basic architecture of the expectancy theory of motivation in SPM using ASCII characters:



The Oldham-Hackman Job Characteristic Model in spm is a framework that proposes that certain job characteristics can enhance the intrinsic motivation, satisfaction, and performance of employees. The model identifies five core job dimensions, three critical psychological states, and four personal and work outcomes. The model also suggests that the relationship between the job dimensions and the outcomes is moderated by the individual's growth need strength, which is the degree to which a person values personal development and learning.

The following diagram illustrates the basic architecture of the model using ASCII characters:



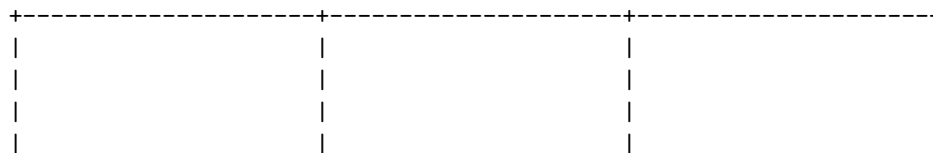


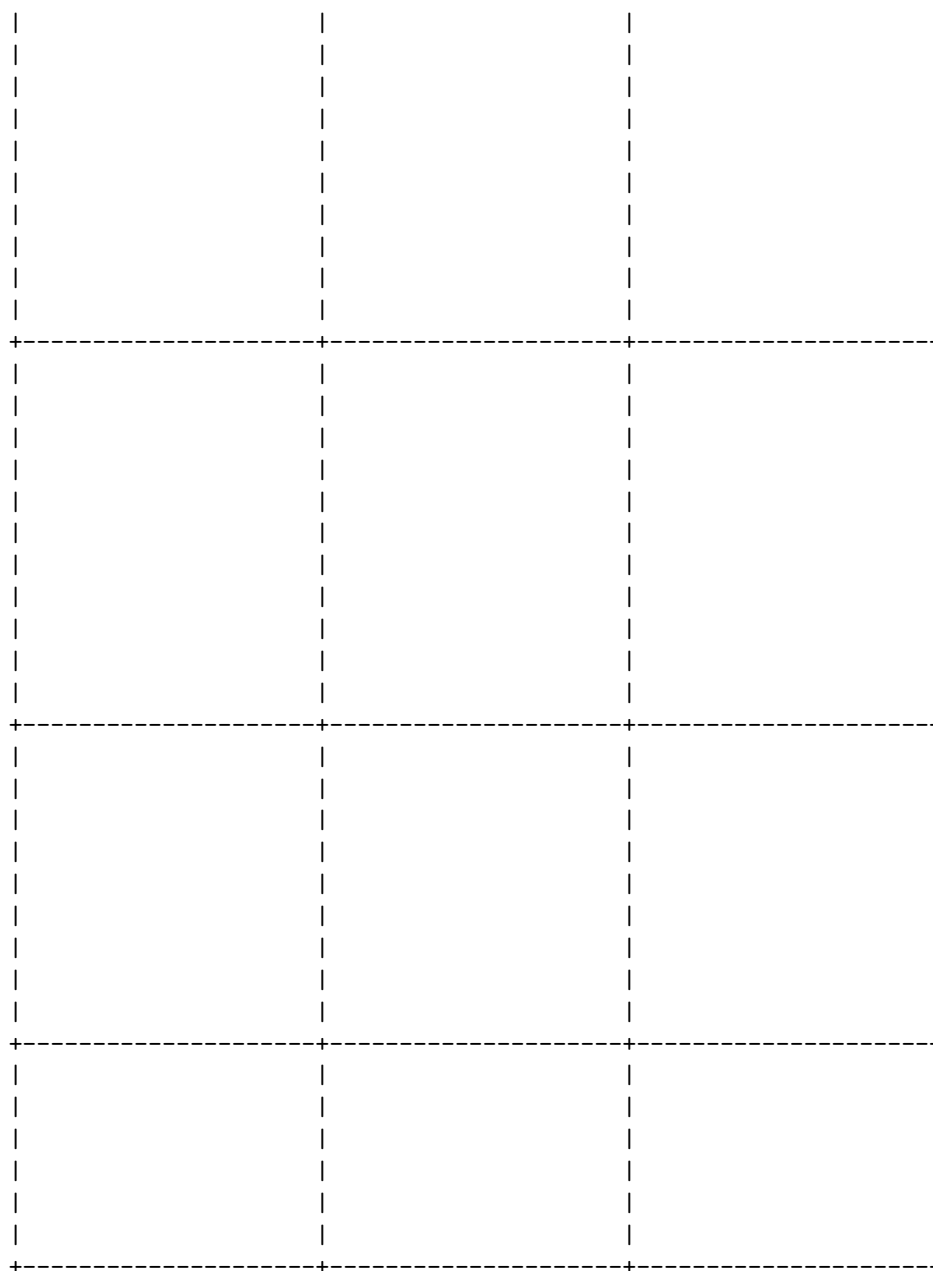
Stress in spm can refer to different contexts, such as stress in project management, stress in students taking the Sijil Pelajaran Malaysia (SPM) examination, or stress in single particle models (SPM) of lithium-ion batteries. I will assume you are asking about the last one, since it involves a diagram. If not, please specify which context you are interested in.

A single particle model (SPM) is a simplified way of modeling the electrochemical and thermal behavior of a lithium-ion battery. It assumes that the battery is composed of spherical particles of active material that undergo intercalation and deintercalation of lithium ions during charging and discharging. The SPM can be coupled with a mechanical stress sub-model that captures the stress developed in the particles due to the volume change associated with the lithium ion insertion and extraction.

The following diagram illustrates the basic architecture of a SPM with mechanical stress sub-model, using ASCII art. The diagram shows the main variables and equations involved in the model, as well as the boundary and initial conditions. The diagram is based on the paper by Mohtat et al. .

Stress in spm





Anode Region	Separator Region	Cathode Region	Particle Region	Stress Region
-----------------	---------------------	-------------------	--------------------	------------------

Variables:

ϕ_s : Solid phase potential
 ϕ_e : Electrolyte phase potential
 c_s : Solid phase concentration
 c_e : Electrolyte phase concentration
 j : Electrolyte phase current density
 i : Applied current density
 u : Displacement
 σ : Stress
 ϵ : Strain

Equations:

$\phi_s' = -a_s i / F$
 $\phi_e'' = -j / \kappa$
 $c_s'' = -(1/r_s) j$
 $c_e'' = -(a_s i / F) / (\epsilon_s \epsilon_e)$
 $j = -(2 \kappa c_e^{1/2} \sinh((F/2RT)(\phi_s - \phi_e - u_s)))$
 $u'' = (6E\epsilon) / (r_s^2)$
 $\sigma = E\epsilon$
 $\epsilon = u' / r_s + (1/2)(c_s / c_{\max} - 1)^2$

Boundary conditions:

$\phi_s(0) = \phi_a$
 $\phi_s(L_a) = \phi_e(L_a)$
 $\phi_e(0) = \phi_e(L_a)$
 $\phi_e(L_a) = \phi_e(L_a + L_s)$
 $\phi_e(L_a + L_s) = \phi_e(L_a + L_s + L_c)$
 $\phi_e(L_a + L_s + L_c) = \phi_c$
 $c_s(0) = c_{s0}$
 $c_s(r_s) = c_{sb}$
 $c_e(0) = c_{e0}$
 $c_e(L_a + L_s + L_c) = c_{e0}$
 $u(0) = 0$
 $u(r_s) = 0$

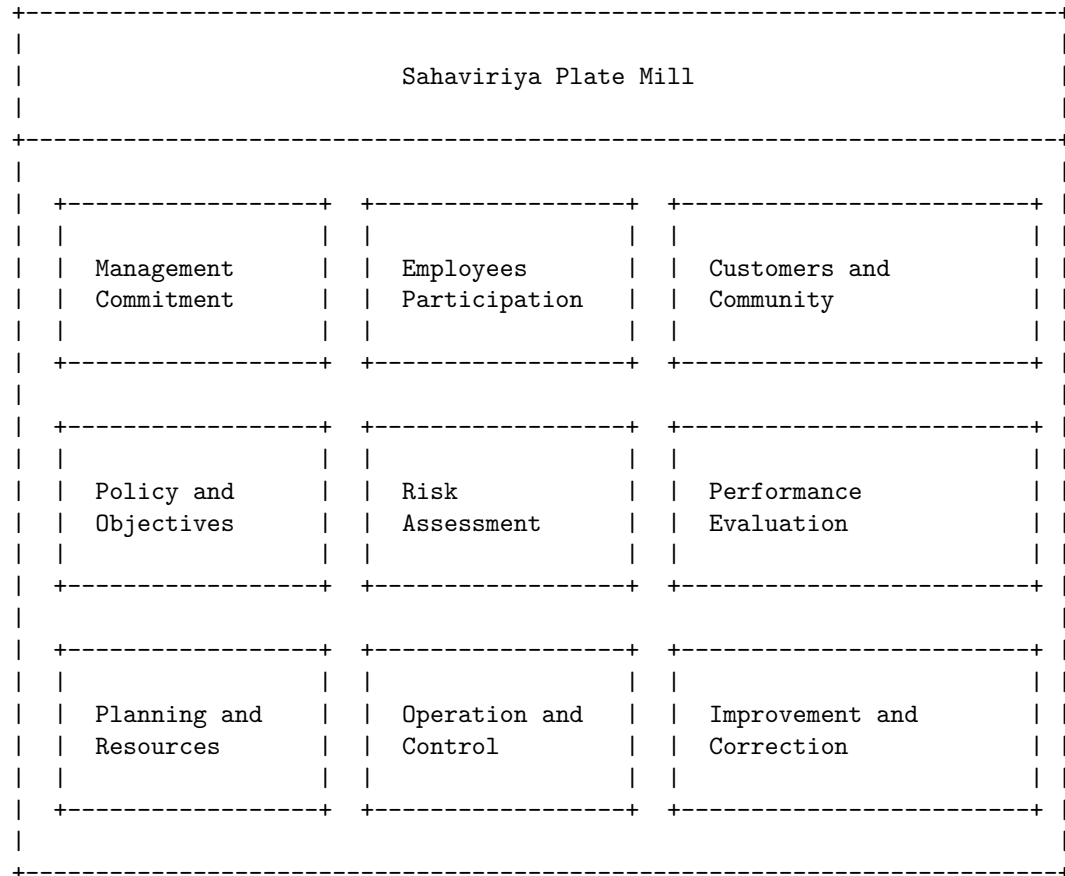
Initial conditions:

$\phi_s(x, 0) = \phi_{s0}$
 $\phi_e(x, 0) = \phi_{e0}$

Health and Safety in spm is a term that can refer to different contexts, such as Sahaviriya Plate Mill Public Company Limited, Specialized Pro-resolving Mediators, or Safe Patient Handling and Mobility. Assuming that you are interested in the first context, I will draw a detailed ASCII diagram for Health and Safety in spm as a company that produces steel plates and complies with TIS 18001

standards.

Health and Safety in spm

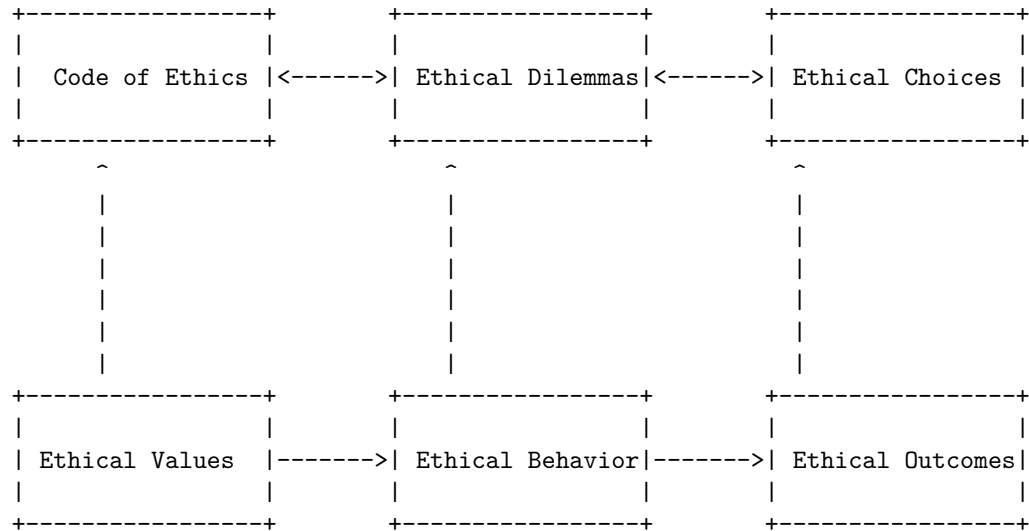


The diagram illustrates the basic architecture of a health and safety management system that spm follows to prevent any health problems and accidents at workplace as well as within the community. The system consists of four main elements: management commitment, policy and objectives, planning and resources, and operation and control. These elements are supported by three cross-cutting elements: employees participation, risk assessment, and performance evaluation. The system also includes a feedback loop for improvement and correction based on the results of the evaluation. The system aims to ensure the safety and health of the employees, customers, and community, as well as to comply with the laws and regulations, reduce costs, and enhance the social responsibility and productivity of the company.

Ethical and Professional Concerns in spm are the personal and corporate rules that govern the behavior of software project managers and their teams. They

are based on values such as honesty, respect, fairness, responsibility, integrity, and accountability. They help to ensure the quality, safety, and legality of the software products and services, as well as the trust and satisfaction of the stakeholders.

The following diagram illustrates the basic architecture of a Ethical and Professional Concerns in spm using ASCII characters:



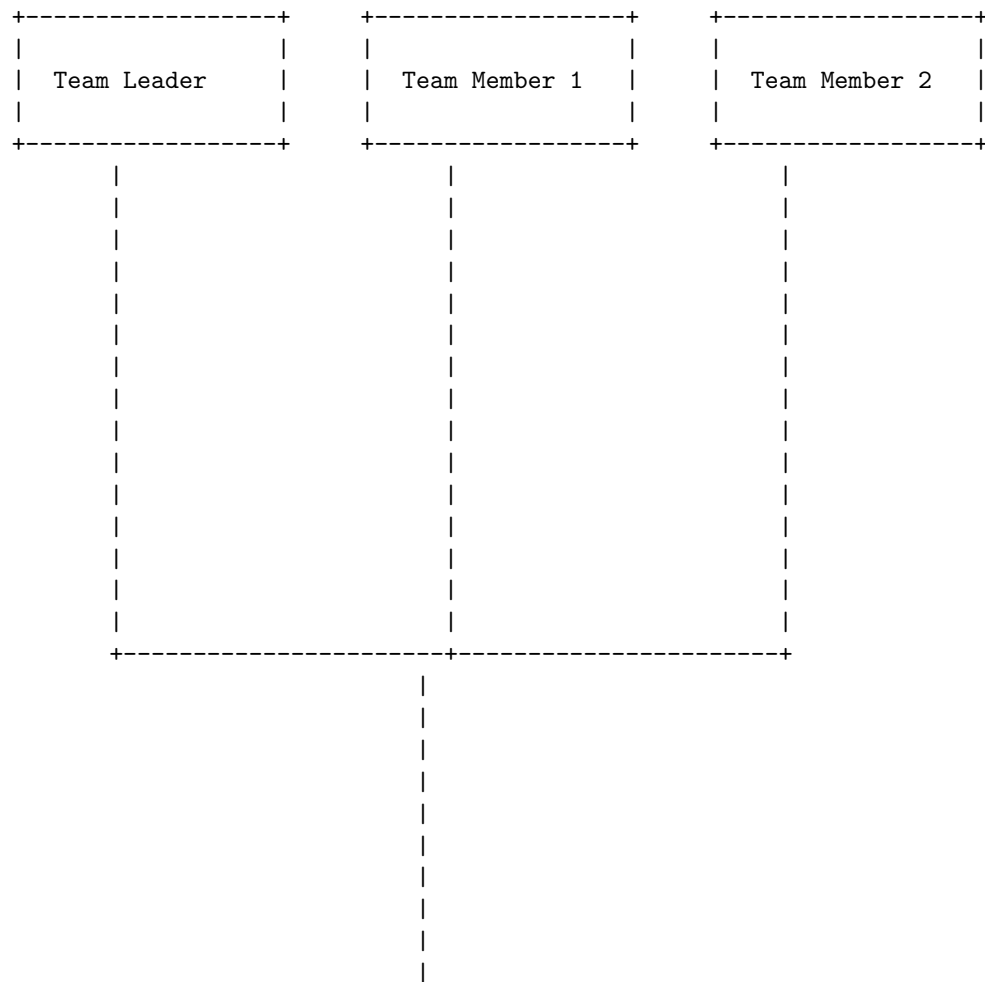
The diagram shows the following components and relationships:

- **Code of Ethics:** A set of principles and standards that guide the professional conduct of software project managers and their teams. It is usually defined by a professional organization, such as the Project Management Institute (PMI).
- **Ethical Values:** The core beliefs and ideals that shape the moral character and judgment of software project managers and their teams. They include honesty, respect, fairness, responsibility, integrity, and accountability.
- **Ethical Dilemmas:** The situations or problems that pose a conflict between two or more ethical values or principles. They may arise from the complexity, uncertainty, or ambiguity of the software project environment, or from the competing interests or expectations of the stakeholders.
- **Ethical Behavior:** The actions or decisions that software project managers and their teams take to resolve ethical dilemmas and uphold ethical values and principles. They may involve applying ethical frameworks, models, or codes, or consulting with experts, peers, or mentors.
- **Ethical Choices:** The outcomes or consequences of ethical behavior that reflect the ethical values and principles of software project managers and their teams. They may affect the quality, safety, and legality of the software products and services, as well as the trust and satisfaction of the stakeholders.

- **Ethical Outcomes:** The impacts or effects of ethical choices on the software project and its stakeholders. They may be positive or negative, intended or unintended, short-term or long-term. They may also provide feedback or learning opportunities for improving ethical behavior and choices in the future.

Working in Teams in SPM is the process of collaborating and coordinating with other members of a software project to achieve a common goal and deliver value to the stakeholders. Working in teams can improve the quality, efficiency, and creativity of the software development process, as well as the satisfaction and motivation of the team members. However, working in teams also poses some challenges, such as communication, conflict, diversity, and coordination.

The following diagram illustrates the basic architecture of a Working in Teams in SPM using ASCII characters:



|

|

|

|
|
|
|
|

Decision making in software product management (SPM) is the process of choosing among different alternatives to achieve the best outcomes for the product and the organization. SPM decisions can be classified into three levels: strategic, tactical and operational. Each level has different goals, criteria, methods and challenges. The following diagram illustrates the basic architecture of a decision making in SPM using ASCII characters:

Strategic level	Tactical level	Operational level
- Product vision - Business model - Market analysis - Stakeholder management	- Release planning - Roadmapping - Lifecycle management - Prioritization - Scoping	- Product design - Infrastructure - Deployment - Operations
- Long-term - High-level - Uncertain - Creative - Collaborative	- Medium-term - Mid-level - Complex - Analytical - Participatory	- Short-term - Low-level - Routine - Technical - Individual

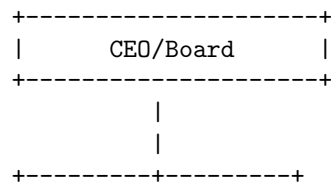
Organizational structures in SPM (Supplier Relationship Management) are the ways of dividing, coordinating, and directing the groups involved in the management of suppliers. There are different types of organizational structures that can be used in SPM, depending on the context, goals, and strategies of the organization. Some of the common types are:

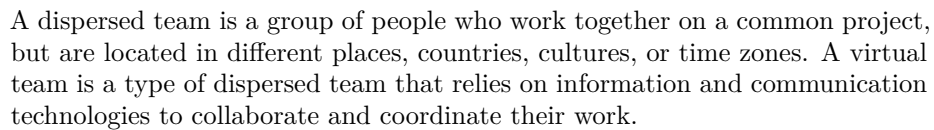
- **Functional/Hierarchical:** This is a traditional structure where the organization is divided into functional units, such as finance, marketing, production, etc. Each unit has its own manager and staff, and they are responsible for their own tasks and decisions. The advantage of this structure is that it allows for specialization and efficiency within each function. The disadvantage is that it can create silos and communication barriers between different functions, and it can limit the flexibility and innovation

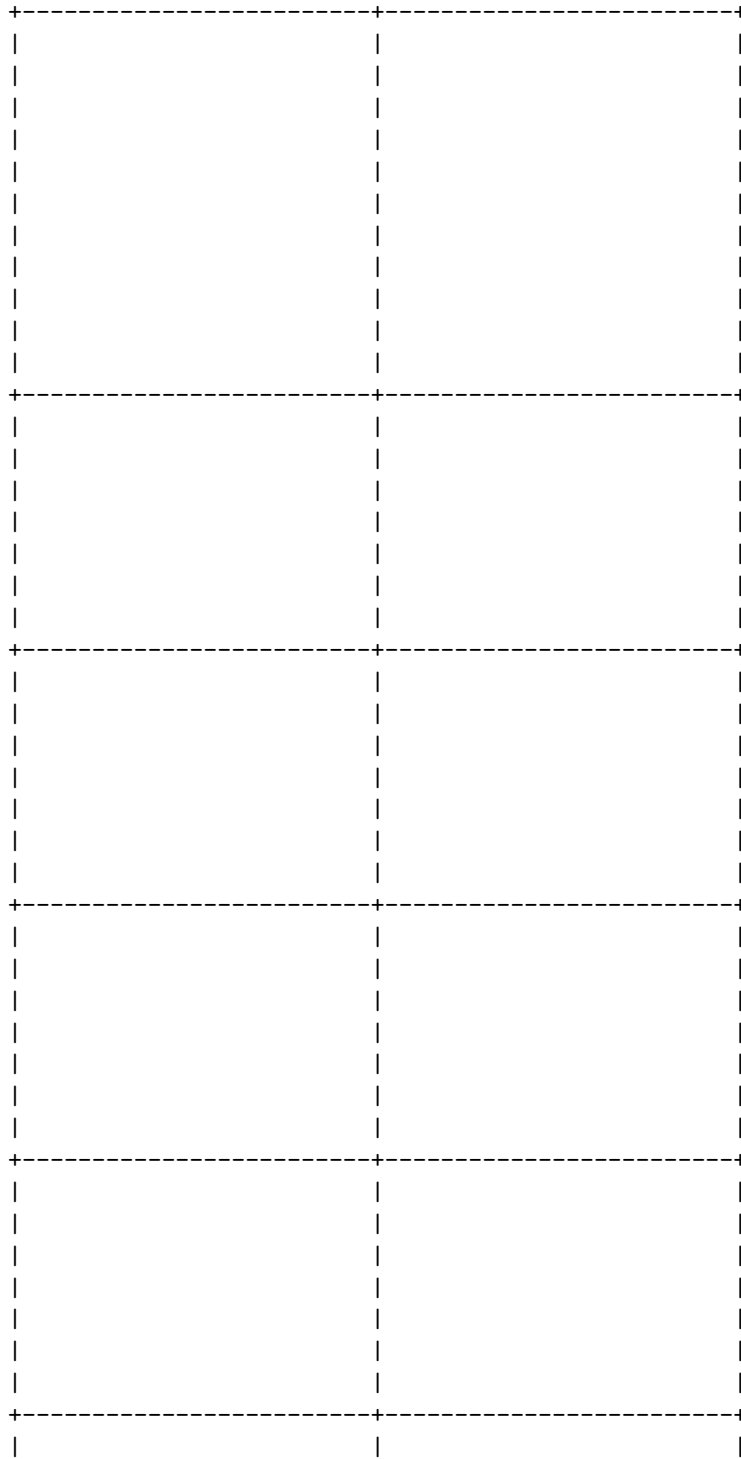
of the organization.

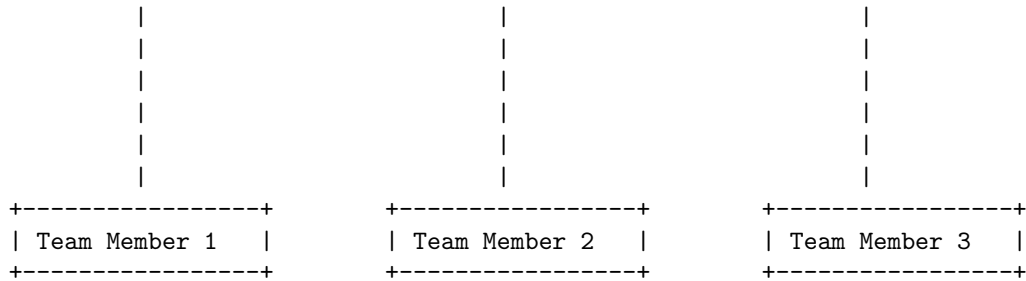
- **Matricial:** This is a hybrid structure where the organization is divided into two dimensions: functional and project. Each employee belongs to a functional unit, but they can also be assigned to a project team that crosses the functional boundaries. The advantage of this structure is that it allows for better coordination and collaboration across functions, and it can leverage the skills and expertise of different employees. The disadvantage is that it can create role ambiguity and conflict, as each employee has to report to two managers: the functional manager and the project manager.
- **RACI:** This is a tool that can be used to define the roles and responsibilities of different stakeholders in a project or a process. RACI stands for Responsible, Accountable, Consulted, and Informed. Responsible means the person who does the work or performs the task. Accountable means the person who has the authority and accountability for the outcome of the work or task. Consulted means the person who provides input or advice to the work or task. Informed means the person who needs to be notified or updated about the work or task. The advantage of this tool is that it can clarify the expectations and communication channels of different stakeholders, and it can avoid duplication or omission of work or tasks. The disadvantage is that it can be time-consuming and complex to create and maintain, and it can change frequently depending on the situation.
- **Agile:** This is a flexible and adaptive structure where the organization is composed of self-organizing and cross-functional teams that deliver value to the customer in short iterations. Each team has a product owner, a scrum master, and a development team. The product owner is the person who represents the customer and defines the product vision and priorities. The scrum master is the person who facilitates the team's work and removes any impediments. The development team is the group of people who design, build, test, and deliver the product. The advantage of this structure is that it can respond quickly and effectively to changing customer needs and market conditions, and it can foster innovation and creativity. The disadvantage is that it can require a high level of trust and collaboration among the team members, and it can challenge the traditional management and control systems of the organization.

The following diagram illustrates the basic architecture of a functional/hierarchical structure in SPM:



[illegible]





Each team member may have different roles, skills, and responsibilities in the project, depending on the project scope, objectives, and requirements. The project manager is responsible for leading, planning, monitoring, and controlling the project activities and deliverables. The project sponsor is the person or entity that provides the resources, support, and authorization for the project. The project client is the person or entity that receives the benefits and outcomes of the project.

The team members communicate and collaborate with each other and with the project manager, sponsor, and client through various channels, such as email, phone, video conferencing, chat, online platforms, etc. The team members may face challenges such as cultural differences, time zone differences, communication barriers, lack of trust, coordination issues, etc. The team members need to overcome these challenges by establishing clear goals, roles, and expectations, building trust and rapport, using appropriate communication tools and methods, managing conflicts and feedback, and celebrating successes and achievements.

Communications Genres in spm

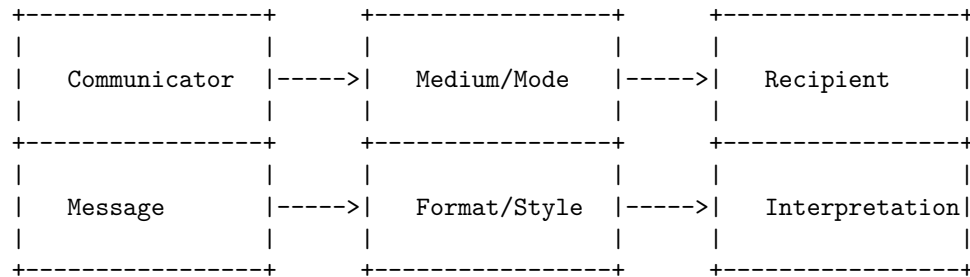
Communications genres are methods of communication that are selected and developed to deal with particular needs for project coordination. They are part of the communication plan, which documents the arrangements for communication between stakeholders. Some examples of communication genres are:

- Meetings: face-to-face or virtual gatherings of project team members or other stakeholders to discuss, plan, monitor, or review the project progress, issues, or deliverables. Meetings can be formal or informal, scheduled or ad hoc, and have different purposes and agendas.
- Reports: written or oral presentations of project information, such as status, risks, issues, quality, or performance. Reports can be periodic or on-demand, and have different formats and audiences.
- Documents: written records of project information, such as requirements, specifications, design, test cases, or user manuals. Documents can be formal or informal, and have different standards and templates.
- Emails: electronic messages sent and received by project team members or other stakeholders to communicate project information, such as updates, queries, feedback, or requests. Emails can be formal or informal, and have

different styles and tones.

- Chats: online conversations between project team members or other stakeholders to communicate project information, such as clarifications, suggestions, or opinions. Chats can be synchronous or asynchronous, and have different modes and platforms.

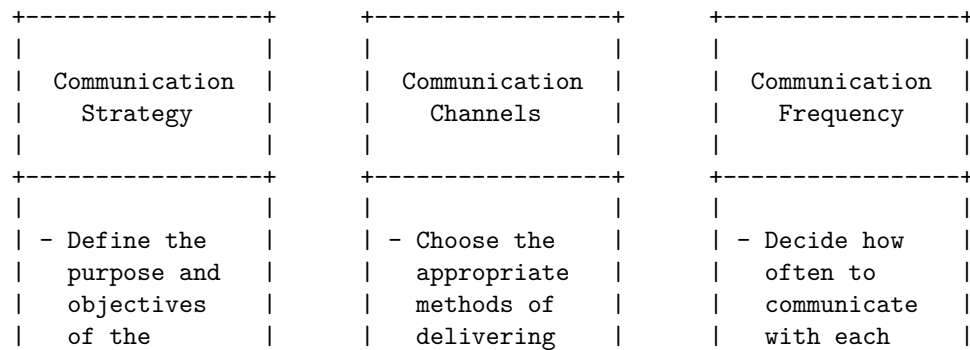
The following diagram illustrates the basic architecture of a communication genre:



The communicator is the person or entity who initiates the communication, such as a project manager, a developer, or a customer. The medium/mode is the channel or method used to transmit the communication, such as a phone, a video conference, or a chat app. The recipient is the person or entity who receives the communication, such as a project sponsor, a tester, or a user. The message is the content or information that is communicated, such as a status update, a bug report, or a feedback form. The format/style is the way the message is organized or presented, such as a table, a chart, or a bullet list. The interpretation is the way the message is understood or perceived by the recipient, which may depend on the context, the background, or the expectations.

A communication plan is an outline of how important information will be communicated to stakeholders throughout the project. It also determines who will be receiving the communication, how those people will receive it, when they'll receive it, and how often they should expect to receive that information .

The following diagram illustrates the basic architecture of a communication plan in SPM:



communication	the	stakeholder
- Identify the	communication	- Establish a
key	such as	communication
stakeholders	email,	calendar or
and their	meeting,	schedule
needs and	report, etc.	- Monitor and
expectations	- Consider the	adjust the
- Assess the	advantages	frequency
risks and	and	based on the
challenges	disadvantages	feedback and
of the	of each	changes
communication	channel	
+-----+	+-----+	+-----+

: How to create a project management communication plan - Lucidchart

How to Write an Effective Communication Plan [2023] • Asana

Project Communication Plan Examples, Template, & How To

Leadership in SPM can refer to different contexts, such as Software Project Management, Sales Performance Management, Strategic Performance Management, or Sport Management. For the purpose of this response, I will assume you are interested in Software Project Management (SPM), which is the process of planning, organizing, leading, and controlling software projects.

One possible way to draw a detailed ASCII diagram for Leadership in SPM is:

Leadership in SPM

+-----+	+-----+	+-----+
Leadership	Leadership	Leadership
Styles	Skills	Roles
+-----+	+-----+	+-----+
- Autocratic	- Communication	- Visionary
- Democratic	- Motivation	- Coach
- Laissez-faire	- Problem-solving	- Facilitator
- Transformational	- Decision-making	- Negotiator
- Servant	- Conflict management	- Mentor
+-----+	+-----+	+-----+

This diagram shows the three main aspects of leadership in SPM: styles, skills, and roles. Each aspect has some examples of possible options or categories. Of course, this is not an exhaustive list, and there may be other ways to classify or illustrate leadership in SPM. However, this diagram can serve as a starting

point for understanding the concept of leadership in SPM.